

**UNIVERSIDADE PAULISTA – UNIP
CIÊNCIA DA COMPUTAÇÃO**

JOÃO VITOR TEIXEIRA FERNANDES

COMPUTAÇÃO GRÁFICA NA PRÁTICA:
o Jogo da Cobrinha em Python com pygame-ce

**OSASCO
2026**

JOÃO VITOR TEIXEIRA FERNANDES

COMPUTAÇÃO GRÁFICA NA PRÁTICA:

o Jogo da Cobrinha em Python com pygame-ce

Ebook didático apresentado à disciplina de Computação Gráfica do curso de Ciência da Computação da Universidade Paulista – UNIP, sobre os conceitos de computação gráfica aplicados no projeto Jogo da Cobrinha.

RESUMO

Este ebook didático apresenta os conceitos fundamentais da Computação Gráfica bidimensional a partir de um projeto real: o Jogo da Cobrinha, desenvolvido em Python com a biblioteca pygame-ce para a disciplina de Computação Gráfica da Universidade Paulista (UNIP). Depois de uma introdução à área e à sua história, o texto relata a evolução do projeto, da primeira versão, feita em grupo em 2025, até a release 3.1.0, e apresenta a arquitetura do código, que separa as regras do jogo, escritas sem dependência gráfica, da interface que as desenha. Em seguida, cada capítulo explica um conceito com base em livros de referência, artigos clássicos e documentação oficial, e mostra onde e como ele foi aplicado no jogo, com trechos reais do código identificados por arquivo e linha. São tratados os gráficos raster e o buffer duplo, os sistemas de coordenadas e a conversão da grade lógica para pixels, o laço de jogo com passo de tempo fixo, o pipeline de desenho 2D e o algoritmo do pintor, a geração procedural de pixel art por funções de distância, as transformações geométricas e a escala pelo vizinho mais próximo, o modelo RGB e a quantização de cores pelo corte pela mediana, a composição alfa segundo Porter e Duff, a animação, a detecção de colisão em grade, o recorte e o envolvimento de bordas, além da interface e do som sintetizado. Um estudo de caso analisa o movimento interpolado da cobra, implementado e depois revertido por decisão de projeto. Todas as figuras, inclusive fluxogramas e capturas de tela, são geradas por scripts versionados no repositório. O ebook inclui glossário e exercícios práticos.

Palavras-chave: computação gráfica; jogos digitais; pixel art; Python; pygame.

ABSTRACT

This educational ebook presents the fundamental concepts of two-dimensional Computer Graphics through a real project: the Snake Game (Jogo da Cobrinha), developed in Python with the pygame-ce library for the Computer Graphics course at Universidade Paulista (UNIP). After an introduction to the field and its history, the text describes the evolution of the project, from the first version, built by a group of students in 2025, to release 3.1.0, and presents the code architecture, which separates the game rules, written without any graphics dependency, from the interface that draws them. Each chapter then explains one concept based on reference textbooks, classic papers and official documentation, and shows where and how it was applied in the game, with real code excerpts identified by file and line. The topics are raster graphics and double buffering, coordinate systems and the mapping from the logical grid to pixels, the game loop with a fixed time step, the 2D drawing pipeline and the painter's algorithm, procedural pixel art generated from distance functions, geometric transformations and nearest-neighbor scaling, the RGB model and median cut color quantization, alpha compositing according to Porter and Duff, animation, grid-based collision detection, clipping and edge wrapping, as well as the user interface and synthesized sound. A case study analyzes the interpolated movement of the snake, which was implemented and later reverted as a design decision. All figures, including flowcharts and screenshots, are generated by scripts kept in the repository. The ebook includes a glossary and hands-on exercises.

Keywords: computer graphics; digital games; pixel art; Python; pygame.

LISTA DE FIGURAS

Figura 1 – Linha do tempo do projeto, da V1 à release v3.1.0	17
Figura 2 – Menu principal do jogo, com título e cobra decorativa desenhados com os próprios sprites	19
Figura 3 – Arquitetura do Jogo da Cobrinha: módulos que dependem do pygame e o domínio puro	20
Figura 4 – Um quadro do jogo (nível 2): HUD, pedras, maçã, maçã dourada, power-up de câmera lenta e o texto "+1" logo depois de comer	23
Figura 5 – Coordenadas de tela do jogo: origem no canto superior esquerdo, HUD de 50 px e a conversão da célula (4, 3) para o pixel (100, 125)	28
Figura 6 – Fluxograma do laço principal: eventos, atualização, desenho e <code>flip()</code>	31
Figura 7 – Valor do acumulador ao longo de 0,6 s, com quadros a 60 FPS e passos a cada 0,125 s	33
Figura 8 – Fluxograma do passo de tempo fixo em <code>Partida.atualizar</code>	34
Figura 9 – Nível 3, "Labirinto": as pedras fazem parte do fundo pré-renderizado	40
Figura 10 – Telas do jogo e as transições entre elas; em lilás, as telas empilhadas sobre a partida	41
Figura 11 – Tela de pausa empilhada sobre a partida congelada: o véu escuro deixa o campo visível ao fundo	42
Figura 12 – Pipeline de desenho de um quadro durante a pausa: etapas (à esquerda) e camadas (à direita)	43
Figura 13 – A paleta de 16 cores do jogo (classe <code>Paleta</code> , em <code>config.py</code>), com os valores RGB	45
Figura 14 – Os dez sprites do jogo (25 × 25 px), ampliados 8 vezes pelo vizinho mais próximo	46
Figura 15 – Sprite da curva ampliado 24 vezes, com os círculos de raio 4 (vermelho) e 21 (azul) centrados no canto inferior esquerdo	48
Figura 16 – Cabeça e curva giradas por <code>pygame.transform.rotate</code> em 0°, 90°, 180° e 270°	54
Figura 17 – A maçã do jogo ampliada 16 vezes pelo vizinho mais próximo e por interpolação bilinear	55

Figura 18 – Trecho de uma captura do jogo quantizado pelo corte pela mediana para 48, 8 e 4 cores (sem pontilhado)	60
Figura 19 – Câmera lenta e pontos em dobro ativos: o véu azul sobre o campo e os efeitos escritos no HUD	63
Figura 20 – O mesmo trecho do campo sem camada, sob o véu azul ($\alpha = 40$), sob o véu da pausa ($\alpha = 170$) e sob o fade no meio da transição ($\alpha = 128$)	64
Figura 21 – Funções de animação do jogo: (a) flutuação dos itens; (b) pisca-pisca dos itens que vão sumir; (c) opacidade dos textos flutuantes	68
Figura 22 – Tela de fim de partida, empilhada sobre o campo congelado depois da animação de morte	70
Figura 23 – Posição da cabeça na lógica (degraus) e no desenho interpolado (rampas): o desenho fica até uma célula atrás	72
Figura 24 – Fluxograma de um passo da partida: mover, colidir, comer, pegar power-up, vencer ou concluir o nível	77
Figura 25 – Envolvimento de bordas numa grade de 10×6 : a cabeça em (9, 2) indo para a direita aparece em (0, 2)	80
Figura 26 – Modo Sem bordas: a cobra atravessa a borda direita e continua pela esquerda do campo	80
Figura 27 – Modo Contra o tempo: no centro do HUD, o relógio em vermelho, abaixo de 10 segundos, no lugar da barra de progresso	83
Figura 28 – Contagem antes do nível 2, empilhada sobre a partida, com o véu escuro e o nome da fase	85
Figura 29 – Dez milissegundos das ondas quadrada e triangular a 440 Hz, com envelope e amostras a 22 050 Hz	87

LISTA DE QUADROS

Quadro 1 – Problemas da V1 diagnosticados no briefing da V2	18
Quadro 2 – Prós e contras do movimento interpolado, segundo o diário de bordo	73
Quadro 3 – Conceitos de Computação Gráfica e onde aparecem no Jogo da Cobrinha	89

LISTA DE CÓDIGOS

Código 1 – Contrato comum a todas as telas (<code>Estado</code>)	20
Código 2 – Carregamento único de um sprite, convertido para o formato da tela	24
Código 3 – Montagem de um quadro em <code>Jogo._desenhar()</code> , encerrada pelo <code>flip()</code>	24
Código 4 – Direções e posições da grade, em células (<code>dominio/grade.py</code>)	26
Código 5 – Tamanho da janela derivado da grade (<code>config.py</code>)	27
Código 6 – Conversão de célula para pixel, com o deslocamento do HUD	27
Código 7 – Laço principal do jogo (<code>Jogo.executar</code>)	30
Código 8 – Passo de tempo fixo com acumulador (<code>Partida.atualizar</code>)	32
Código 9 – Velocidade da cobra: nível, aceleração e câmera lenta (<code>Partida.passos_por_segundo</code>)	35
Código 10 – Ordem de pintura da partida (<code>EstadoJogando.desenhar</code>)	38
Código 11 – Peças da cobra desenhadas da cauda para a cabeça	38
Código 12 – Fundo em xadrez pré-renderizado uma vez (<code>criar_fundo_campo</code>)	39
Código 13 – A pausa pinta só o véu e o menu (<code>EstadoPausa.desenhar</code>)	42
Código 14 – Corpo e sombra da maçã por uma função de distância (<code>comida</code>)	46
Código 15 – A curva do corpo como um quarto de anel (<code>corpo_curva</code>)	47
Código 16 – Cor de um pixel do corpo em função da profundidade (<code>cor_do_corpo</code>)	48
Código 17 – Contorno calculado pela vizinhança de 4 (<code>contornar</code>)	49
Código 18 – Ângulos de rotação por direção e o giro anti-horário dos lados	52
Código 19 – Escolha da peça e da rotação de cada segmento (<code>classificar_pecas</code>)	53
Código 20 – Ângulo da curva obtido girando os lados da curva-base (<code>angulo_da_curva</code>)	53
Código 21 – As 16 rotações preparadas uma vez (<code>Sprites.__init__</code>)	54
Código 22 – A paleta única do jogo (<code>config.py</code>)	58
Código 23 – Quadros do GIF quantizados para 48 cores pelo corte pela mediana	59
Código 24 – Véu escuro semitransparente (<code>criar_veu</code>)	64

Código 25 – Texto flutuante com alfa decrescente (<code>Efeitos.desenhar</code>)	65
Código 26 – Itens flutuando com um seno arredondado (<code>_desenhar_flutuando</code>)	67
Código 27 – Itens temporários piscando antes de sumir (<code>_desenhar_temporario</code>)	69
Código 28 – Fração do passo atual, usada para interpolar o desenho (versão removida)	71
Código 29 – Interpolação linear entre duas células vizinhas (versão removida)	71
Código 30 – Colisão com o próprio corpo e avanço da cobra (<code>Cobra.colidiria</code> e <code>Cobra.avancar</code>)	75
Código 31 – Um passo da partida: movimento, colisão e itens (<code>Partida.passo</code>)	76
Código 32 – Recorte ao campo e desenho dos dois lados da borda (versão removida com o movimento interpolado)	78
Código 33 – Teste de pertinência e envolvimento das bordas (<code>Grade.contem</code> e <code>Grade.envolver</code>)	79
Código 34 – Desenho do HUD (<code>desenhar_hud</code>)	82
Código 35 – Texto sem suavização, com cache e medida do excesso acima da fonte	84
Código 36 – Ondas quadrada e triangular, frequência das notas e tom com envelope	86

LISTA DE ABREVIATURAS E SIGLAS

Sigla	Significado
AABB	<i>Axis-Aligned Bounding Box</i> (caixa envolvente alinhada aos eixos)
ABNT	Associação Brasileira de Normas Técnicas
CG	Computação Gráfica
CI	Integração contínua (<i>Continuous Integration</i>)
dBFS	Decibéis em relação à escala cheia (<i>decibels relative to full scale</i>)
FPS	Quadros por segundo (<i>frames per second</i>)
GIF	<i>Graphics Interchange Format</i>
HUD	<i>Heads-Up Display</i> (faixa de informações do jogo)
JPEG	<i>Joint Photographic Experts Group</i> (formato de imagem)
MVC	<i>Model-View-Controller</i> (modelo, visão e controlador)
PNG	<i>Portable Network Graphics</i>
PR	<i>Pull request</i>
RGB	<i>Red, Green, Blue</i> (vermelho, verde e azul)
RGBA	RGB com canal alfa
SDL	<i>Simple DirectMedia Layer</i>
UNIP	Universidade Paulista
WAV	<i>Waveform Audio File Format</i>

SUMÁRIO

1	INTRODUÇÃO	14
1.1	OBJETIVO E PÚBLICO	14
1.2	O QUE É COMPUTAÇÃO GRÁFICA	14
1.3	BREVE HISTÓRIA	15
1.4	A FERRAMENTA: PYTHON E PYGAME-CE	16
1.5	COMO O EBOOK ESTÁ ORGANIZADO	16
2	O PROJETO: DA V1 À V3	17
2.1	A V1 (2025): O PROTÓTIPO DA DISCIPLINA	17
2.2	A V2: REESCRITA COMPLETA	18
2.3	A V3: POWER-UPS, UMA TENTATIVA REVERTIDA E O MODO CONTRA O TEMPO	19
2.4	A ARQUITETURA DO CÓDIGO	20
3	GRÁFICOS RASTER: PIXEL, RESOLUÇÃO E FRAMEBUFFER	22
3.1	IMAGENS VETORIAIS E IMAGENS RASTER	22
3.2	O QUE É UM PIXEL	22
3.3	RESOLUÇÃO E MEMÓRIA	23
3.4	SUPERFÍCIES NO PYGAME-CE	23
3.5	FRAMEBUFFER E BUFFER DUPLO	24
4	SISTEMAS DE COORDENADAS	26
4.1	COORDENADAS DE TELA	26
4.2	O MUNDO DO JOGO: UMA GRADE DE CÉLULAS	27
4.3	A TRANSFORMAÇÃO DE CÉLULA PARA PIXEL	27
4.4	CONTRAEXEMPLO: O BUG B2 DA V1	29
4.5	ESCALAS ALÉM DA JANELA: SCALED E O MOUSE	29
5	LAÇO DE JOGO E PASSO DE TEMPO FIXO	30
5.1	O LAÇO PRINCIPAL	30
5.2	DUAS VELOCIDADES DIFERENTES	31
5.3	O ACUMULADOR NA PARTIDA	32
5.4	O TETO DE PASSOS E A "ESPIRAL DA MORTE"	34

5.5	VELOCIDADE VARIÁVEL SEM MUDAR O LAÇO	35
5.6	PASSO FIXO E TESTES	35
6	PIPELINE DE DESENHO 2D	37
6.1	O ALGORITMO DO PINTOR	37
6.2	BLIT: COPIAR BLOCOS DE PIXELS	37
6.3	A ORDEM DE PINTURA DE UMA PARTIDA	37
6.4	PRÉ-RENDERIZAÇÃO: O FUNDO DESENHADO UMA VEZ	39
6.5	TELAS EMPILHADAS	40
6.6	O QUADRO COMPLETO	42
6.7	REDESENHAR TUDO A CADA QUADRO?	43
7	PIXEL ART GERADA POR CÓDIGO	44
7.1	PIXEL ART E PALETA LIMITADA	44
7.2	FORMAS DESCRITAS POR FUNÇÕES	46
7.3	O ANEL DA CURVA	47
7.4	SOMBREAMENTO POR FAIXAS	48
7.5	CONTORNO POR VIZINHANÇA	49
7.6	POR QUE GERAR ARTE POR CÓDIGO	50
8	TRANSFORMAÇÕES GEOMÉTRICAS	51
8.1	AS TRANSFORMAÇÕES NO PLANO	51
8.2	ROTACIONAR IMAGENS RASTER	51
8.3	AS PEÇAS DA COBRA	52
8.4	PRÉ-CALCULAR AS TRANSFORMAÇÕES	54
8.5	ESCALA: VIZINHO MAIS PRÓXIMO E INTERPOLAÇÃO	54
8.6	TELA CHEIA SEM BORRAR	55
9	COR: RGB, PALETA E QUANTIZAÇÃO	57
9.1	O MODELO RGB	57
9.2	PALETA INDEXADA	57
9.3	QUANTIZAÇÃO DE CORES	58
9.4	O MEDIAN CUT NO GIF DE DEMONSTRAÇÃO	59
9.5	COR E ACESSIBILIDADE	60
10	TRANSPARÊNCIA E COMPOSIÇÃO ALFA	62

10.1	O CANAL ALFA	62
10.2	O OPERADOR *OVER*	62
10.3	EXEMPLO NUMÉRICO: O VÉU AZUL DA CÂMERA LENTA	63
10.4	ALFA POR PIXEL E ALFA DA SUPERFÍCIE	64
10.4.1	Véus escuros das telas empilhadas	64
10.4.2	O fade de transição	65
10.4.3	Textos que somem	65
11	ANIMAÇÃO	67
11.1	ANIMAR EM FUNÇÃO DO TEMPO	67
11.2	FLUTUAÇÃO SENOIDAL	67
11.3	PISCAR	68
11.4	TEXTOS QUE SOBEM E SOMEM	70
11.5	ESTUDO DE CASO: O MOVIMENTO INTERPOLADO	70
11.5.1	A técnica	71
11.5.2	O custo: um passo de atraso	72
11.5.3	O que o diário de bordo registra	73
11.5.4	Lições do caso	73
12	COLISÃO, RECORTE E ENVOLVIMENTO DE BORDAS	74
12.1	MODELO E VISÃO	74
12.2	DETECÇÃO DE COLISÃO	74
12.3	UM PASSO DA PARTIDA	75
12.4	A ENTRADA TAMBÉM É MODELO: A FILA DE DIREÇÕES	78
12.5	RECORTE (*CLIPPING*)	78
12.6	ENVOLVIMENTO DE BORDAS: O MODO SEM BORDAS	79
12.7	SORTEIO DA COMIDA SEM LAÇO CEGO	81
13	INTERFACE E MULTIMÍDIA	82
13.1	INTERFACE DESENHADA POR CÓDIGO	82
13.2	TEXTO: FONTE BITMAP, CACHE E LINHA DE BASE	83
13.3	TELAS DE TRANSIÇÃO	85
13.4	SOM SINTETIZADO POR CÓDIGO	85
13.4.1	Amostragem	85

13.4.2	Ondas e envelope	86
13.4.3	Volume medido, e não suposto	87
13.4.4	Som ligado aos eventos	88
14	CONSIDERAÇÕES FINAIS	89
14.1	TRABALHOS FUTUROS	90
	REFERÊNCIAS	91
	GLOSSÁRIO	93
	APÊNDICE A – EXPERIMENTE VOCÊ MESMO	96
	CAPÍTULOS 3 E 4: RASTER E COORDENADAS	96
	CAPÍTULO 5: LAÇO DE JOGO E PASSO FIXO	96
	CAPÍTULO 6: PIPELINE DE DESENHO	96
	CAPÍTULOS 7 E 8: PIXEL ART E TRANSFORMAÇÕES	97
	CAPÍTULOS 9 E 10: COR E COMPOSIÇÃO	97
	CAPÍTULO 11: ANIMAÇÃO	97
	CAPÍTULOS 12 E 13: COLISÃO, INTERFACE E SOM	97
	APÊNDICE B – COMO ESTE EBOOK FOI GERADO	99
	ESTRUTURA DA PASTA	99
	COMO GERAR	99
	COMO FUNCIONA	99
	NORMAS SEGUIDAS	100

1 INTRODUÇÃO

Todo jogo de computador é, antes de qualquer coisa, um programa que desenha. Sessenta vezes por segundo ele decide a cor de cada um dos milhares de pixels da janela, e é dessa sequência de imagens que o jogador tira a ilusão de movimento, de profundidade e de um mundo que reage aos seus comandos. Estudar como essas imagens são produzidas é o objeto da Computação Gráfica (CG).

Este ebook usa um projeto real, o Jogo da Cobrinha desenvolvido para a disciplina de Computação Gráfica da Universidade Paulista (UNIP), como fio condutor para apresentar os conceitos fundamentais da área. Cada capítulo explica um conceito com base na literatura e mostra, em seguida, onde e como ele aparece no código do jogo, com trechos copiados diretamente do repositório e identificados por arquivo e número de linha.

1.1 OBJETIVO E PÚBLICO

O objetivo é que o leitor consiga, ao final, ligar a teoria clássica de CG (raster, sistemas de coordenadas, transformações, cor, composição alfa, animação e recorte) a decisões concretas de implementação, e entender por que cada decisão foi tomada. O público-alvo são estudantes de graduação em computação que já conhecem o básico de Python; nenhum conhecimento prévio de pygame é necessário.

O texto foi escrito para ser lido com o código aberto ao lado. Os trechos citados estão no repositório público do projeto (https://github.com/Joaofernandes-DEV/v2_jogoCobrinha), e todas as figuras deste ebook foram geradas por scripts versionados junto com ele, inclusive as capturas de tela, que são produzidas rodando o próprio jogo sem janela (Apêndice B).

1.2 O QUE É COMPUTAÇÃO GRÁFICA

Hughes *et al.* (2014) definem a Computação Gráfica, de forma ampla, como a área que trata da criação, da manipulação e da exibição de imagens por meio do computador. Na literatura nacional, Azevedo e Conci (2003) apresentam a área de forma semelhante e a relacionam a disciplinas vizinhas, como o processamento de imagens e a visão computacional. Marschner e Shirley (2021) dividem o campo em grandes áreas: modelagem (a descrição matemática das formas), renderização (a produção de imagens a partir dessa descrição), animação (a ilusão de movimento

por sequências de imagens), interação com o usuário e processamento de imagens. Um jogo 2D simples toca em quase todas elas: há um modelo (a cobra, a grade e a comida), uma etapa de renderização (o desenho de cada quadro), animação (a comida que flutua, os textos que sobem e somem) e interação (o teclado e o mouse).

Uma distinção importante, que percorre todo este ebook, é entre o **modelo** e a **imagem**. O modelo é a descrição do que existe: "a cabeça da cobra está na coluna 9, linha 2". A imagem é uma matriz de pixels coloridos. A tarefa da CG é a ponte entre os dois, e boa parte da qualidade do Jogo da Cobrinha V2 vem de manter essa ponte explícita: as regras do jogo não sabem nada de pixels, e o desenho não decide nenhuma regra (capítulo 12).

1.3 BREVE HISTÓRIA

A história da Computação Gráfica interativa costuma começar com o Sketchpad, sistema apresentado por Ivan Sutherland em 1963, no qual o usuário desenhava diretamente na tela com uma caneta óptica e o computador interpretava o desenho, mantendo restrições geométricas entre as partes (Sutherland, 1963). O Sketchpad trabalhava com uma tela vetorial: o feixe de elétrons percorria as linhas do desenho, uma a uma.

Pouco depois surgiram os algoritmos que permitiram desenhar em dispositivos discretos. O algoritmo de Bresenham (1965), criado para controlar um *plotter* digital, escolhe os pontos de uma grade que melhor aproximam um segmento de reta usando apenas soma, subtração e comparação de inteiros. Ele ainda é ensinado hoje porque resume a ideia central dos gráficos raster: o mundo contínuo precisa ser aproximado por uma grade finita.

Nas décadas de 1970 e 1980, a queda no preço da memória tornou viáveis os *frame buffers*, memórias que guardam a cor de cada pixel da tela. Com eles vieram os problemas que este ebook trata: como representar cores com poucos bits por pixel (Heckbert, 1982), como combinar imagens parcialmente transparentes (Porter; Duff, 1984) e como recortar o que fica fora da área visível (Sutherland; Hodgman, 1974). Em 1987, Lasseter mostrou como os princípios da animação tradicional se aplicavam às imagens geradas por computador, aproximando a CG do cinema (Lasseter, 1987).

Os jogos 2D, como o gênero Snake ao qual pertence o Jogo da Cobrinha, vivem exatamente nesse mundo raster: uma grade de pixels, imagens pequenas (*sprites*) copiadas para a tela a cada quadro, uma paleta de cores limitada e um laço que repete tudo dezenas de vezes por segundo.

1.4 A FERRAMENTA: PYTHON E PYGAME-CE

O jogo foi escrito em Python 3.11 com a biblioteca pygame-ce (*pygame Community Edition*), uma camada sobre a SDL (*Simple DirectMedia Layer*), biblioteca em C que dá acesso portátil a janela, vídeo, teclado, mouse e áudio. O pygame-ce oferece superfícies (`pygame.Surface`, matrizes de pixels), cópia de blocos de imagem (`blit`), transformações geométricas simples (`pygame.transform`), fontes e som (Pygame-ce Developers, c2023). Ele não esconde o modelo raster, e por isso é uma boa ferramenta didática: quase todo conceito deste ebook corresponde a uma chamada visível no código.

1.5 COMO O EBOOK ESTÁ ORGANIZADO

O capítulo 2 conta a história do projeto, da V1 feita em grupo em 2025 até a versão 3.1.0, e apresenta a arquitetura do código. Os capítulos 3 a 13 tratam, cada um, de um conceito: gráficos raster e buffer duplo (3), sistemas de coordenadas (4), laço de jogo e passo de tempo fixo (5), o pipeline de desenho 2D (6), pixel art gerada por código (7), transformações geométricas (8), cor e quantização (9), transparência e composição (10), animação, com o estudo de caso do movimento interpolado que foi revertido (11), colisão, recorte e envolvimento de bordas (12) e interface e som (13). O capítulo 14 traz as considerações finais.

Ao final estão as referências, um glossário, o Apêndice A, com exercícios do tipo "experimente você mesmo" organizados por capítulo, e o Apêndice B, que explica como regenerar este ebook, suas figuras e as capturas de tela.

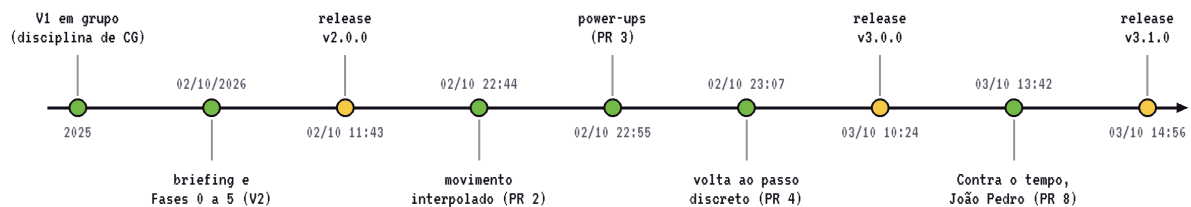
Convenções

Trechos de código aparecem como "Código *n*", com o caminho do arquivo e as linhas exatas na fonte, abaixo do quadro. Nomes de funções, variáveis e arquivos aparecem em fonte monoespaçada. Os identificadores do projeto estão em português, por decisão registrada no briefing da V2 (item E6).

2 O PROJETO: DA V1 À V3

Antes de entrar nos conceitos, vale conhecer o objeto de estudo e a sua história. O Jogo da Cobrinha passou por três versões, e várias decisões técnicas discutidas nos próximos capítulos só fazem sentido quando se sabe qual problema da versão anterior elas resolveram. As fontes deste capítulo são os documentos do próprio repositório: o `briefing_v2.md`, com o diagnóstico da V1 e o plano da V2, e o `LOG.md`, o diário de bordo com todas as mudanças, em ordem cronológica, com data, horário e motivo. A Figura 1 resume essa trajetória.

Figura 1 – Linha do tempo do projeto, da V1 à release v3.1.0



Fonte: elaborada pelo autor (2026), com base no `LOG.md` e nos *pull requests* do repositório.

2.1 A V1 (2025): O PROTÓTIPO DA DISCIPLINA

A primeira versão foi feita em grupo, em 2025, para a disciplina de Computação Gráfica, por João Vitor Fernandes, João Pedro Sinhorini Silva, Vitor Barsotti de Souza e Alex Barbosa Lourenço. Era um Snake funcional em Pygame, com três níveis de velocidade, telas de transição desenhadas no Canva e sprites direcionais para a cobra, tudo num único arquivo de 348 linhas.

O briefing da V2 reconhece que a V1 cumpria o objetivo acadêmico, mas registra problemas que reaparecerão neste ebook como contraexemplos. O Quadro 1 resume os principais.

Quadro 1 – Problemas da V1 diagnosticados no briefing da V2

Código	Problema	Conceito relacionado (capítulo)
B1, B5	<code>pygame.quit()</code> chamado e o desenho continuando depois; o jogo fechava com erro	laço de jogo (5)
B2	Cobra nascendo em <code>x = 400</code> (não múltiplo de 30) e comida em múltiplos de 30: grade desalinhada	sistemas de coordenadas (4)
B3	Direção validada contra a direção atual: duas teclas rápidas causavam meia-volta e morte	modelo e entrada (12)
B7	Laço infinito ao sortear a comida com o campo cheio	colisão em grade (12)
—	Sprites de até 840×462 px esticados para 30×30 e fundo de 319×161 ampliado para 800×600 , borrado	escala e pixel art (7, 8)
—	Comida em JPG, sem transparência: um quadrado de fundo aparecia em volta	canal alfa (10)
—	Lógica e desenho na mesma taxa (8 a 16 quadros por segundo) e menus sem limite de quadros, com 100% de CPU	passo de tempo fixo (5)

Fonte: adaptado de `briefing_v2.md`, seções 2.1 a 2.5.

2.2 A V2: REESCRITA COMPLETA

A V2 foi uma reescrita do zero feita por João Vitor Fernandes, planejada no briefing em seis fases (0 a 5), cada uma terminando com o jogo jogável. O diário de bordo registra todas elas em 2 de outubro de 2026, da fundação (08:58) à publicação da release v2.0.0 (11:43). As decisões centrais, numeradas no briefing, foram:

- **Grade lógica em células (E2):** a cobra e a comida guardam (`coluna`, `linha`), e a conversão para pixels acontece só no desenho. A janela tem 800×600 px: 32×22 células de 25 px mais uma faixa de 50 px para o HUD (capítulo 4).
- **Máquina de estados com um único laço (E1):** cada tela é um objeto com `tratar_evento`, `atualizar` e `desenhar`, e `pygame.quit()` aparece em um único lugar (capítulos 5 e 6).
- **Lógica em passo fixo e desenho a 60 FPS (D1):** a cobra anda num ritmo próprio, independente da taxa de quadros, e continua se movendo célula a célula, fiel ao clássico (capítulo 5).
- **Domínio sem pygame (E4):** as regras ficam em `src/cobrinha/dominio/` e podem ser testadas sem abrir janela (capítulo 12).
- **Pixel art em 25×25 px nativos, paleta de até 16 cores, peças giradas em múltiplos de 90° e tela cheia com `pygame.SCALED`** (seção 8.3 do briefing; capítulos 7, 8 e 9).

A V2 também trouxe som sintetizado por código, recordes salvos em disco, três níveis com obstáculos, a fruta dourada, os modos Clássico e Sem bordas, testes automatizados (190 na v2.0.0), integração contínua no GitHub Actions e um executável para Windows. A Figura 2 mostra o menu principal da versão atual.

Figura 2 – Menu principal do jogo, com título e cobra decorativa desenhados com os próprios sprites



Fonte: captura gerada pelo autor (2026) com docs/ebook/capturas.py.

2.3 A V3: POWER-UPS, UMA TENTATIVA REVERTIDA E O MODO CONTRA O TEMPO

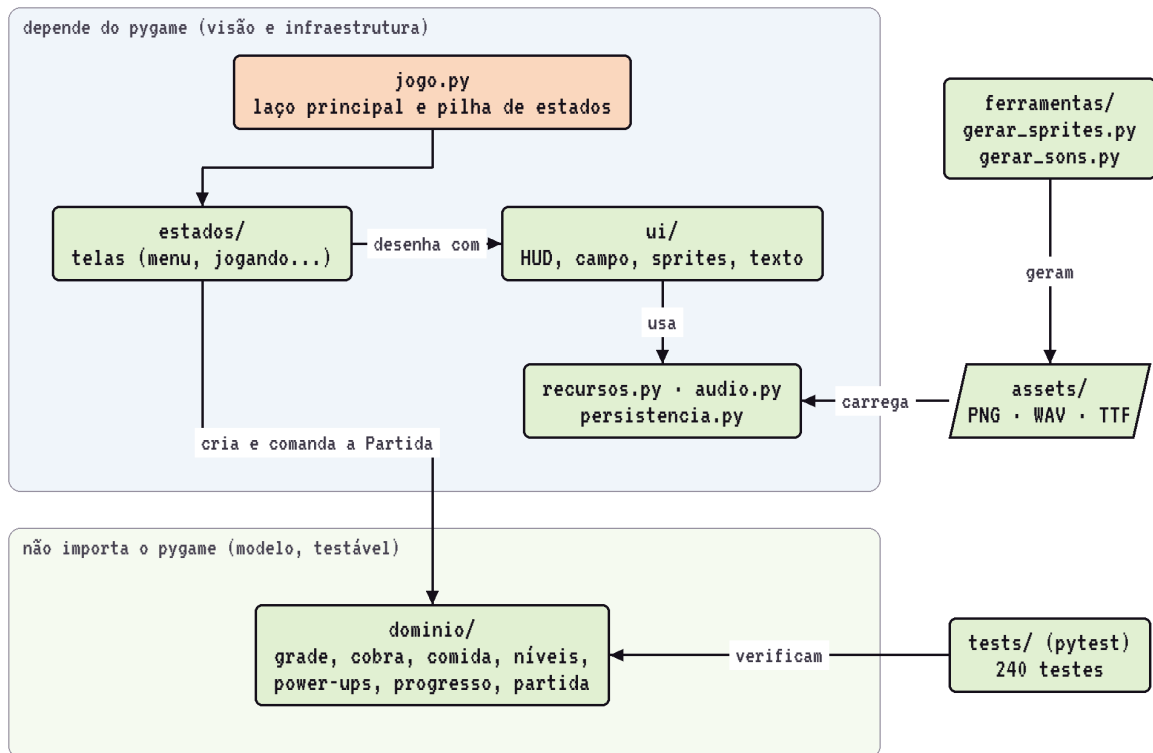
A V3 começou na mesma noite. O primeiro item do backlog foi o **movimento interpolado**: em vez de pular de célula em célula, cada segmento da cobra deslizaria entre as células ao longo do intervalo de cada passo (*pull request* 2). Onze minutos depois entraram os **power-ups** (câmera lenta, pontos em dobro e encolher; *pull request* 3). Às 23:07, depois de jogar com o movimento suave, o autor decidiu voltar ao movimento discreto, e a remoção foi feita como um *pull request* próprio (4), para que o histórico registrasse a tentativa e a volta. Esse episódio é o estudo de caso do capítulo 11.

A release v3.0.0 saiu em 3 de outubro, e a v3.1.0, no mesmo dia, trouxe o modo **Contra o tempo**, implementado pelo colaborador João Pedro Sinhorini Silva: um relógio de 60 s em que cada maçã devolve segundos (*pull request* 8). O patch chegou sobre a v2.0.0 e foi integrado à V3 resolvendo conflitos com os power-ups, como descreve o diário de bordo.

2.4 A ARQUITETURA DO CÓDIGO

A Figura 3 mostra como o código está dividido. A separação mais importante é horizontal: tudo o que está em `dominio/` é Python puro, sem nenhum `import pygame`; o resto (telas, desenho, carregamento de recursos e áudio) depende da biblioteca.

Figura 3 – Arquitetura do Jogo da Cobrinha: módulos que dependem do pygame e o domínio puro



Fonte: elaborada pelo autor (2026), com base na estrutura de `src/cobrinha/`.

Cada tela do jogo (menu, contagem, partida, pausa, fim de partida, recordes, opções, créditos) é uma subclasse de `Estado`, cujo contrato aparece no Código 1. O laço principal chama esses três métodos a cada quadro, e é essa regularidade que permite tratar todas as telas da mesma forma no pipeline de desenho (capítulo 6).

Código 1 – Contrato comum a todas as telas (`Estado`)

```

14 class Estado(ABC):
15     """Uma tela do jogo. O loop principal chama, a cada quadro, os três métodos
    abaixo."""
16
17     def __init__(self, jogo: Jogo) -> None:
18         self.jogo = jogo
19
20     @abstractmethod
21     def tratar_evento(self, evento: pygame.Event) -> None:
22         """Reage a um evento do pygame (teclado, mouse etc.)."""
23
24     def atualizar(self, dt: float) -> None: # noqa: B027 - opcional nas subclasses
  
```

```
25     """Avança a lógica da tela. `dt` é o tempo do quadro, em segundos."""
26
27     @abstractmethod
28     def desenhar(self, superficie: pygame.Surface) -> None:
29         """Desenha a tela inteira na superfície recebida."""
```

Fonte: `src/cobrinha/estados/base.py`, linhas 14–29.

O padrão é conhecido como *State* na literatura de projeto orientado a objetos (Gamma *et al.*, 1994) e é recomendado por Nystrom (2014) para organizar as telas e os modos de um jogo. No Jogo da Cobrinha, ele substituiu os laços `while True` aninhados da V1, em que cada tela tinha o próprio laço e devolvia valores de tipos misturados para indicar a próxima tela, origem dos erros B1 e B5.

Para ir além

O arquivo `src/cobrinha/estados/navegacao.py` concentra todas as trocas de tela e traz no topo um diagrama em texto do fluxo. Compare esse diagrama com a Figura 10, no capítulo 6.

3 GRÁFICOS RASTER: PIXEL, RESOLUÇÃO E FRAMEBUFFER

Quase toda tela moderna é um dispositivo **raster**: a imagem é uma matriz retangular de pontos, e cada ponto recebe uma cor. Marschner e Shirley (2021) observam que, mesmo quando a cena é descrita por formas contínuas (retas, círculos, polígonos), ela precisa ser convertida em uma imagem raster para ser exibida. Este capítulo apresenta os conceitos básicos desse modelo (pixel, resolução, framebuffer e buffer duplo) e mostra como o pygame-ce os expõe.

3.1 IMAGENS VETORIAIS E IMAGENS RASTER

Uma imagem **vetorial** descreve formas: "um círculo de raio 8 centrado em (12,5; 14,5)". Uma imagem **raster** descreve amostras: "o pixel (12, 14) é vermelho". A primeira pode ser ampliada sem perda, porque basta recalcular a forma; a segunda tem uma resolução fixa. O Sketchpad, citado no capítulo 1, desenhava vetores diretamente na tela (Sutherland, 1963); os monitores atuais e a biblioteca pygame-ce trabalham com rasters.

O Jogo da Cobrinha mostra as duas ideias convivendo. Os sprites são gerados a partir de descrições geométricas (o corpo da maçã é "todo pixel cuja distância ao centro é menor que 8,5"), mas o que se grava no arquivo PNG e o que se desenha na tela é sempre uma matriz de pixels (capítulo 7).

3.2 O QUE É UM PIXEL

O termo *pixel* vem de *picture element*. É comum imaginá-lo como um pequeno quadrado colorido, mas Smith (1995) argumenta que essa imagem é enganosa: um pixel é uma **amostra pontual** de uma imagem contínua, e o "quadrado" que vemos é só o resultado de como o monitor reconstrói essa amostra. A distinção importa quando se amplia ou se gira uma imagem: para decidir a cor de um novo pixel, é preciso decidir como reconstruir a imagem entre as amostras (capítulo 8).

No caso da pixel art, porém, a convenção do "quadrado" é exatamente o efeito desejado: cada pixel do sprite deve aparecer como um bloco nítido de cor uniforme. É por isso que o projeto toma cuidado para nunca suavizar os sprites, como se verá nos capítulos 7 e 8.

3.3 RESOLUÇÃO E MEMÓRIA

A **resolução** de uma imagem raster é o número de colunas e linhas de pixels. A janela do jogo tem $800 \times 600 = 480\,000$ pixels. Se cada pixel ocupar 4 bytes (vermelho, verde, azul e alfa, com 8 bits cada), um quadro inteiro ocupa 1 920 000 bytes, cerca de 1,9 MB. Esse cálculo simples explica um problema histórico da área: até o fim da década de 1970, memória para guardar uma imagem inteira era cara, e por isso os primeiros *frame buffers* usavam poucos bits por pixel e uma tabela de cores (Heckbert, 1982), assunto do capítulo 9.

A Figura 4 é um quadro real do jogo, exatamente como ele fica na memória antes de ser mostrado: 800×600 pixels, com a faixa do HUD no topo e o campo de 32×22 células abaixo.

Figura 4 – Um quadro do jogo (nível 2): HUD, pedras, maçã, maçã dourada, power-up de câmera lenta e o texto "+1" logo depois de comer



Fonte: captura gerada pelo autor (2026) com docs/ebook/capturas.py.

3.4 SUPERFÍCIES NO PYGAME-CE

No pygame-ce, toda imagem raster é uma `pygame.Surface`: uma matriz de pixels com resolução e formato de pixel fixos (Pygame-ce Developers, c2023). A janela também é uma superfície, devolvida por `pygame.display.set_mode`. Desenhar, no pygame-ce, significa alterar os pixels de uma superfície: preencher um retângulo (`fill`), mudar um pixel (`set_at`) ou copiar outra superfície para dentro dela (`blit`).

O **formato de pixel** é a forma como cada cor é codificada na memória (quantos bits por canal, em que ordem, e se há canal alfa). Copiar uma superfície

para outra de formato diferente obriga a biblioteca a converter cada pixel no caminho. Por isso a documentação recomenda converter as imagens carregadas para o formato da tela: `convert()` para imagens opacas e `convert_alpha()` para imagens com transparência por pixel (Pygame-ce Developers, c2023). O Código 2 mostra como o jogo faz isso uma única vez por sprite, guardando o resultado em cache.

Código 2 – Carregamento único de um sprite, convertido para o formato da tela

```
36 @cache
37 def imagem(nome: str) -> pygame.Surface:
38     """Sprite PNG de `assets/imagens/`, convertido para o formato da tela (exige janela
    aberta)."""
39     return pygame.image.load(PASTA_IMAGENS / f"{nome}.png").convert_alpha()
```

Fonte: `src/cobrinha/recursos.py`, linhas 36–39.

O decorador `@cache` garante que cada PNG seja lido do disco e convertido uma só vez (item D6 do briefing). O comentário da função lembra uma restrição importante: a conversão precisa de uma janela aberta, porque o formato de destino é o da tela.

3.5 FRAMEBUFFER E BUFFER DUPLO

O **framebuffer** é a região de memória que guarda a imagem a ser exibida. Se o programa desenhasse diretamente na imagem que o monitor está lendo, o usuário poderia ver o quadro pela metade: o fundo já pintado, mas a cobra ainda não. O efeito, conhecido como *tearing* ou cintilação, é evitado com o **buffer duplo**: o programa desenha num buffer de trás (*back buffer*), invisível, e só quando o quadro está completo os papéis se invertem e ele passa a ser exibido. Nystrom (2014) descreve essa técnica como o padrão *Double Buffer*, cuja ideia central é fazer uma sequência de operações parecer instantânea para quem observa.

No pygame-ce, essa troca é feita por `pygame.display.flip()`, que atualiza a tela inteira com o conteúdo da superfície da janela; no modo OpenGL, ela faz a troca de buffers (Pygame-ce Developers, c2023). O Código 3 mostra o método do jogo que monta cada quadro e termina com o `flip()`.

Código 3 – Montagem de um quadro em `Jogo._desenhar()`, encerrada pelo `flip()`

```
119 def _desenhar(self) -> None:
120     self.tela.fill(Paleta.PRETO)
121     for estado in self.pilha:
122         estado.desenhar(self.tela)
123     if self.opacidade_fade > 0:
124         self._camada_fade.set_alpha(round(255 * self.opacidade_fade))
```

```
125         self.tela.blit(self._camada_fade, (0, 0))
126         pygame.display.flip()
```

Fonte: `src/cobrinha/jogo.py`, linhas 119–126.

Entre o `fill` da linha 120 e o `flip` da linha 126, nada do que é desenhado aparece para o jogador. Só o quadro completo (fundo, HUD, cobra, telas empilhadas e o fade) é mostrado, de uma vez. O capítulo 6 detalha a ordem dessas etapas.

Para ir além

A janela é criada com a opção `pygame.SCALED` (`src/cobrinha/jogo.py`, linha 46). Nesse modo, o código-fonte do `pygame-ce 2.5.8` cria um renderizador da SDL associado à janela e define a resolução lógica com `SDL_RenderSetLogicalSize` (arquivo `src_c/display.c`, linhas 1648 a 1705; Pygame-ce Developers, 2026). O jogo continua desenhando numa superfície de 800×600 , e a SDL apresenta essa imagem ampliada na janela real. O capítulo 8 mostra por que essa ampliação não borra a pixel art.

4 SISTEMAS DE COORDENADAS

Todo desenho começa com uma pergunta: *onde?* Em CG, essa pergunta tem sempre mais de uma resposta, porque o mesmo objeto pode ser descrito em vários sistemas de coordenadas. Hearn, Baker e Carithers (2011) distinguem as **coordenadas de mundo**, em que a cena é modelada, das **coordenadas de dispositivo**, os pixels da tela, e chamam de transformação de visualização (*window-to-viewport*) a função que leva de um sistema ao outro. Este capítulo mostra que o Jogo da Cobrinha tem exatamente essa estrutura, em escala pequena: um mundo de células e uma tela de pixels.

4.1 COORDENADAS DE TELA

Nas bibliotecas raster, incluindo o pygame-ce e a SDL, a origem (0, 0) fica no **canto superior esquerdo** da janela: o eixo x cresce para a direita e o eixo y cresce **para baixo**. Isso é o contrário da convenção da matemática escolar, e herda a forma como os monitores de tubo varriam a tela, linha a linha, de cima para baixo. Um pixel na posição (x, y) está x colunas à direita e y linhas abaixo da origem.

Essa inversão do eixo y aparece no próprio domínio do jogo. O Código 4 mostra que "para cima" é o deslocamento (0, -1): subir na tela é diminuir a linha.

Código 4 – Direções e posições da grade, em células (dominio/grade.py)

```

13 class Direcao(Enum):
14     """Direções de movimento; o valor é o deslocamento (colunas, linhas)."""
15
16     CIMA = (0, -1)
17     BAIXO = (0, 1)
18     ESQUERDA = (-1, 0)
19     DIREITA = (1, 0)
20
21     @property
22     def oposta(self) -> Direcao:
23         dx, dy = self.value
24         return Direcao((-dx, -dy))
25
26
27 class Posicao(NamedTuple):
28     """Uma célula da grade: (0, 0) é o canto superior esquerdo do campo."""
29
30     coluna: int
31     linha: int
32
33     def vizinha(self, direcao: Direcao) -> Posicao:
34         dx, dy = direcao.value
35         return Posicao(self.coluna + dx, self.linha + dy)

```

Fonte: src/cobrinha/dominio/grade.py, linhas 13–35.

4.2 O MUNDO DO JOGO: UMA GRADE DE CÉLULAS

A decisão E2 do briefing estabelece que a lógica do jogo nunca trabalha em pixels. A cobra, a comida, as pedras e os power-ups têm posições do tipo `Posicao(coluna, linha)`, com coluna entre 0 e 31 e linha entre 0 e 21. Nesse mundo, mover a cobra é somar 1 a uma coordenada inteira, e saber se ela bateu é comparar tuplas. Não há frações de pixel, arredondamentos nem a possibilidade de dois objetos ficarem "meio desalinhados".

As medidas da janela, por sua vez, derivam da grade, e não o contrário. O Código 5 mostra como a configuração define primeiro a célula, as colunas e as linhas, e calcula a largura e a altura da janela a partir delas.

Código 5 – Tamanho da janela derivado da grade (`config.py`)

```
13 # Grade
14 TAMANHO_CELULA: Final = 25
15 COLUNAS: Final = 32
16 LINHAS: Final = 22
17
18 # Janela
19 ALTURA_HUD: Final = 50
20 LARGURA_JANELA: Final = COLUNAS * TAMANHO_CELULA
21 ALTURA_JANELA: Final = ALTURA_HUD + LINHAS * TAMANHO_CELULA
```

Fonte: `src/cobrinha/config.py`, linhas 13–21.

Com 32 colunas de 25 px, a largura é $32 \times 25 = 800$ px; com 22 linhas e o HUD de 50 px, a altura é $50 + 22 \times 25 = 600$ px. Mudar o tamanho da célula ou o número de linhas recalcula a janela inteira de forma consistente.

4.3 A TRANSFORMAÇÃO DE CÉLULA PARA PIXEL

A ponte entre o mundo e a tela é a função `celula_para_pixel`, no Código 6. Ela devolve o canto superior esquerdo da célula na janela.

Código 6 – Conversão de célula para pixel, com o deslocamento do HUD

```
15 def celula_para_pixel(posicao: Posicao) -> tuple[int, int]:
16     """Canto superior esquerdo da célula na janela, já descontando o HUD."""
17     return posicao.coluna * TAMANHO_CELULA, ALTURA_HUD + posicao.linha * TAMANHO_CELULA
```

Fonte: `src/cobrinha/ui/campo.py`, linhas 15–17.

Escrita como fórmula, sendo $T = 25$ px o tamanho da célula e $h = 50$ px a altura do HUD:

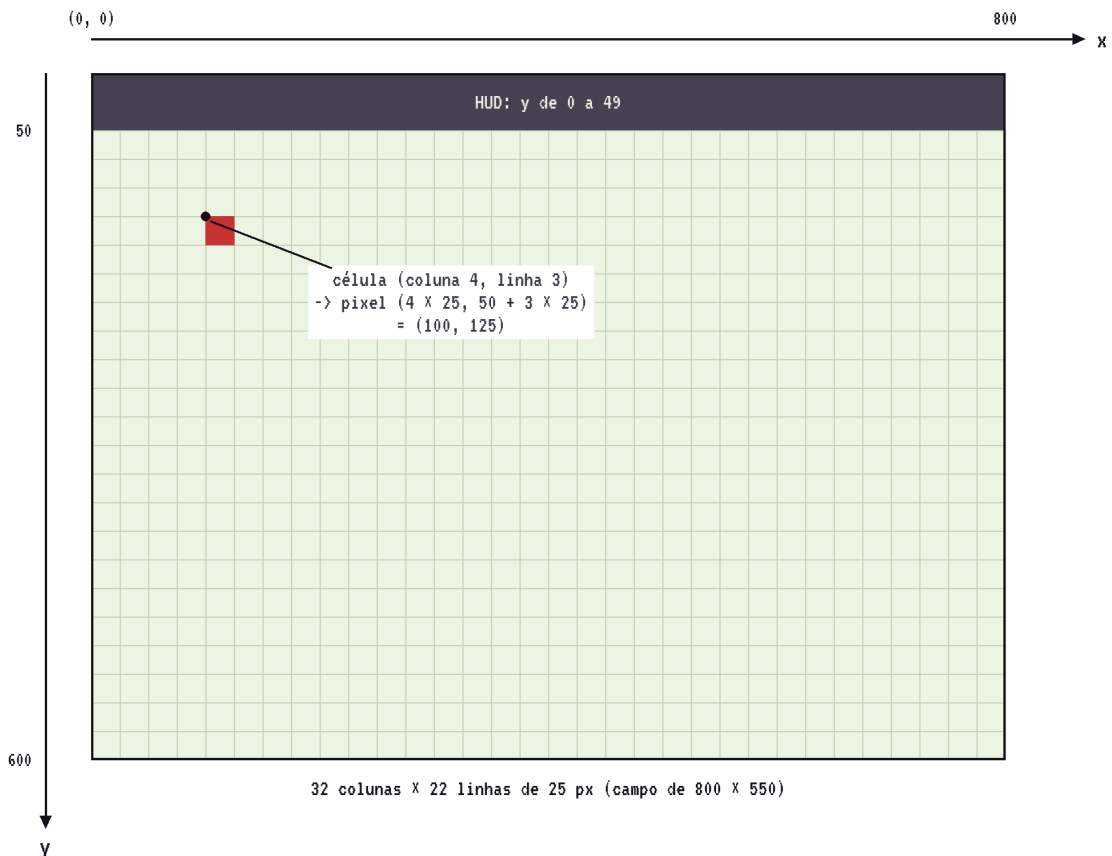
$$x = T \cdot \text{coluna} \text{ e } y = h + T \cdot \text{linha}$$

É uma **transformação afim**: uma escala uniforme por T seguida de uma translação de $(0, h)$. Em coordenadas homogêneas, que os livros de CG usam para escrever translações também como produto de matrizes (Hughes *et al.*, 2014; Marschner; Shirley, 2021), ela fica:

$$\begin{array}{|c|c|} \hline x & \\ \hline y & \\ \hline 1 & \\ \hline \end{array} = \begin{array}{|ccc|} \hline 25 & 0 & 0 \\ \hline 0 & 25 & 50 \\ \hline 0 & 0 & 1 \\ \hline \end{array} \cdot \begin{array}{|c|} \hline \text{coluna} \\ \hline \text{linha} \\ \hline 1 \\ \hline \end{array}$$

A Figura 5 ilustra a conversão para a célula (4, 3), que vai parar no pixel (100, 125).

Figura 5 – Coordenadas de tela do jogo: origem no canto superior esquerdo, HUD de 50 px e a conversão da célula (4, 3) para o pixel (100, 125)



Fonte: elaborada pelo autor (2026) com `docs/ebook/figuras.py`.

Como toda a conversão está em uma função, mover o HUD para baixo, ou acrescentar uma borda em volta do campo, exige mudar uma única linha. É também por isso que o fundo do campo é desenhado numa superfície própria, com origem no topo do campo (sem o HUD), e copiado para a janela na posição (0, 50) (capítulo 6).

4.4 CONTRAEXEMPLO: O BUG B2 DA V1

A V1 não tinha essa separação. A cobra nascia em `x = 400` px, que não é múltiplo do tamanho de célula de 30 px usado na época ($400 = 13 \times 30 + 10$), enquanto a comida nascia em múltiplos de 30. Além disso, $800 / 30 \approx 26,67$: a janela não comportava um número inteiro de células, e sobrava uma faixa de 20 px na borda direita. O resultado, registrado no briefing como B2, era uma cobra 10 px deslocada da comida, que "comia" por sobreposição parcial, e uma colisão inconsistente com a borda direita.

O problema não era de aritmética, mas de modelo: a posição da cobra existia em pixels e podia assumir qualquer valor. Na V2, a grade é a única fonte de verdade, e um desalinhamento desse tipo se torna impossível por construção.

4.5 ESCALAS ALÉM DA JANELA: SCALED E O MOUSE

Há ainda um terceiro sistema de coordenadas no jogo, o da janela física. Com `pygame.SCALED`, o jogo desenha sempre numa superfície de 800×600 , mas a janela pode estar ampliada ou em tela cheia. A documentação do `pygame-ce` descreve exatamente esse comportamento: o jogo "pensa" que a janela tem o tamanho original, mas ela pode ser maior, e os eventos de mouse são convertidos automaticamente (Pygame-ce Developers, c2023). Por isso os menus que respondem ao mouse (`ui/menu.py`) comparam a posição do clique com os retângulos dos itens sem nenhuma conta extra: a SDL já desfez a escala.

Resumo do capítulo

O jogo tem três sistemas de coordenadas: células (o mundo lógico, usado pelas regras), pixels lógicos de 800×600 (onde tudo é desenhado) e pixels físicos da janela (cuidados pela SDL). As conversões entre eles ficam em um único lugar cada: `celula_para_pixel` e a opção `SCALED`.

5 LAÇO DE JOGO E PASSO DE TEMPO FIXO

Um programa comum espera o usuário: lê um comando, executa e espera o próximo. Um jogo não pode esperar. Mesmo sem nenhuma tecla pressionada, a cobra continua andando, a comida continua flutuando e o relógio do modo Contra o tempo continua correndo. Nystrom (2014) descreve a solução com o padrão *Game Loop*: um laço que, a cada volta, processa a entrada do usuário sem bloquear, atualiza o estado do jogo e desenha o resultado, controlando a passagem do tempo. Este capítulo mostra esse laço no Jogo da Cobrinha e a decisão D1 do briefing: separar o ritmo da lógica do ritmo do desenho.

5.1 O LAÇO PRINCIPAL

O laço do jogo está no método `Jogo.executar`, no Código 7. É o único `while` que roda durante toda a execução: todas as telas são atendidas por ele, por meio da pilha de estados (capítulo 6).

Código 7 – Laço principal do jogo (`Jogo.executar`)

```

92     def executar(self) -> None:
93         self.rodando = True
94         try:
95             while self.rodando and self.estado_atual is not None:
96                 dt = self.relogio.tick(FPS) / 1000
97                 self._processar_eventos()
98                 if not self.rodando:
99                     break
100                self.estado_atual.atualizar(dt)
101                self.opacidade_fade = max(0.0, self.opacidade_fade - dt / DURACAO_FADE)
102                self._desenhar()
103            finally:
104                # Único ponto de encerramento do pygame (corrige B1 e B5 da V1).
105                pygame.quit()

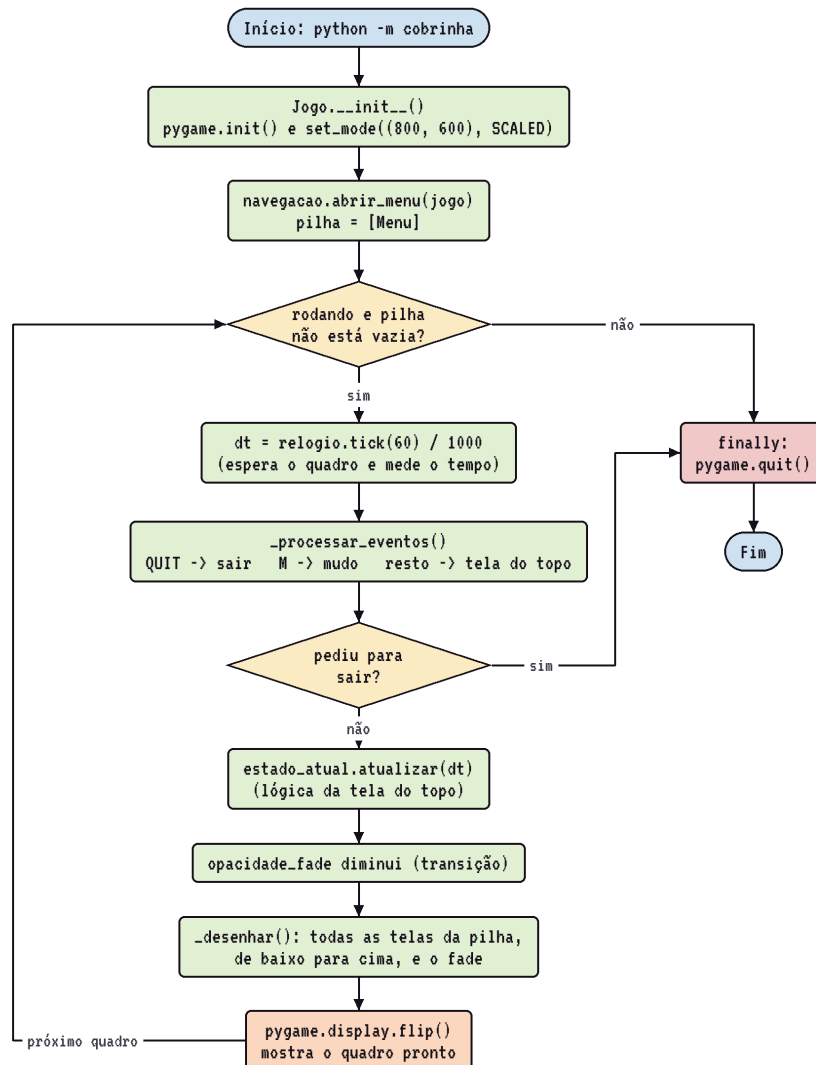
```

Fonte: `src/cobrinha/jogo.py`, linhas 92–105.

Cada volta tem quatro etapas, que a Figura 6 mostra como fluxograma:

1. **Medir o tempo:** `relogio.tick(FPS)` espera o necessário para não passar de 60 quadros por segundo e devolve quantos milissegundos se passaram desde a volta anterior. Dividido por 1000, esse valor vira `dt`, em segundos.
2. **Processar eventos:** teclas, mouse, perda de foco e o pedido de fechar a janela são lidos da fila da SDL e entregues à tela do topo da pilha.
3. **Atualizar:** a tela do topo avança sua lógica em `dt` segundos.
4. **Desenhar e exibir:** o quadro é montado no buffer de trás e mostrado com `flip()` (capítulo 3).

Figura 6 – Fluxograma do laço principal: eventos, atualização, desenho e flip()



Fonte: elaborada pelo autor (2026) com base em `src/cobrinha/jogo.py`.

Dois detalhes do código corrigem diretamente bugs da V1. O bloco `try ... finally` garante que `pygame.quit()` seja chamado em um único ponto, depois que o laço termina, por qualquer motivo (B1 e B5). E o `clock.tick` vale para todas as telas, inclusive os menus, que na V1 rodavam em laços próprios sem limite de quadros e ocupavam 100% de um núcleo da CPU enquanto o jogador lia as opções (item D2 do briefing).

5.2 DUAS VELOCIDADES DIFERENTES

Na V1, o desenho e a lógica rodavam no mesmo ritmo: entre 8 e 16 quadros por segundo, conforme o nível. A animação ficava "aos trancos" e o teclado só era lido a cada passo da cobra. A tentação oposta seria mover a cobra uma célula por quadro a 60 FPS, mas aí a velocidade do jogo dependeria do computador: numa

máquina lenta, que só conseguisse 30 quadros por segundo, a cobra andaria na metade da velocidade.

Fiedler (2004) discute as alternativas para avançar uma simulação no tempo e defende a seguinte: o programa desenha tão rápido quanto puder (ou quanto o limite de quadros permitir), mas a simulação avança em **passos de tamanho fixo**. Um **acumulador** guarda o tempo real que ainda não foi "consumido" pela simulação; a cada quadro, o tempo do quadro entra no acumulador, e enquanto houver tempo suficiente para um passo inteiro, um passo é executado e seu tempo é descontado.

5.3 O ACUMULADOR NA PARTIDA

O Código 8 é a implementação dessa ideia no domínio do jogo, sem nenhuma dependência do pygame.

Código 8 – Passo de tempo fixo com acumulador (Partida.atualizar)

```

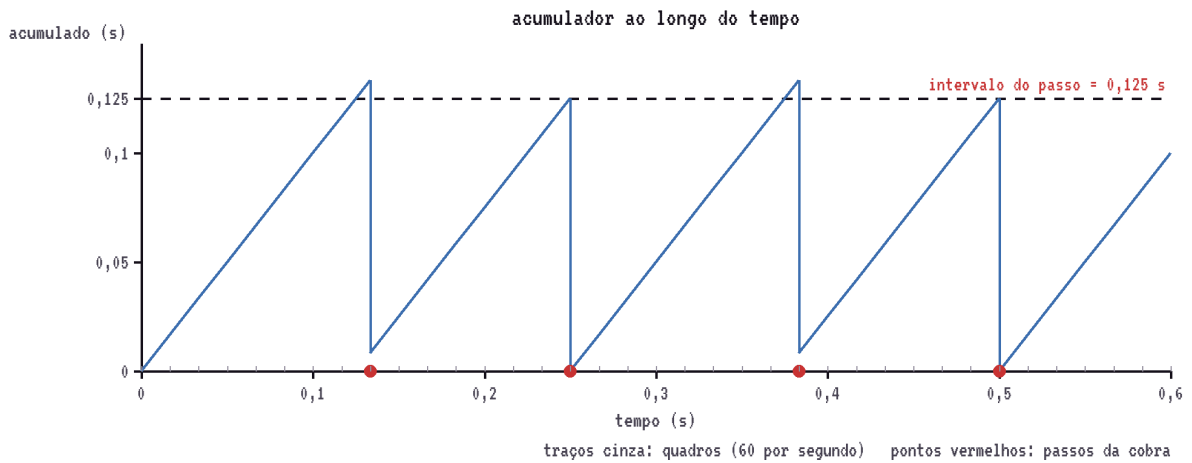
141 def atualizar(self, dt: float) -> list[Evento]:
142     """Avança o relógio da partida em `dt` segundos e executa os passos devidos.
143
144     A lógica anda em passo fixo, independente do FPS da tela (D1).
145     """
146     if not self.em_andamento:
147         return []
148     self._envelhecer_fruta_dourada(dt)
149     self._envelhecer_power_ups(dt)
150     limite = self.intervalo_passo * MAX_PASSOS_POR_QUADRO
151     self._tempo_acumulado = min(self._tempo_acumulado + dt, limite)
152     eventos = []
153     while self.em_andamento and self._tempo_acumulado >= self.intervalo_passo:
154         self._tempo_acumulado -= self.intervalo_passo
155         eventos.append(self.passo())
156     # O relógio nunca anda mais rápido que a cobra: um travamento não rouba
    segundos.
157     if self.em_andamento and self._descontar_tempo(min(dt, limite)):
158         eventos.append(Evento.TEMPO_ESGOTADO)
159     return eventos

```

Fonte: src/cobrinha/dominio/partida.py, linhas 141–159.

O intervalo de um passo é o inverso da velocidade da cobra, `1 / passos_por_segundo`. No nível 1, que começa com 8 passos por segundo, o intervalo é de 0,125 s, enquanto um quadro a 60 FPS dura cerca de 0,0167 s. Por isso, na maioria dos quadros o laço `while` da linha 153 não executa nenhum passo: o acumulador só cresce. A cada sete ou oito quadros, ele passa do intervalo, um passo é executado, e o excesso fica guardado para o próximo. A Figura 7 simula esse comportamento com os valores reais do jogo e mostra o "dente de serra" característico do acumulador.

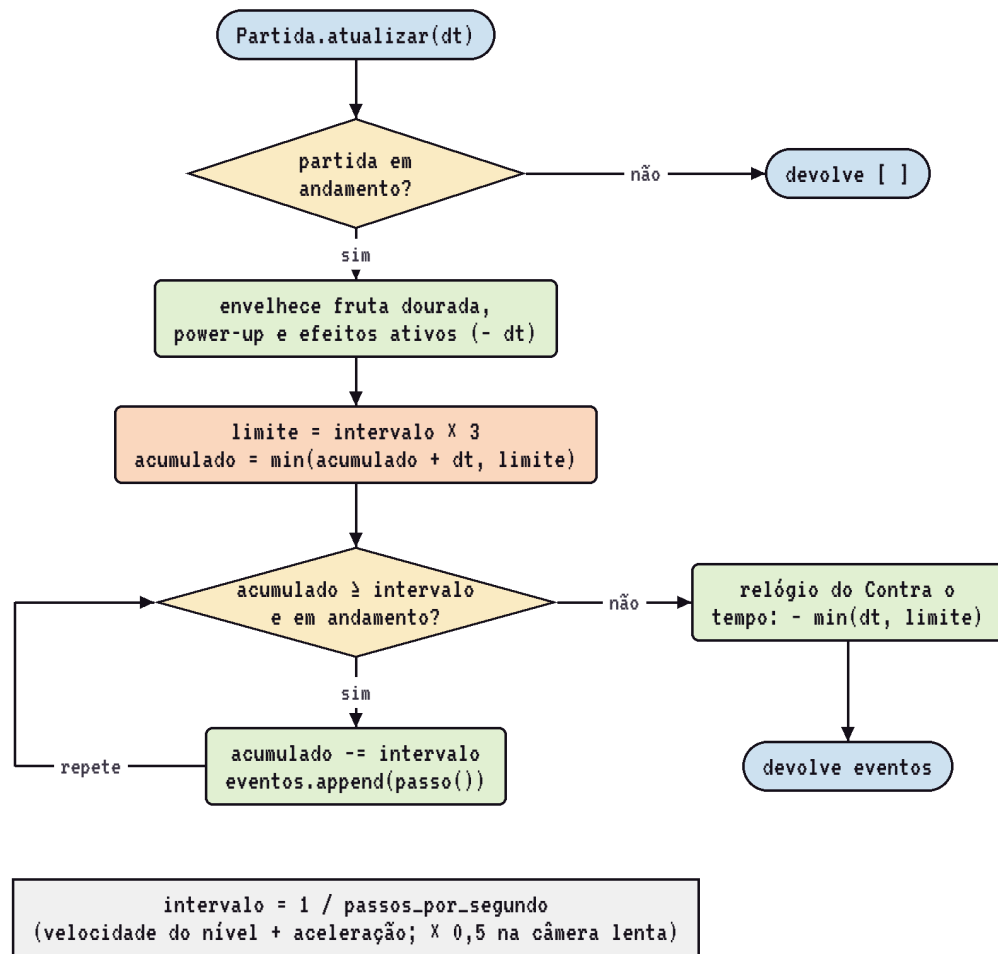
Figura 7 – Valor do acumulador ao longo de 0,6 s, com quadros a 60 FPS e passos a cada 0,125 s



Fonte: elaborada pelo autor (2026) com `docs/ebook/figuras.py`, simulando a regra do Código 8.

Note que o acumulador às vezes passa um pouco do intervalo antes de o passo acontecer (por exemplo, $8 \times 1/60 \approx 0,133$ s), porque o tempo só é verificado uma vez por quadro. O excesso não se perde: ele fica no acumulador e antecipa o passo seguinte. Em média, a cobra anda exatamente 8 células por segundo, qualquer que seja a taxa de quadros.

A Figura 8 mostra o método inteiro como fluxograma, incluindo o envelhecimento dos itens temporários (fruta dourada e power-ups) e o relógio do modo Contra o tempo.

Figura 8 – Fluxograma do passo de tempo fixo em `Partida.atualizar`

Fonte: elaborada pelo autor (2026) com base em `src/cobrinha/dominio/partida.py`.

5.4 O TETO DE PASSOS E A "ESPIRAL DA MORTE"

Fiedler (2004) aponta um risco da técnica: se a simulação de um passo demorar mais do que o tempo que ele representa, o programa nunca alcança o relógio, e cada quadro precisa executar mais passos que o anterior, numa "espiral da morte". No Jogo da Cobrinha, um passo é barato, mas existe um caso parecido: quando o usuário arrasta a janela no Windows, o laço fica parado por um tempo, e o `dt` seguinte chega enorme. Sem proteção, a cobra executaria dezenas de passos de uma vez e apareceria "teleportada", provavelmente já morta.

As linhas 150 e 151 resolvem isso limitando o acumulador a três intervalos (`MAX_PASSOS_POR_QUADRO`, em `config.py`). O jogo prefere perder tempo de simulação a mostrar um salto que o jogador não teve chance de ver. O relógio do modo Contra o tempo segue a mesma regra (linhas 156 a 158): ele desconta no máximo o mesmo limite, de modo que um travamento não "rouba" segundos do jogador. A regra está descrita no diário de bordo, na entrada do modo Contra o tempo.

5.5 VELOCIDADE VARIÁVEL SEM MUDAR O LAÇO

Como a lógica só conhece o intervalo do passo, mudar a velocidade da cobra é só mudar `passos_por_segundo`. O Código 9 mostra que três regras de jogabilidade (velocidade do nível, aceleração a cada comida e o power-up de câmera lenta) se reduzem a uma conta.

Código 9 – Velocidade da cobra: nível, aceleração e câmera lenta (`Partida.passos_por_segundo`)

```

112     @property
113     def passos_por_segundo(self) -> float:
114         """Velocidade atual: a do nível mais a aceleração por comida (J9).
115
116         Com a câmera lenta ativa (V3), a velocidade cai pelo `FATOR_CAMERA_LENTA`.
117         """
118         velocidade = self.nivel.passos_por_segundo + self.nivel.aceleracao_por_comida *
119         (
120             self.comidas_no_nivel
121         )
122         if TipoPowerUp.CAMERA_LENTA in self.efeitos_ativos:
123             velocidade *= FATOR_CAMERA_LENTA
124         return velocidade

```

Fonte: `src/cobrinha/dominio/partida.py`, linhas 112–123.

A câmera lenta, por exemplo, multiplica a velocidade por 0,5: o intervalo dobra e o acumulador leva o dobro de quadros para completar um passo. Nada no laço principal, no desenho ou na taxa de quadros precisa saber que o jogo está em câmera lenta.

5.6 PASSO FIXO E TESTES

Um efeito colateral valioso do passo fixo é a **reprodutibilidade**. Fiedler (2004) observa que ele permite repetir exatamente a mesma simulação de uma execução para outra. Os testes do projeto exploram isso: em `tests/test_partida.py`, partidas inteiras são simuladas chamando `atualizar` com valores de `dt` escolhidos pelo teste, sem janela e sem esperar o tempo real passar, e o gerador de números aleatórios recebe uma semente fixa. O GIF de demonstração do README e as capturas deste ebook são gerados da mesma forma: o jogo roda "de verdade", mas o tempo é controlado pelo script.

Para ir além

O artigo de Fiedler (2004) termina com uma etapa que o Jogo da Cobrinha não usa: **interpolar** o desenho entre os dois últimos estados da simulação, usando a fração do passo que já passou. O capítulo 11 conta como essa técnica foi implementada na V3 e por que ela foi removida.

6 PIPELINE DE DESENHO 2D

Em CG 3D, *pipeline* é a sequência de etapas que leva uma cena (vértices, triângulos, luzes e câmera) até os pixels da tela (Angel; Shreiner, 2020). Em um jogo 2D raster, o pipeline é bem mais curto, mas a ideia é a mesma: uma sequência fixa de operações, sempre na mesma ordem, que transforma o estado do jogo em um quadro. Este capítulo descreve essa sequência no Jogo da Cobrinha e os dois conceitos que a sustentam: o **algoritmo do pintor** e a operação de **blit**.

6.1 O ALGORITMO DO PINTOR

Quando vários objetos ocupam o mesmo lugar da tela, é preciso decidir qual deles aparece. A solução mais antiga é a do pintor de quadros: pintar primeiro o que está mais longe e por último o que está mais perto, deixando que cada camada cubra as anteriores. Hearn, Baker e Carithers (2011) apresentam esse método, chamado de **algoritmo do pintor** (*painter's algorithm*), entre as técnicas de determinação de superfícies visíveis: os objetos são ordenados por profundidade e desenhados do fundo para a frente.

Em 3D, ordenar por profundidade pode ser difícil, porque dois polígonos podem se cruzar ou se sobrepor de forma cíclica. Em um jogo 2D, a "profundidade" é uma decisão de projeto, e a ordem é fixa: o fundo vem antes dos objetos, que vêm antes da interface. O que importa é que essa ordem seja explícita e respeitada em todo quadro.

6.2 BLIT: COPIAR BLOCOS DE PIXELS

A operação básica do pipeline 2D é o *blit* (de *bit block transfer*): copiar um retângulo de pixels de uma superfície para outra, numa posição dada. No pygame, `destino.blit(origem, posicao)` faz essa cópia e, quando a origem tem canal alfa, combina as cores com a do destino em vez de simplesmente substituí-las (Pygame Developers, c2023). O capítulo 10 trata dessa combinação; aqui basta saber que cada `blit` é uma camada de tinta sobre o quadro.

6.3 A ORDEM DE PINTURA DE UMA PARTIDA

O Código 10 é o método que desenha a tela de jogo. Lido de cima para baixo, ele é a própria lista de camadas, da mais funda para a mais próxima.

Código 10 – Ordem de pintura da partida (EstadoJogando.desenhar)

```

183     desenhar_hud(superficie, dados)
184     superficie.blit(self.fundo_campo, (0, ALTURA_HUD))
185     lento = TipoPowerUp.CAMERA_LENTA in partida.efeitos_ativos
186     if lento and self.jogo.opcoes.efeitos_visuais:
187         superficie.blit(self.veu_lento, (0, ALTURA_HUD))
188     if partida.comida is not None:
189         self.sprites.desenhar_comida(superficie, partida.comida, self.tempo)
190     if partida.fruta_dourada is not None:
191         self.sprites.desenhar_fruta_dourada(
192             superficie,
193             partida.fruta_dourada.posicao,
194             self.tempo,
195             partida.fruta_dourada.tempo_restante,
196         )
197     if partida.power_up is not None:
198         self.sprites.desenhar_power_up(superficie, partida.power_up, self.tempo)
199     if self.cobra_visivel:
200         self.sprites.desenhar_cobra(superficie, partida.cobra)
201     self.efeitos.desenhar(superficie)
202     if self.jogo.debug:
203         self._desenhar_depuracao(superficie)

```

Fonte: `src/cobrinha/estados/jogando.py`, linhas 183–203.

1. **HUD** (linha 183): a faixa de 50 px no topo, com pontos, nível, recorde e efeitos ativos.
2. **Fundo do campo** (linha 184): o xadrez de grama e as pedras, copiados de uma só vez.
3. **Véu azul** (linhas 185 a 187): só durante a câmera lenta, com transparência (capítulo 10).
4. **Itens** (linhas 188 a 198): maçã, maçã dourada e power-up, cada um com sua animação (capítulo 11).
5. **Cobra** (linhas 199 e 200), que pisca durante a animação de morte.
6. **Textos flutuantes** (linha 201): o "+1" ou "X2!" que sobe e some.
7. **Informações de depuração** (linhas 202 e 203), só com a opção `--debug`.

A ordem tem consequências visíveis. A cobra é desenhada depois dos itens, então, se a maçã "flutuar" 1 px para dentro da célula vizinha, a cobra a cobre. Os textos flutuantes vêm depois da cobra, para nunca ficarem escondidos sob o corpo, como se vê no "+1" da Figura 4, no capítulo 3.

A própria cobra também segue o algoritmo do pintor internamente. O Código 11 percorre as peças da cauda para a cabeça, com `reversed`, para que a cabeça seja sempre a última a ser pintada.

Código 11 – Peças da cobra desenhadas da cauda para a cabeça

```

130     def desenhar_cobra(self, superficie: pygame.Surface, cobra: Cobra) -> None:
131         # Da cauda para a cabeça, para a cabeça ficar sempre por cima.
132         for peca in reversed(classificar_pecas(cobra.segmentos, cobra.direcao)):

```

```
133             superficie.blit(self._pecas[peca.tipo, peca.angulo],
celula_para_pixel(peca.posicao))
```

Fonte: `src/cobrinha/ui/pecas.py`, linhas 130–133.

Com o movimento célula a célula, as peças quase não se sobrepõem, porque cada uma ocupa a própria célula. A regra fazia diferença na versão com movimento interpolado (capítulo 11), em que segmentos vizinhos deslizavam e se cobriam parcialmente, e continua garantindo que a cabeça, a peça mais importante para o jogador, nunca fique escondida.

6.4 PRÉ-RENDERIZAÇÃO: O FUNDO DESENHADO UMA VEZ

O fundo do campo tem 704 células (32×22), cada uma pintada com uma de duas cores, e as pedras de cada nível são dezenas de sprites. Nada disso muda durante o nível. Em vez de repetir esse trabalho a cada quadro, o jogo o faz uma única vez, ao criar a tela da partida, numa superfície própria (Código 12 e linhas 79 e 80 do `jogando.py`). Depois, cada quadro só precisa de um `blit` do fundo pronto.

Código 12 – Fundo em xadrez pré-renderizado uma vez (`criar_fundo_campo`)

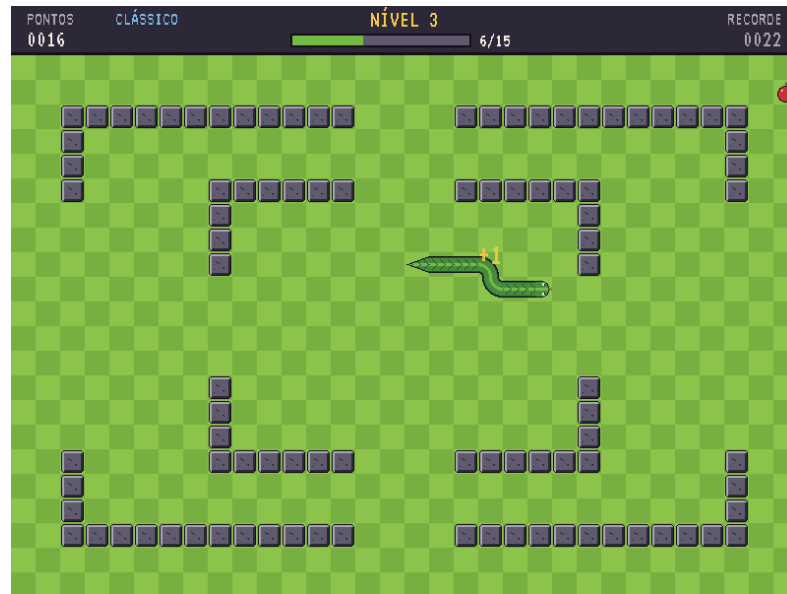
```
20 def criar_fundo_campo() -> pygame.Surface:
21     """Xadrez de grama em dois tons, desenhado uma vez e reaproveitado a cada quadro."""
22     superficie = pygame.Surface((COLUNAS * TAMANHO_CELULA, LINHAS * TAMANHO_CELULA))
23     for posicao in GRADE_PADRAO.todas:
24         clara = (posicao.coluna + posicao.linha) % 2 == 0
25         cor = Paleta.GRAMA_CLARA if clara else Paleta.GRAMA_ESCURA
26         x, y = posicao.coluna * TAMANHO_CELULA, posicao.linha * TAMANHO_CELULA
27         superficie.fill(cor, (x, y, TAMANHO_CELULA, TAMANHO_CELULA))
28     return superficie
```

Fonte: `src/cobrinha/ui/campo.py`, linhas 20–28.

Essa técnica, conhecida como **pré-renderização** ou *cache* de camada, troca memória por tempo: o fundo ocupa 800×550 pixels a mais na memória, mas o custo de desenhá-lo por quadro passa de centenas de operações para uma. Na V1, o fundo era uma imagem de 319×161 px ampliada a cada quadro para 800×600 , o que além de custar caro a deixava borrada (briefing, seção 2.3).

A economia cresce com o número de pedras. No nível 3, "Labirinto" (Figura 9), as paredes formam corredores com dezenas de blocos, todos pintados uma única vez no início do nível.

Figura 9 – Nível 3, "Labirinto": as pedras fazem parte do fundo pré-renderizado



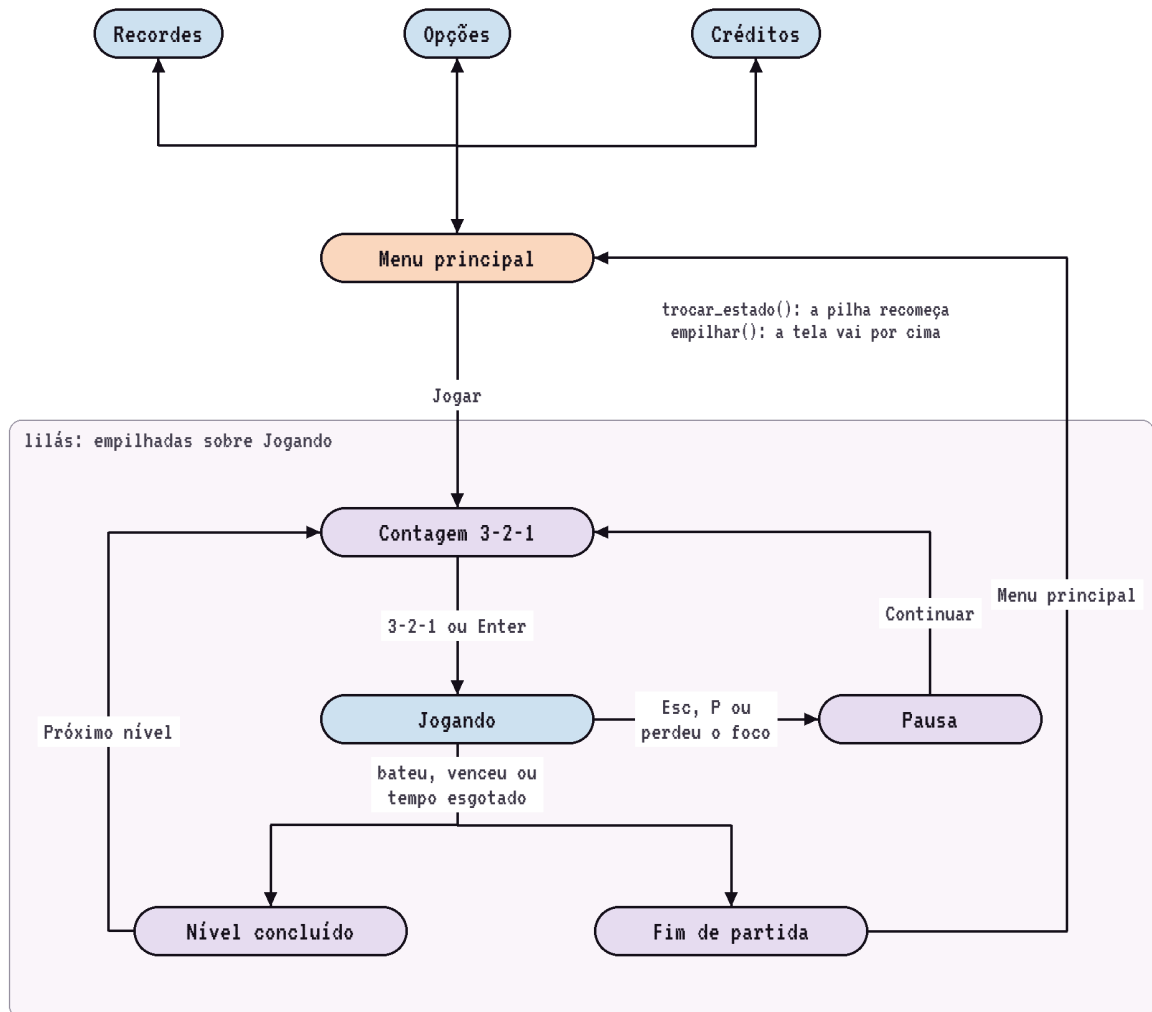
Fonte: captura gerada pelo autor (2026) com docs/ebook/capturas.py.

O xadrez em si tem uma função de interface, registrada no item I5 do briefing: as duas cores alternadas por célula, definidas pela paridade de `coluna + linha`, mostram a grade ao jogador sem precisar de linhas.

6.5 TELAS EMPILHADAS

O pipeline não termina na tela de jogo. A pausa, a contagem 3-2-1, a tela de nível concluído e a de fim de partida são desenhadas **por cima** da partida congelada. Isso é possível porque o jogo guarda as telas numa pilha: `empilhar` põe uma tela nova no topo, `desempilhar` a retira, e `trocar_estado` esvazia a pilha e começa de novo (`jogo.py`, linhas 74 a 86). A Figura 10 mostra todas as transições entre telas.

Figura 10 – Telas do jogo e as transições entre elas; em lilás, as telas empilhadas sobre a partida



Fonte: elaborada pelo autor (2026) com base em `src/cobrinha/estados/navegacao.py`.

Na hora de desenhar, `Jogo._desenhar` (Código 3, no capítulo 3) percorre a pilha **de baixo para cima**. Só a tela do topo recebe eventos e é atualizada, mas todas são pintadas. É o algoritmo do pintor aplicado a telas inteiras: a partida é pintada primeiro e a pausa, por último, como mostra a Figura 11.

Figura 11 – Tela de pausa empilhada sobre a partida congelada: o véu escuro deixa o campo visível ao fundo



Fonte: captura gerada pelo autor (2026) com docs/ebook/capturas.py.

O Código 13 mostra que a tela de pausa não precisa redesenhar nada da partida: ela só pinta um véu escuro semitransparente sobre o que já está no quadro e escreve o menu.

Código 13 – A pausa pinta só o véu e o menu (EstadoPausa.desenhar)

```

67     def desenhar(self, superficie: pygame.Surface) -> None:
68         superficie.blit(self.veu, (0, 0))
69         y = desenhar_linhas(
70             superficie,
71             [{"PAUSADO", TamanhoFonte.ENORME, Paleta.BRANCO}],
72             ALTURA_JANELA // 2 - 170,
73         )
74         self.menu.desenhar(superficie, topo=y + 20)
75         desenhar_linhas(
76             superficie,
77             [{"ESC ou P: continuar", "M: som", TamanhoFonte.PEQUENO,
78             Paleta.CINZA_CLARO}],
79             ALTURA_JANELA - 50,
80         )

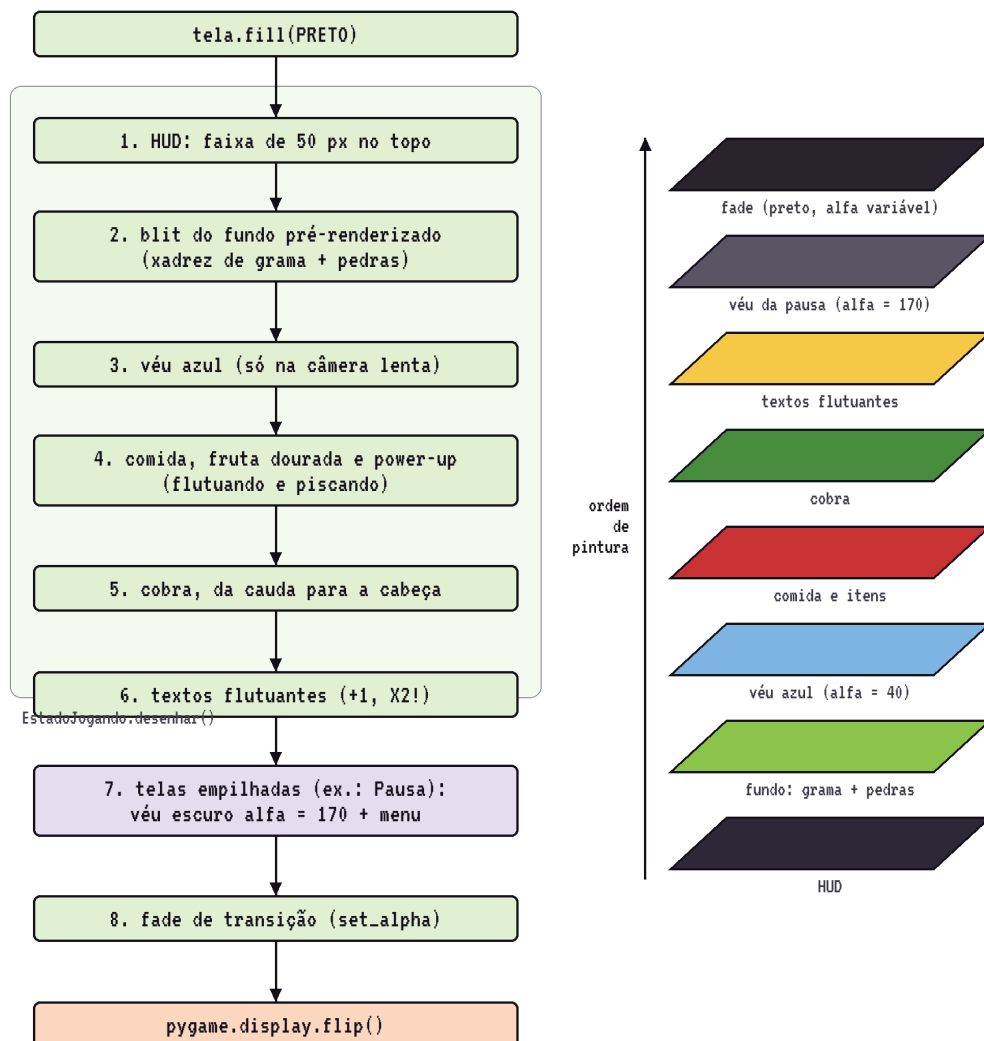
```

Fonte: src/cobrinha/estados/pausa.py, linhas 67–79.

6.6 O QUADRO COMPLETO

A Figura 12 junta tudo: o fluxograma à esquerda segue a ordem das chamadas, e a ilustração à direita mostra as camadas empilhadas na ordem em que são pintadas.

Figura 12 – Pipeline de desenho de um quadro durante a pausa: etapas (à esquerda) e camadas (à direita)



Fonte: elaborada pelo autor (2026) com base em `jogo.py`, `estados/jogando.py` e `estados/pausa.py`.

6.7 REDESENHAR TUDO A CADA QUADRO?

O jogo limpa e redesenha a tela inteira a cada quadro, mesmo quando quase nada mudou. Uma alternativa clássica é atualizar só os retângulos que mudaram (*dirty rectangles*), técnica que o pygame suporta com `pygame.display.update(retangulos)`. O briefing a deixa de propósito como último recurso (item D7, "medir antes de otimizar"): com 480 000 pixels, uma dúzia de `blit` por quadro e cópias feitas em C pela SDL, o redesenho completo cabe com folga nos 16,7 ms de um quadro a 60 FPS, e o código fica muito mais simples. A opção `--debug` mostra o FPS real no canto da tela, justamente para que essa decisão possa ser verificada.

7 PIXEL ART GERADA POR CÓDIGO

A V1 usava sprites desenhados em outras ferramentas, com até 840×462 px, que o jogo esticava para 30×30 px a cada quadro. O resultado eram imagens distorcidas e borradas (briefing, seção 2.3). A V2 inverteu a lógica: os sprites passaram a ser desenhados **no tamanho exato de uso**, 25×25 px, pixel a pixel, por um programa: `ferramentas/gerar_sprites.py`. Este capítulo mostra como formas geométricas descritas por funções viram pixel art, e por que gerar arte por código é uma escolha interessante num projeto de CG.

7.1 PIXEL ART E PALETA LIMITADA

Pixel art é o estilo em que cada pixel é posicionado deliberadamente e permanece visível como um bloco de cor. Ele nasceu da limitação dos primeiros computadores e consoles, com resolução baixa e poucas cores simultâneas, e hoje é usado como escolha estética. Duas regras do estilo orientam o projeto: os sprites são desenhados na resolução final, sem redimensionamento, e usam uma **paleta limitada**.

A paleta do jogo tem 16 cores, definidas na classe `Paleta` do `config.py` e usadas tanto pelos sprites quanto pela interface (HUD, menus e textos). A Figura 13 mostra as 16 cores com seus valores RGB. Ter uma paleta única dá coesão visual ao jogo: a sombra da maçã, a terra do cabinho e o chapéu do cogumelo usam o mesmo marrom, e o verde-escuro do contorno da cobra é o mesmo das escamas. O capítulo 9 volta a esse assunto do ponto de vista da teoria das cores.

Figura 13 – A paleta de 16 cores do jogo (classe `Paleta`, em `config.py`), com os valores RGB



Fonte: elaborada pelo autor (2026) com `docs/ebook/figuras.py`, a partir de `src/cobrinha/config.py`.

A Figura 14 mostra todos os sprites gerados, ampliados oito vezes com o método do vizinho mais próximo (capítulo 8), para que cada pixel possa ser visto. O xadrez cinza ao fundo indica as áreas transparentes.

Figura 14 – Os dez sprites do jogo (25 × 25 px), ampliados 8 vezes pelo vizinho mais próximo



Fonte: elaborada pelo autor (2026) a partir dos PNGs gerados por `ferramentas/gerar_sprites.py`.

7.2 FORMAS DESCRITAS POR FUNÇÕES

Como dizer a um programa "desenhe uma maçã"? A resposta do gerador é descrever cada forma por uma **função de pertinência**: uma função `dentro(x, y)` que responde se o pixel (x, y) faz parte da forma. Para pintar a forma, basta percorrer os 625 pixels da célula e colorir aqueles para os quais a função responde que sim.

Esse é o princípio das **curvas implícitas**, apresentadas por Marschner e Shirley (2021): uma curva definida por uma equação $f(x, y) = 0$, que divide o plano em pontos com $f < 0$ (dentro) e $f > 0$ (fora). Um círculo de raio r e centro (c_x, c_y) é o conjunto dos pontos em que a distância ao centro é igual a r ; seu interior, os pontos em que essa distância é menor. O Código 14 mostra exatamente isso no desenho da maçã.

Código 14 – Corpo e sombra da maçã por uma função de distância (comida)

```
163 def comida(cor: Cor = Paleta.VERMELHO, sombra_cor: Cor = Paleta.MARROM) ->
pygame.Surface:
164     """Maçã: corpo com sombra, brilho, cabinho e folha."""
165     superficie = nova_superficie()
166     centro_x, centro_y, raio = 12.5, 14.5, 8.5
167
168     def dentro(x: int, y: int) -> bool:
169         return math.hypot(x + 0.5 - centro_x, y + 0.5 - centro_y) <= raio
170
171     for y in range(T):
```

```

172     for x in range(T):
173         if dentro(x, y):
174             # Sombra no lado de baixo/direita, para dar volume.
175             sombra = (x + 0.5 - centro_x) + (y + 0.5 - centro_y) > raio * 0.9
176             superficie.set_at((x, y), sombra_cor if sombra else cor)
177     contornar(superficie, dentro, abertos=set())

```

Fonte: ferramentas/gerar_sprites.py, linhas 163–177.

Dois detalhes do código são conceitos de CG.

O primeiro é o `+ 0.5` nas linhas 169 e 175. O pixel (x, y) é tratado como uma **amostra no seu centro**, $(x + 0,5, y + 0,5)$, e não no canto superior esquerdo. É a mesma visão de Smith (1995), para quem um pixel é uma amostra pontual. Sem esse deslocamento, as formas sairiam meio pixel deslocadas para cima e para a esquerda e perderiam a simetria.

O segundo é a sombra da linha 175. A condição $(x - c_x) + (y - c_y) > 0,9 r$ descreve um **semiplano**: todos os pontos de um lado de uma reta diagonal. Os pixels da maçã que caem nesse semiplano, no canto inferior direito, recebem a cor de sombra. O resultado sugere uma luz vindo do canto superior esquerdo, convenção comum na pixel art e seguida em todos os sprites do jogo (veja a pedra e o cogumelo na Figura 14).

7.3 O ANEL DA CURVA

A peça mais interessante é a curva do corpo da cobra, que liga dois lados vizinhos da célula. O Código 15 desenha um **quarto de anel**: os pixels cuja distância ao canto inferior esquerdo da célula fica entre 4 e 21 px.

Código 15 – A curva do corpo como um quarto de anel (`corpo_curva`)

```

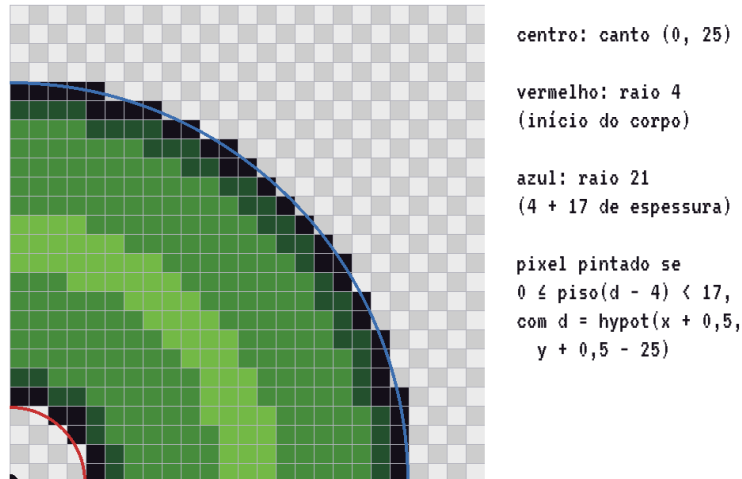
90 def corpo_curva() -> pygame.Surface:
91     """Quarto de anel centrado no canto inferior esquerdo da célula.
92
93     Sem escamas: dobradas no arco, elas ficavam distorcidas; as faixas de cor bastam.
94     """
95     superficie = nova_superficie()
96     centro_x, centro_y = 0.0, float(T)
97     for y in range(T):
98         for x in range(T):
99             distancia = math.hypot(x + 0.5 - centro_x, y + 0.5 - centro_y)
100             profundidade = math.floor(distancia - BORDA_CORPO)
101             if not 0 <= profundidade < ESPESSURA:
102                 continue
103             superficie.set_at((x, y), cor_do_corpo(profundidade, None))
104     return superficie

```

Fonte: ferramentas/gerar_sprites.py, linhas 90–104.

A distância d ao centro, menos a borda de 4 px, vira a **profundidade** do pixel dentro do corpo: 0 na borda interna e 16 na externa. A Figura 15 mostra o sprite ampliado com os dois círculos que delimitam o anel.

Figura 15 – Sprite da curva ampliado 24 vezes, com os círculos de raio 4 (vermelho) e 21 (azul) centrados no canto inferior esquerdo



Fonte: elaborada pelo autor (2026) com docs/ebook/figuras.py, a partir de corpo_curva.png.

O ganho dessa formulação é que o corpo reto e a curva usam **a mesma função de cor**. No corpo reto, a profundidade de um pixel é a distância vertical até a borda superior do corpo ($y - \text{BORDA_CORPO}$, linha 86); na curva, é a distância radial. Como a cor depende só da profundidade, as duas peças se encaixam sem emenda: a faixa clara do meio do corpo reto continua exatamente na faixa clara da curva.

7.4 SOMBREAMENTO POR FAIXAS

A função `cor_do_corpo`, no Código 16, transforma a profundidade em cor. É um sombreamento em faixas: contorno preto nas bordas, verde-escuro logo dentro, verde no corpo e verde-claro no centro, que dá a impressão de um corpo cilíndrico iluminado de frente.

Código 16 – Cor de um pixel do corpo em função da profundidade (`cor_do_corpo`)

```
38 def cor_do_corpo(profundidade: int, ao_longo: float | None) -> Cor:
39     """Cor de um pixel do corpo a partir da posição na seção transversal.
40
41     `profundidade` vai de 0 a 16 de uma borda à outra; `ao_longo` é a posição no
42     sentido do comprimento, usada para desenhar as escamas em "V" (None = sem escamas).
43     """
44     if profundidade in (0, ESPESSURA - 1):
45         return Paleta.PRETO
46     if profundidade in (1, ESPESSURA - 2):
47         return Paleta.VERDE_ESCURO
```

```

48     distancia_meio = abs(profundidade - ESPESSURA // 2)
49     escama = ao_longo is not None and (round(ao_longo) + distancia_meio) % 8 == 0
50     if escama and 3 <= profundidade <= ESPESSURA - 4:
51         return Paleta.VERDE_ESCURO
52     if distancia_meio <= 1:
53         return Paleta.VERDE_CLARO
54     return Paleta.VERDE

```

Fonte: ferramentas/gerar_sprites.py, linhas 38–54.

As escamas em "V" do corpo reto (linha 49) usam a posição ao longo do corpo: um pixel é escama quando a soma da posição com a distância ao meio é múltipla de 8, o que desenha diagonais que se encontram no centro. Na curva, o parâmetro `ao_longo` é `None` e não há escamas. O diário de bordo explica por quê: dobradas no arco, as escamas ficavam distorcidas, e a decisão foi tirada das capturas de tela da Fase 3.

7.5 CONTORNO POR VIZINHANÇA

O contorno preto das formas é calculado, e não desenhado à mão. A função `contornar`, no Código 17, pinta de preto todo pixel da forma que tem pelo menos um **vizinho de 4** (esquerda, direita, acima ou abaixo) fora da forma.

Código 17 – Contorno calculado pela vizinhança de 4 (`contornar`)

```

63 def contornar(superficie: pygame.Surface, dentro: Callable[[int, int], bool], abertos:
64     set[str]):
65     """Pinta de preto os pixels da forma que encostam no lado de fora.
66     `abertos` lista as bordas da célula por onde a peça continua (sem contorno ali).
67     """
68     for y in range(T):
69         for x in range(T):
70             if not dentro(x, y):
71                 continue
72             for dx, dy, lado in ((-1, 0, "esq"), (1, 0, "dir"), (0, -1, "cima"), (0, 1,
73 "baixo")):
74                 vx, vy = x + dx, y + dy
75                 fora_da_celula = not (0 <= vx < T and 0 <= vy < T)
76                 if fora_da_celula and lado in abertos:
77                     continue
78                 if fora_da_celula or not dentro(vx, vy):
79                     superficie.set_at((x, y), Paleta.PRETO)
80                     break

```

Fonte: ferramentas/gerar_sprites.py, linhas 63–79.

O parâmetro `abertos` resolve um problema de encaixe: a cabeça, a cauda e o corpo continuam na célula vizinha, e por isso não podem ter contorno no lado por onde se ligam. A cabeça, por exemplo, é contornada com `abertos={"esq"}`, porque o pescoço sai pela esquerda.

A escolha da vizinhança de 4, e não de 8 (que incluiria as diagonais), produz um contorno de 1 pixel que nunca fica "grosso" nas diagonais, o que é desejável na pixel art.

7.6 POR QUE GERAR ARTE POR CÓDIGO

O briefing e o diário de bordo registram as vantagens que motivaram essa escolha:

- **Reprodutibilidade:** o gerador é determinístico. Rodar `python ferramentas/gerar_sprites.py` recria exatamente os mesmos PNGs, e o diário de bordo registra que, ao acrescentar os power-ups na V3, os sprites antigos foram regenerados e saíram idênticos.
- **Coesão:** todos os sprites usam a mesma paleta e as mesmas convenções de luz, contorno e espessura.
- **Ajuste fino:** mudar a espessura do corpo (`ESPESSURA = 17`) ou a cor da sombra recalcula todas as peças de uma vez, mantendo o encaixe.
- **Testabilidade:** `tests/test_recursos.py` confere o tamanho de 25×25 px e a transparência dos sprites gerados.
- **Documentação:** o script é, ele próprio, a descrição de como cada asset foi feito. As regras do repositório (`CLAUDE.md` e `CONTRIBUTING.md`) proíbem editar os PNGs à mão: muda-se o gerador e regenera-se.

Experimente

Mude `ESPESSURA = 17` para `13` em `ferramentas/gerar_sprites.py`, regenere os sprites e jogue uma partida. Observe que a curva e o corpo reto continuam se encaixando. Depois, desfça a mudança com `git checkout -- src/cobrinha/assets ferramentas/gerar_sprites.py`.

8 TRANSFORMAÇÕES GEOMÉTRICAS

As transformações geométricas (translação, rotação e escala) são o vocabulário básico da CG: com elas, um mesmo modelo pode ser posicionado, orientado e dimensionado de muitas formas, sem ser redesenhado. Hughes *et al.* (2014) e Marschner e Shirley (2021) dedicam capítulos inteiros a elas, porque são a base de todo o pipeline gráfico. Este capítulo apresenta as três transformações no plano e mostra duas delas aplicadas a imagens raster no jogo: a rotação das peças da cobra e a escala da tela cheia.

8.1 AS TRANSFORMAÇÕES NO PLANO

Um ponto do plano é um par (x, y) . As transformações mais simples são:

- **Translação** por (t_x, t_y) : $(x, y) \rightarrow (x + t_x, y + t_y)$.
- **Escala** por (s_x, s_y) , em relação à origem: $(x, y) \rightarrow (s_x x, s_y y)$.
- **Rotação** por um ângulo θ em torno da origem: $(x, y) \rightarrow (x \cos \theta - y \sin \theta, x \sin \theta + y \cos \theta)$.

A escala e a rotação são **transformações lineares** e podem ser escritas como multiplicação por uma matriz 2×2 . A translação não é linear, mas, com **coordenadas homogêneas**, em que o ponto vira $(x, y, 1)$, as três passam a ser matrizes 3×3 e podem ser compostas por multiplicação (Hughes *et al.*, 2014; Marschner; Shirley, 2021). O capítulo 4 já mostrou um exemplo: a conversão de célula para pixel é uma escala por 25 seguida de uma translação de 50 px no eixo y .

Há um cuidado com o sentido da rotação. A fórmula acima gira no sentido anti-horário quando o eixo y aponta para cima, como na matemática. Na tela, com y para baixo (capítulo 4), a mesma fórmula gira no sentido horário. A função `pygame.transform.rotate` do pygame-ce gira no sentido anti-horário **como visto na tela** (Pygame-ce Developers, c2023). Em coordenadas de tela, uma rotação de 90° nesse sentido leva a direção "direita", $(1, 0)$, para "cima", $(0, -1)$, e corresponde à matriz:

$$\begin{vmatrix} x' \\ y' \end{vmatrix} = \begin{vmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{vmatrix} \cdot \begin{vmatrix} x \\ y \end{vmatrix}$$

8.2 ROTACIONAR IMAGENS RASTER

Rotacionar um modelo geométrico é simples: aplica-se a matriz a cada vértice. Rotacionar uma **imagem raster** é outra história. Cada pixel da imagem nova

tem de buscar sua cor na imagem original, na posição obtida pela rotação inversa, e essa posição em geral cai **entre** pixels. É preciso então reconstruir a imagem entre as amostras, por exemplo copiando a amostra mais próxima ou interpolando as vizinhas (Smith, 1995; Marschner; Shirley, 2021). Para ângulos quaisquer, a imagem também cresce, porque o retângulo girado precisa de uma caixa maior. A documentação do pygame-ce registra as duas coisas: fora dos múltiplos de 90°, a rotação aumenta a superfície e preenche as sobras com transparência, e o resultado não é filtrado (Pygame-ce Developers, c2023).

Os múltiplos de 90° são um caso especial. A matriz da rotação de 90° tem só 0, 1 e -1: ela leva cada pixel da grade **exatamente** em outro pixel da grade, sem nenhuma posição fracionária. Não há reamostragem, não há pixel inventado nem perdido, e um sprite de 25 × 25 continua com 25 × 25. É por isso que o briefing estabelece, na seção 8.3, que as quatro direções das peças são geradas por rotação de 90°, que "não distorce pixel art".

8.3 AS PEÇAS DA COBRA

O gerador de sprites desenha cada peça em uma única orientação: a cabeça olhando para a direita, o corpo reto na horizontal, a curva ligando os lados esquerdo e de baixo, e a cauda presa à direita. As outras orientações são obtidas por rotação. O Código 18 mostra as tabelas que relacionam direções e ângulos.

Código 18 – Ângulos de rotação por direção e o giro anti-horário dos lados

```

33 # Rotação (anti-horária, como em pygame.transform.rotate) que leva "→" para cada direção.
34 ANGULO_DA_DIRECAO = {
35     Direcao.DIREITA: 0,
36     Direcao.CIMA: 90,
37     Direcao.ESQUERDA: 180,
38     Direcao.BAIXO: 270,
39 }
40 _GIRO_ANTI_HORARIO = {
41     Direcao.DIREITA: Direcao.CIMA,
42     Direcao.CIMA: Direcao.ESQUERDA,
43     Direcao.ESQUERDA: Direcao.BAIXO,
44     Direcao.BAIXO: Direcao.DIREITA,
45 }
46 LADOS_CURVA_BASE = frozenset({Direcao.ESQUERDA, Direcao.BAIXO})

```

Fonte: src/cobrinha/ui/pecas.py, linhas 33–46.

Para escolher a peça e o ângulo de cada segmento, a função `classificar_pecas` (Código 19) olha para os vizinhos de cada segmento. A cabeça aponta para longe do segmento seguinte; a cauda aponta para o anterior; um segmento do meio é reto se os vizinhos estão em lados opostos e é curva em caso contrário.

Código 19 – Escolha da peça e da rotação de cada segmento (classificar_pecas)

```

93 def classificar_pecas(segmentos: Sequence[Posicao], direcao: Direcao) -> list[Peca]:
94     """Escolhe a peça e a rotação de cada segmento, da cabeça à cauda."""
95     pecas = []
96     ultimo = len(segmentos) - 1
97     for indice, posicao in enumerate(segmentos):
98         if indice == 0:
99             frente = direcao if ultimo == 0 else direcao_entre(segmentos[1], posicao)
100             pecas.append(Peca(TipoPeca.CABECA, posicao, ANGULO_DA_DIRECAO[frente]))
101         elif indice == ultimo:
102             ligacao = direcao_entre(posicao, segmentos[indice - 1])
103             pecas.append(Peca(TipoPeca.CAUDA, posicao, ANGULO_DA_DIRECAO[ligacao]))
104         else:
105             para_frente = direcao_entre(posicao, segmentos[indice - 1])
106             para_tras = direcao_entre(posicao, segmentos[indice + 1])
107             if para_frente is para_tras.oposta:
108                 horizontal = para_frente in (Direcao.ESQUERDA, Direcao.DIREITA)
109                 pecas.append(Peca(TipoPeca.RETO, posicao, 0 if horizontal else 90))
110             else:
111                 angulo = angulo_da_curva(frozenset({para_frente, para_tras}))
112                 pecas.append(Peca(TipoPeca.CURVA, posicao, angulo))
113     return pecas

```

Fonte: src/cobrinha/ui/pecas.py, linhas 93–113.

O caso mais elegante é o da curva. A curva-base liga os lados {esquerda, baixo}. Girada 90° no sentido anti-horário, cada lado vai para o seguinte da tabela `_GIRO_ANTI_HORARIO`: esquerda vira baixo e baixo vira direita, e a curva passa a ligar {baixo, direita}. A função `angulo_da_curva` (Código 20) gira o conjunto de lados da curva-base até ele coincidir com os lados pedidos e devolve o ângulo correspondente. Em vez de uma tabela com as quatro curvas escrita à mão, o código usa a própria definição da rotação.

Código 20 – Ângulo da curva obtido girando os lados da curva-base (angulo_da_curva)

```

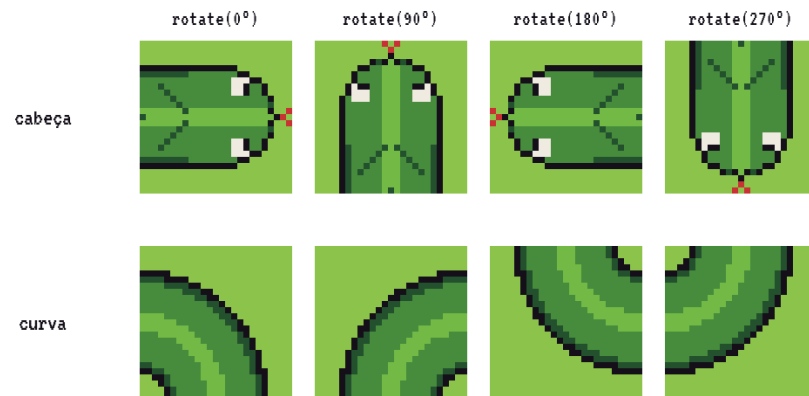
83 def angulo_da_curva(lados: frozenset[Direcao]) -> int:
84     """Rotação da curva-base (esquerda + baixo) para ligar os dois `lados` pedidos."""
85     atual = LADOS_CURVA_BASE
86     for angulo in (0, 90, 180, 270):
87         if atual == lados:
88             return angulo
89         atual = frozenset(_GIRO_ANTI_HORARIO[lado] for lado in atual)
90     raise ValueError(f"Lados não formam uma curva: {lados}")

```

Fonte: src/cobrinha/ui/pecas.py, linhas 83–90.

A Figura 16 mostra a cabeça e a curva nas quatro rotações, produzidas pela própria chamada `pygame.transform.rotate` que o jogo usa.

Figura 16 – Cabeça e curva giradas por `pygame.transform.rotate` em 0°, 90°, 180° e 270°



Fonte: elaborada pelo autor (2026) com `docs/ebook/capturas.py`.

8.4 PRÉ-CALCULAR AS TRANSFORMAÇÕES

Rotacionar uma imagem tem custo, mesmo que pequeno. Como só existem 4 peças $\times$ 4 ângulos = 16 combinações, o jogo calcula todas uma única vez, ao criar a tela de jogo, e guarda o resultado num dicionário (Código 21). Durante a partida, desenhar uma peça é só procurar a imagem pronta e fazer um `blit`. É a mesma troca de memória por tempo da pré-renderização do fundo (capítulo 6).

Código 21 – As 16 rotações preparadas uma vez (`Sprites.__init__`)

```

116 class Sprites:
117     """Todas as rotações das peças, preparadas uma vez (exige a janela aberta)."""
118
119     def __init__(self) -> None:
120         self._pecas = {
121             (tipo, angulo): pygame.transform.rotate(recursos.imagem(tipo.value), angulo)
122             for tipo in TipoPeca
123             for angulo in (0, 90, 180, 270)
124         }

```

Fonte: `src/cobrinha/ui/pecas.py`, linhas 116–124.

8.5 ESCALA: VIZINHO MAIS PRÓXIMO E INTERPOLAÇÃO

Ampliar uma imagem raster é reamostrá-la numa grade mais densa. Os dois métodos mais comuns são:

- **Vizinho mais próximo** (*nearest neighbor*): cada pixel novo copia a cor da amostra original mais próxima. Ampliando por um fator inteiro k , cada pixel vira um bloco de $k \times k$ pixels idênticos. As bordas continuam nítidas, e nenhuma cor nova é criada.

- **Interpolação bilinear:** cada pixel novo é uma média ponderada das quatro amostras vizinhas. O resultado é suave, adequado para fotos, mas borra contornos e cria cores intermediárias que não estavam na paleta.

A Figura 17 compara os dois métodos na maçã do jogo, ampliada 16 vezes.

Figura 17 – A maçã do jogo ampliada 16 vezes pelo vizinho mais próximo e por interpolação bilinear



Fonte: elaborada pelo autor (2026) com `docs/ebook/figuras.py`.

Para pixel art, a escolha é o vizinho mais próximo. O projeto o usa em vários lugares: nas figuras deste ebook, na redução das capturas do GIF de demonstração (`ferramentas/gravar_demo.py`, linha 116) e na geração do ícone do executável a partir da maçã (`ferramentas/empacotar.py`).

8.6 TELA CHEIA SEM BORRAR

O caso mais importante de escala no jogo é a janela. Com a opção `pygame.SCALED` (capítulo 3), o jogo desenha sempre em 800×600 , e a SDL amplia a imagem para o tamanho da janela ou da tela. O filtro usado nessa ampliação é controlado pela dica `SDL_HINT_RENDER_SCALE_QUALITY`, que aceita `nearest` (vizinho mais próximo) ou `linear` (SDL, [20--]). No código-fonte do pygame-ce 2.5.8, ao criar o renderizador para o modo `SCALED`, a biblioteca define essa dica como `nearest` (arquivo `src_c/display.c`, linhas 1650 a 1655; Pygame-ce Developers, 2026). É por isso que a tela cheia do jogo não borra a pixel art, como prometia a seção 8.3 do briefing.

O mesmo trecho do código-fonte revela uma sutileza. Em modo janela, o pygame-ce pede à SDL uma escala **inteira** (`SDL_RenderSetIntegerScale`), e cada pixel lógico vira um bloco de 2×2 , 3×3 etc. pixels físicos, todos do mesmo tamanho. Em tela cheia, o comentário do código explica que a escala inteira é desligada para usar a tela toda, "com pixels desiguais": numa tela de 1920×1080 , por exemplo, o fator

vertical seria $1080 / 600 = 1,8$, e alguns pixels lógicos ocupariam 2 linhas físicas enquanto outros ocupariam só 1. A imagem continua nítida (não há mistura de cores), mas não perfeitamente regular. É uma troca entre aproveitar a tela e manter a grade perfeita, e a biblioteca escolheu aproveitar a tela.

Experimente

Num terminal Python, carregue um sprite com `pygame.image.load` e compare `pygame.transform.rotate(sprite, 90)` com `pygame.transform.rotate(sprite, 45)`. Imprima `get_size()` das duas superfícies e salve-as com `pygame.image.save` para ver o efeito da reamostragem no ângulo de 45°.

9 COR: RGB, PALETA E QUANTIZAÇÃO

A cor de um pixel é, para o computador, um número. Para o olho humano, é uma sensação produzida pela luz. A CG precisa ligar os dois: escolher uma forma de codificar cores que seja eficiente para a máquina e que produza, na tela, as sensações desejadas. Este capítulo apresenta o modelo RGB, as paletas indexadas e a quantização de cores, e mostra como o jogo usa cada um.

9.1 O MODELO RGB

A retina humana tem três tipos de cones, sensíveis a faixas diferentes do espectro, e por isso três números bastam para descrever, de forma aproximada, a maioria das cores que enxergamos (Marschner; Shirley, 2021). Os monitores aproveitam esse fato: cada pixel é formado por três emissores, vermelho (*R*, de *red*), verde (*G*, de *green*) e azul (*B*, de *blue*), e a cor percebida é a soma das três luzes. É um modelo **aditivo**: (0, 0, 0) é o preto, a ausência de luz, e a soma das três no máximo é o branco.

Com 8 bits por canal, cada componente vai de 0 a 255, e são possíveis $256^3 = 16\,777\,216$ combinações. É essa a representação usada pelo pygame-ce, pela SDL e pelos arquivos PNG do jogo. No código, uma cor é uma tupla de três inteiros, como define o tipo `Cor` em `config.py`: `Cor = tuple[int, int, int]`.

Algumas relações do modelo ajudam a ler as cores do jogo. Quando os três canais são iguais, a cor é um cinza; quanto maiores, mais clara. A `Paleta` (Figura 13, no capítulo 7) não usa cinzas "puros": o `PRETO` é (20, 16, 26) e o `CINZA` é (98, 94, 110), ambos com um pouco mais de azul. Esse leve tom frio, comum em paletas de pixel art, faz as sombras parecerem menos "mortas" do que um cinza neutro.

9.2 PALETA INDEXADA

Guardar 24 bits por pixel nem sempre foi possível. Os primeiros *frame buffers* tinham 8 bits por pixel ou menos, e a saída foi a **cor indexada**: cada pixel guarda um índice, e uma tabela de cores, a **paleta** (*colormap*), traduz o índice para uma cor RGB completa (Heckbert, 1982). Com 8 bits por pixel, a imagem pode ter até 256 cores diferentes, mas cada uma delas pode ser qualquer uma das 16,7 milhões.

O pygame-ce ainda suporta esse formato: superfícies de 8 bits usam uma paleta para mapear seus valores para cores de 24 bits (Pygame-ce Developers, c2023). O jogo não usa superfícies indexadas, mas segue o espírito da técnica:

todas as cores de sprites e interface vêm de uma única tabela de 16 entradas, a classe `Paleta` (Código 22).

Código 22 – A paleta única do jogo (`config.py`)

```

76 class Paleta:
77     """Paleta única do jogo (até 16 cores), usada pelos sprites e pela interface."""
78
79     PRETO: Final[Cor] = (20, 16, 26)
80     CINZA_ESCURO: Final[Cor] = (46, 42, 58)
81     CINZA: Final[Cor] = (98, 94, 110)
82     CINZA_CLARO: Final[Cor] = (170, 166, 178)
83     BRANCO: Final[Cor] = (240, 236, 228)
84     VERDE_ESCURO: Final[Cor] = (36, 82, 46)
85     VERDE: Final[Cor] = (70, 140, 60)
86     VERDE_CLARO: Final[Cor] = (118, 186, 72)
87     GRAMA_ESCURA: Final[Cor] = (124, 178, 66)
88     GRAMA_CLARA: Final[Cor] = (142, 196, 78)
89     AMARELO: Final[Cor] = (246, 204, 70)
90     LARANJA: Final[Cor] = (226, 128, 48)
91     VERMELHO: Final[Cor] = (200, 52, 52)
92     MARROM: Final[Cor] = (110, 72, 46)
93     AZUL: Final[Cor] = (64, 112, 178)
94     AZUL_CLARO: Final[Cor] = (128, 182, 226)

```

Fonte: `src/cobrinha/config.py`, linhas 76–94.

Uma paleta limitada traz três benefícios. Visualmente, ela dá unidade ao jogo, como discutido no capítulo 7. Tecnicamente, ela torna as imagens muito compressíveis: o PNG de cada sprite tem só algumas cores, e o GIF de demonstração pode ser guardado com poucas entradas de paleta sem perda visível. E, do ponto de vista do código, ela evita "números mágicos": nenhuma tela inventa uma cor nova; todas falam em `Paleta.VERMELHO`, `Paleta.AZUL_CLARO` etc.

9.3 QUANTIZAÇÃO DE CORES

Às vezes é preciso fazer o caminho inverso: representar uma imagem com muitas cores usando poucas. Isso é a **quantização de cores**. O formato GIF, por exemplo, guarda cada quadro com uma paleta de no máximo 256 cores, e o GIF de demonstração do README é gravado a partir de capturas RGB do jogo.

A quantização mais simples é a **uniforme**: dividir cada canal em faixas fixas (por exemplo, 8 níveis de vermelho, 8 de verde e 4 de azul, totalizando 256 cores) e arredondar cada pixel para a faixa mais próxima. Ela ignora a imagem: reserva cores para regiões do espaço RGB que talvez nem apareçam e trata mal as que aparecem muito.

Heckbert (1982) propôs dividir o problema em quatro fases: (1) amostrar a imagem para obter estatísticas de cor; (2) escolher uma paleta com base nessas estatísticas; (3) mapear cada cor original para a cor mais próxima da paleta; (4)

redesenhar a imagem, opcionalmente com pontilhado (*dithering*). Para a fase 2, ele apresentou o algoritmo de **corte pela mediana** (*median cut*):

1. Coloque todas as cores da imagem numa caixa, no espaço RGB.
2. Escolha a caixa a dividir e, nela, o eixo (R, G ou B) em que as cores estão mais espalhadas.
3. Divida a caixa na **mediana** das cores ao longo desse eixo, de modo que cada metade fique com o mesmo número de pixels.
4. Repita até ter tantas caixas quanto as cores desejadas; a cor representativa de cada caixa é a média das cores dentro dela.

Como cada corte divide a população de pixels ao meio, as regiões do espaço RGB com muitos pixels recebem muitas cores na paleta, e as regiões vazias não recebem nenhuma. Heckbert (1982) mostrou que, com isso, imagens que exigiriam 15 bits por pixel podiam ser reduzidas a 8 bits ou menos com pouca degradação perceptível, com resultado bem melhor que o da quantização uniforme.

9.4 O MEDIAN CUT NO GIF DE DEMONSTRAÇÃO

O script `ferramentas/gravar_demo.py` grava o GIF do README jogando o jogo sem janela. Na hora de salvar, cada quadro é quantizado pela biblioteca Pillow com o método de corte pela mediana, como mostra o Código 23.

Código 23 – Quadros do GIF quantizados para 48 cores pelo corte pela mediana

```

125     def salvar(self) -> None:
126         DESTINO.parent.mkdir(parents=True, exist_ok=True)
127         paleta = [
128             quadro.quantize(colors=48, method=Image.Quantize.MEDIANCUT) for quadro in
self.quadros
129         ]
130         paleta[0].save(
131             DESTINO,
132             save_all=True,
133             append_images=paleta[1:],
134             duration=round(1000 / FPS),
135             loop=0,
136             optimize=True,
137             disposal=1,
138         )

```

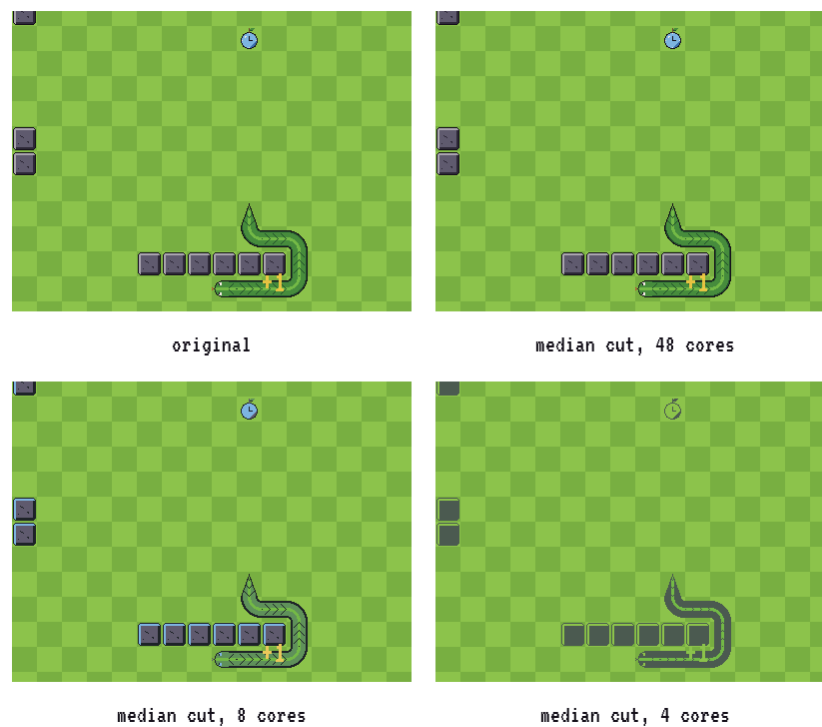
Fonte: `ferramentas/gravar_demo.py`, linhas 125–138.

A documentação da Pillow informa que `Quantize.MEDIANCUT` é o método padrão para imagens RGB e que, se nada for dito, a quantização aplica pontilhado de Floyd-Steinberg (PILLOW..., [2026?]). O jogo pede 48 cores por quadro, bem abaixo do limite de 256 do GIF, e mesmo assim o resultado é praticamente indistinguível do

original, porque cada quadro usa só as cores da paleta do jogo mais as misturas criadas pelas transparências (véus, fade e o antisserrilhamento das bordas, que no jogo é mínimo, porque os textos são desenhados sem suavização).

A Figura 18 mostra um trecho de uma captura real quantizado para 48, 8 e 4 cores, sem pontilhado para deixar o efeito visível. Com 48 cores, não há diferença perceptível. Com 8, o algoritmo ainda preserva os dois verdes do xadrez e as cores principais. Com 4, as caixas do corte pela mediana precisam juntar cores muito diferentes, e detalhes como as rachaduras das pedras e as escamas da cobra desaparecem.

Figura 18 – Trecho de uma captura do jogo quantizado pelo corte pela mediana para 48, 8 e 4 cores (sem pontilhado)



Fonte: elaborada pelo autor (2026) com `docs/ebook/capturas.py` e a biblioteca Pillow.

9.5 COR E ACESSIBILIDADE

Cor também é informação, e o briefing pede que ela não seja a única forma de transmiti-la (item I11). O jogo segue essa regra em vários pontos: o relógio do modo Contra o tempo fica vermelho abaixo de 10 s, mas continua mostrando os segundos em números; os power-ups ativos aparecem no HUD por escrito (`LENTO 4 x2 7`), e não só pelo véu azul; e a maçã dourada, além de amarela, vale mais pontos e mostra "+5" ao ser comida. A mesma lógica orienta a escolha das cores do HUD: texto claro sobre o cinza-escuro da faixa, com alto contraste.

Experimente

Abra uma captura do jogo com a Pillow e rode `imagem.quantize(colors=16, method=Image.Quantize.MEDIANCUT).getpalette()[:48]`. Compare as 16 cores escolhidas pelo algoritmo com as 16 cores da `Paleta`. Quais coincidem? Quais são misturas criadas pelas transparências?

10 TRANSPARÊNCIA E COMPOSIÇÃO ALFA

Uma maçã de 25 × 25 pixels não é um quadrado: é um círculo com cabinho e folha, e os pixels em volta dela devem deixar ver a grama. Uma tela de pausa não apaga a partida: escurece o campo e deixa o jogador ver onde a cobra parou. Esses dois efeitos dependem do mesmo conceito, a **transparência**, e da operação que combina uma imagem sobre outra, a **composição** (*compositing*). Este capítulo apresenta o canal alfa e o operador *over* de Porter e Duff (1984) e mostra os quatro usos de transparência no jogo.

10.1 O CANAL ALFA

Porter e Duff (1984) defenderam que imagens geradas por computador deveriam ter **quatro canais**: os três de cor e um quarto, que chamaram de *matte* e que hoje conhecemos como **canal alfa** (o "A" de RGBA). O alfa de um pixel diz quanto daquele pixel é coberto pela imagem: 1 (ou 255, com 8 bits) significa totalmente opaco; 0 significa totalmente transparente; valores intermediários indicam cobertura parcial, como nas bordas suavizadas de uma forma ou numa camada semitransparente. Os autores mostraram que esse canal pode ser calculado e armazenado junto com as cores e, a partir dele, definiram uma **álgebra de composição**: um conjunto de operadores para combinar duas imagens.

Sem alfa, uma imagem retangular só pode substituir o que está embaixo. Era o problema da maçã da V1, registrado no briefing: o arquivo `comida.jpg` não tinha transparência, porque o formato JPEG não guarda canal alfa, e um quadrado de fundo aparecia em volta da fruta. Na V2, todos os sprites são PNGs com alfa, criados sobre uma superfície totalmente transparente (`pygame.SRCALPHA`, com todos os pixels em (0, 0, 0, 0)), como faz a função `nova_superficie` do gerador de sprites (`ferramentas/gerar_sprites.py`, linhas 57 a 60).

10.2 O OPERADOR *OVER*

Dos operadores de Porter e Duff (1984), o mais usado é o *over*: a imagem *A* sobre a imagem *B*. Para um fundo *B* opaco e uma cor *A* com alfa α entre 0 e 1, a cor resultante *C*, canal a canal, é:

$$C = \alpha \cdot A + (1 - \alpha) \cdot B$$

Ou seja, uma média ponderada: α da cor de cima e o restante da cor de baixo. Com $\alpha = 1$, só *A* aparece; com $\alpha = 0$, só *B*. Porter e Duff trabalham com cores **pré-multiplicadas** pelo alfa (guardando $\alpha \cdot A$ em vez de *A*), o que simplifica a

álgebra quando as duas imagens têm transparência. Para o caso do jogo, com fundo sempre opaco, a fórmula acima basta.

10.3 EXEMPLO NUMÉRICO: O VÉU AZUL DA CÂMERA LENTA

Quando o power-up de câmera lenta está ativo, a tela de jogo pinta sobre o campo uma camada azul com alfa 40 (em 255), criada uma vez no início da partida (estados/jogando.py, linhas 86 e 87). Tomemos um pixel da grama clara, de cor (142, 196, 78), sob o azul (64, 112, 178), com $\alpha = 40/255 \approx 0,157$:

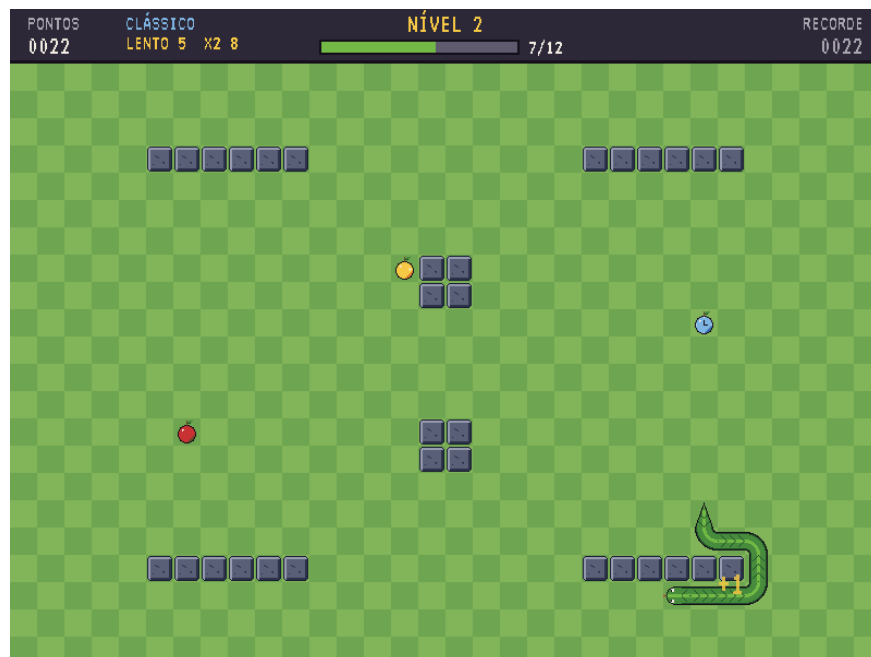
$$R = 0,157 \cdot 64 + 0,843 \cdot 142 \approx 130$$

$$G = 0,157 \cdot 112 + 0,843 \cdot 196 \approx 183$$

$$B = 0,157 \cdot 178 + 0,843 \cdot 78 \approx 94$$

A captura da Figura 19, gerada rodando o jogo, tem exatamente a cor (130, 183, 94) nesse pixel, enquanto a captura do mesmo instante sem o power-up (Figura 4, no capítulo 3) tem (142, 196, 78). O azul é sutil de propósito: ele avisa o jogador sem atrapalhar a leitura do campo, e o efeito também é escrito no HUD ("LENTO 5"), como discutido no capítulo 9.

Figura 19 – Câmera lenta e pontos em dobro ativos: o véu azul sobre o campo e os efeitos escritos no HUD



Fonte: captura gerada pelo autor (2026) com docs/ebook/capturas.py.

A composição também pode ser **encadeada**. Na captura da pausa (Figura 11, no capítulo 6), feita com a câmera lenta ainda ativa, o mesmo pixel recebe primeiro o

véu azul e depois o véu preto da pausa, (20, 16, 26) com alfa 170. Aplicando o *over* outra vez sobre (130, 183, 94), com $\alpha = 170/255 \approx 0,667$, obtém-se aproximadamente (57, 72, 49); a captura tem (57, 72, 48), diferença de uma unidade explicada pelos arredondamentos da SDL. A ordem importa: o *over* não é comutativo, e é o algoritmo do pintor (capítulo 6) que garante a ordem certa.

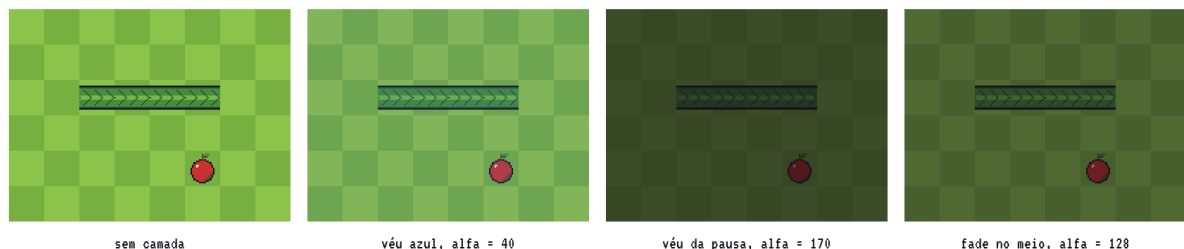
10.4 ALFA POR PIXEL E ALFA DA SUPERFÍCIE

O pygame-ce oferece dois tipos de transparência (Pygame-ce Developers, c2023):

- **Alfa por pixel:** cada pixel tem seu próprio alfa. Surge em superfícies criadas com `pygame.SRCALPHA` ou convertidas com `convert_alpha()`. É o que recorta a maçã em forma de círculo.
- **Alfa da superfície:** um único valor, definido com `set_alpha`, que vale para a superfície inteira. A documentação registra que, desde o pygame 2.0, ele pode ser combinado com o alfa por pixel.

O jogo usa os dois, e a Figura 20 mostra o mesmo trecho do campo sob três das camadas reais.

Figura 20 – O mesmo trecho do campo sem camada, sob o véu azul ($\alpha = 40$), sob o véu da pausa ($\alpha = 170$) e sob o fade no meio da transição ($\alpha = 128$)



Fonte: elaborada pelo autor (2026) com `docs/ebook/capturas.py`, usando as funções e constantes do jogo.

10.4.1 Véus escuros das telas empilhadas

As telas que ficam por cima da partida (pausa, contagem, nível concluído, fim de partida) e as telas de menu usam um véu criado pela função do Código 24: uma superfície do tamanho da janela, com alfa por pixel, preenchida com o preto da paleta e a opacidade pedida.

Código 24 – Véu escuro semitransparente (`criar_veu`)

```
14 def criar_veu(opacidade: int) -> pygame.Surface:
15     """Camada escura semitransparente do tamanho da janela."""
```

```

16     veu = pygame.Surface((LARGURA_JANELA, ALTURA_JANELA), pygame.SRCALPHA)
17     veu.fill((*Paleta.PRETO, opacidade))
18     return veu

```

Fonte: `src/cobrinha/ui/painel.py`, linhas 14–18.

Cada tela escolhe sua opacidade: 150 na contagem e no menu, 170 na pausa e no fim de partida, 200 nas telas de recordes, opções e créditos, em que o texto é longo e o fundo precisa atrapalhar menos. O diário de bordo registra um ajuste feito a partir das capturas: o véu da contagem ficou mais escuro porque as pedras do nível 2 apareciam atrás do nome do nível.

10.4.2 O fade de transição

Ao trocar de tela, o jogo faz a nova tela "surgir do preto" em 0,25 s. A técnica é a mais simples possível: uma superfície preta do tamanho da janela, sem alfa por pixel, desenhada por cima de tudo com alfa de superfície decrescente. No método `Jogo._desenhar` (Código 3, no capítulo 3), as linhas 124 e 125 fazem `set_alpha(round(255 * opacidade_fade))` e o `blit`, e o laço principal reduz `opacidade_fade` de 1 a 0 ao longo da transição. Pelo *over*, cada pixel da tela vai do preto à sua cor real por interpolação linear.

10.4.3 Textos que somem

Os textos flutuantes ("+1", "X2!") sobem e desaparecem em 0,8 s. O Código 25 mostra que o desaparecimento é um alfa de superfície que cai de 255 a 0.

Código 25 – Texto flutuante com alfa decrescente (`Efeitos.desenhar`)

```

37     def desenhar(self, superficie: pygame.Surface) -> None:
38         for efeito in self.textos:
39             progresso = efeito.idade / DURACAO
40             imagem = texto.renderizar(efeito.conteudo, TamanhoFonte.MEDIO,
efeito.cor).copy()
41             imagem.set_alpha(round(255 * (1 - progresso**2)))
42             centro = (efeito.x, round(efeito.y - SUBIDA * progresso))
43             superficie.blit(imagem, imagem.get_rect(center=centro))

```

Fonte: `src/cobrinha/ui/efeitos.py`, linhas 37–43.

Há um detalhe importante na linha 40: o `.copy()`. A função `texto.renderizar` guarda as imagens de texto em cache (capítulo 13), e a mesma superfície do "+1" amarelo é reaproveitada por todos os textos iguais. Se o código mudasse o alfa da superfície em cache, todos os "+1" da tela, inclusive os recém-criados, ficariam com

a mesma transparência. A cópia isola o efeito. A curva do alfa, $255 \times (1 - p^2)$, é discutida no capítulo 11.

Para ir além

Porter e Duff (1984) definem doze operadores de composição, entre eles *over*, *in*, *out*, *atop* e *xor*. O pygame-ce oferece modos de mistura por meio do parâmetro `special_flags` do `blit` (por exemplo, `pygame.BLEND_RGBA_MULT`). Como você usaria um desses modos para escurecer o campo sem criar uma superfície de véu?

11 ANIMAÇÃO

Animar é mostrar uma sequência de imagens estáticas rápido o bastante para que o observador perceba movimento. Marschner e Shirley (2021) tratam a animação por computador como a especificação de como os parâmetros de uma cena variam ao longo do tempo; o computador calcula, para cada quadro, o valor de cada parâmetro e desenha a imagem correspondente. Lasseter (1987) mostrou que os princípios desenvolvidos pelos animadores tradicionais (temporização, aceleração e desaceleração, antecipação, exagero) continuam válidos quando as imagens são geradas por computador. Este capítulo apresenta as pequenas animações do Jogo da Cobrinha e termina com um estudo de caso: o movimento interpolado da cobra, que foi implementado e depois revertido.

11.1 ANIMAR EM FUNÇÃO DO TEMPO

Há duas formas de escrever uma animação. Na primeira, cada quadro muda um pouco o parâmetro: "a cada quadro, suba 1 px". Na segunda, o parâmetro é uma **função do tempo**: "a posição é $f(t)$ ". A primeira depende da taxa de quadros: a 30 FPS, a animação fica duas vezes mais lenta do que a 60 FPS. A segunda não depende, e é a que o jogo usa em todas as animações.

Cada tela guarda um relógio próprio, somando o `dt` recebido em `atualizar` (por exemplo, `self.tempo += dt` em `EstadoJogando`), e as funções de desenho recebem esse tempo como parâmetro. É a mesma separação do capítulo 5, agora aplicada ao desenho: a lógica avança em passos fixos; as animações, em tempo contínuo.

11.2 FLUTUAÇÃO SENOIDAL

A maçã, a maçã dourada e os power-ups não ficam parados: sobem e descem levemente, chamando a atenção do jogador. O Código 26 mostra como.

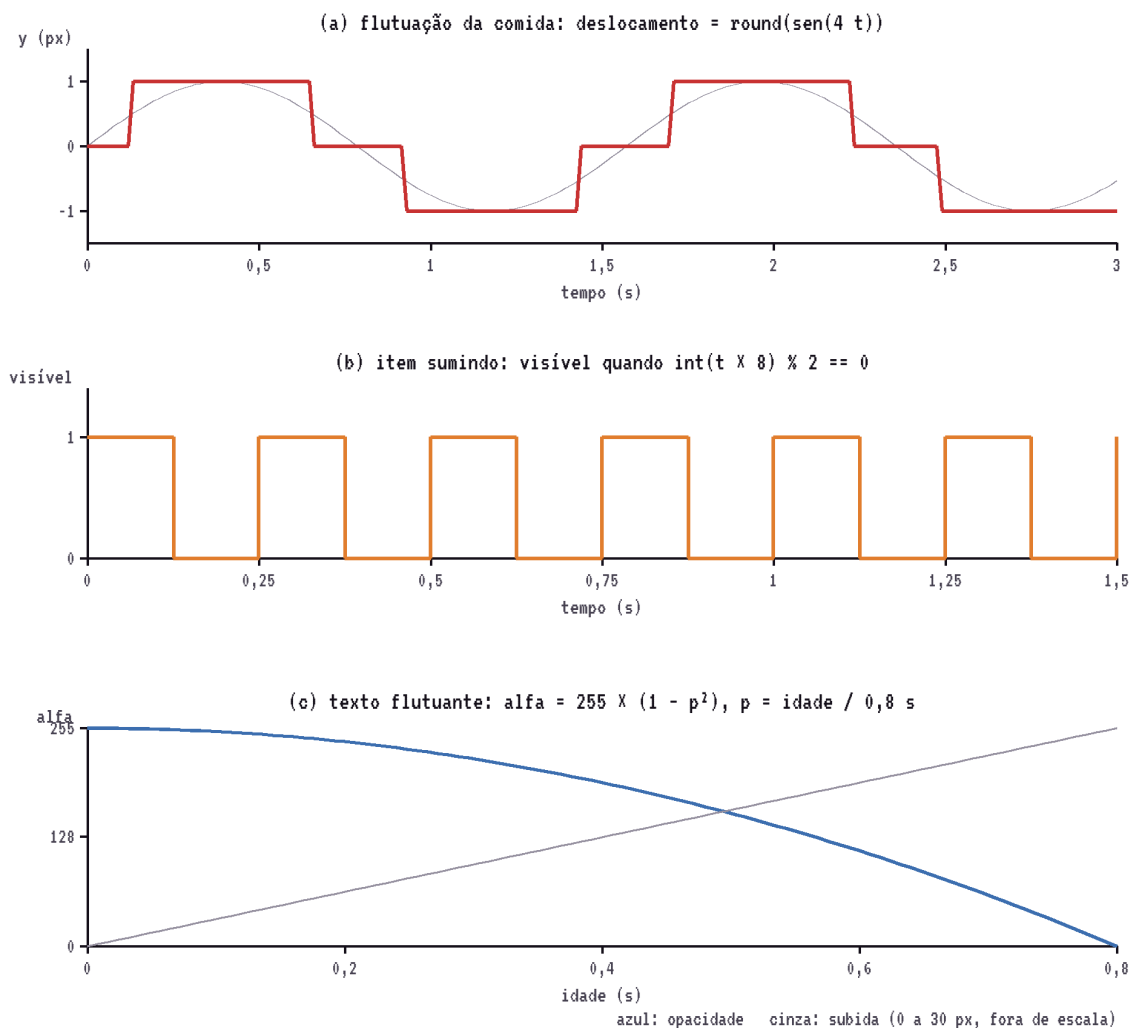
Código 26 – Itens flutuando com um seno arredondado (`_desenhar_flutuando`)

```
173     def _desenhar_flutuando(
174         self, superficie: pygame.Surface, imagem: pygame.Surface, posicao: Posicao,
175         tempo: float
176     ) -> None:
177         x, y = celula_para_pixel(posicao)
178         deslocamento = round(math.sin(tempo * VELOCIDADE_FLUTUACAO))
179         superficie.blit(imagem, (x, y + deslocamento))
```

Fonte: `src/cobrinha/ui/pecas.py`, linhas 173–178.

O deslocamento vertical é $\text{sen}(4t)$, uma oscilação suave entre -1 e 1 com período de $2\pi/4 \approx 1,57$ s. O `round` da linha 177 é uma decisão de pixel art: em vez de posições fracionárias, que exigiriam reamostrar o sprite (capítulo 8), o item ocupa só três posições, -1 , 0 e $+1$ px, e continua alinhado à grade de pixels. O resultado é um movimento "em degraus" que combina com o estilo do jogo, como mostra a parte (a) da Figura 21.

Figura 21 – Funções de animação do jogo: (a) flutuação dos itens; (b) pisca-pisca dos itens que vão sumir; (c) opacidade dos textos flutuantes



Fonte: elaborada pelo autor (2026) com `docs/ebook/figuras.py`, com as constantes de `ui/pecas.py` e `ui/efeitos.py`.

11.3 PISCAR

O pisca-pisca é a animação mais antiga dos jogos e uma forma eficiente de dizer "atenção". O jogo o usa em dois momentos.

Os itens temporários (maçã dourada e power-ups) piscam no último 1,5 s antes de sumir, para avisar o jogador de que o tempo está acabando (Código 27).

Código 27 – Itens temporários piscando antes de sumir (`_desenhar_temporario`)

```

151     def _desenhar_temporario(
152         self,
153         superficie: pygame.Surface,
154         imagem: pygame.Surface,
155         posicao: Posicao,
156         tempo: float,
157         tempo_restante: float,
158     ) -> None:
159         """Item que some sozinho: pisca quando está para sumir."""
160         sumindo = tempo_restante < AVISO_FRUTA_SUMINDO
161         if sumindo and int(tempo * PISCADAS_POR_SEGUNDO) % 2:
162             return
163         self._desenhar_flutuando(superficie, imagem, posicao, tempo)

```

Fonte: `src/cobrinha/ui/pecas.py`, linhas 151–163.

A expressão `int(tempo * 8) % 2` vale 0 e 1 alternadamente, mudando a cada 1/8 s: o item aparece por 0,125 s e some por 0,125 s, quatro vezes por segundo, como mostra a parte (b) da Figura 21. O item não é apagado: ele simplesmente deixa de ser desenhado naquele quadro.

O segundo uso é a animação de morte. Ao bater, a cobra pisca durante 0,9 s, com 7 alternâncias por segundo, antes da tela de fim de partida (`estados/jogando.py`, linhas 159 a 164). O intervalo dá ao jogador tempo de ver onde bateu, e as teclas são ignoradas nesse período, para que um toque nervoso não pule a tela de fim (diário de bordo, Fase 4). Terminada a animação, a tela de fim de partida é empilhada sobre o campo congelado (Figura 22).

Figura 22 – Tela de fim de partida, empilhada sobre o campo congelado depois da animação de morte



Fonte: captura gerada pelo autor (2026) com docs/ebook/capturas.py.

Piscar pode incomodar algumas pessoas e, em casos extremos, é um risco para pessoas com fotossensibilidade. Por isso a tela de opções tem "efeitos visuais: NÃO", que desliga o pisca-pisca da morte e os textos flutuantes (item I11 do briefing).

11.4 TEXTOS QUE SOBEM E SOMEM

Ao comer, um texto como "+1" sobe 30 px e desaparece em 0,8 s (Código 25, no capítulo 10). Com $p = \text{idade} / 0,8$ indo de 0 a 1, a altura cresce linearmente, $30p$, mas a opacidade segue $255 \times (1 - p^2)$, parte (c) da Figura 21. A curva quadrática faz o texto ficar quase opaco no início e sumir rapidamente no fim. Na linguagem de Lasseter (1987), é uma forma de *slow in*: a mudança começa devagar e acelera, de modo que o jogador tem tempo de ler o valor antes de ele desaparecer.

11.5 ESTUDO DE CASO: O MOVIMENTO INTERPOLADO

A decisão D1 do briefing estabeleceu que a cobra se moveria célula a célula, "fiel ao clássico e à pixel art", e deixou o movimento suave para a V3. O primeiro trabalho da V3, em 2 de outubro de 2026, às 22:44, foi exatamente esse (*pull request* 2). Às 23:07, o autor decidiu desfazê-lo (*pull request* 4). O episódio, bem documentado no diário de bordo e no histórico do Git, é um bom exemplo de decisão de projeto em CG.

11.5.1 A técnica

Fiedler (2004) propõe, ao final da sua discussão sobre passo fixo, que o desenho seja feito **entre** os dois últimos estados da simulação, usando a fração do passo que já se passou. Na V3, essa fração foi exposta pela propriedade `progresso_passo` da partida: o valor do acumulador dividido pelo intervalo do passo (Código 28).

Código 28 – Fração do passo atual, usada para interpolar o desenho (versão removida)

```

118     @property
119     def progresso_passo(self) -> float:
120         """Quanto do passo atual já passou (0 a 1), para animar o movimento entre
121         células (V3).
122         Parada a partida (fim do nível, derrota ou vitória), a cobra fica na célula
123         final.
124         """
125         if not self.em_andamento:
126             return 1.0
127         return min(1.0, self._tempo_acumulado / self.intervalo_passo)

```

Fonte: `src/cobrinha/dominio/partida.py`, linhas 118–126, na versão do commit `c0fa005^`.

A cobra passou a lembrar, para cada segmento, de onde ele saiu no último passo e onde está agora (`Cobra.trajetos`, no commit `ba941b8`), e a interface calculava a posição intermediária por **interpolação linear** entre as duas células (Código 29).

Código 29 – Interpolação linear entre duas células vizinhas (versão removida)

```

22 def interpolar_celula(origem: Posicao, destino: Posicao, progresso: float) ->
23     tuple[float, float]:
24     """Ponto entre duas células vizinhas, em células (V3): 0 = `origem`, 1 = `destino`.
25     Se o salto atravessa a borda (modo sem bordas), o segmento anda 1 célula para fora
26     do campo e o resultado volta ao lado oposto, em vez de cruzar o campo inteiro.
27     """
28     dx = destino.coluna - origem.coluna
29     dy = destino.linha - origem.linha
30     if abs(dx) > 1:
31         dx = -int(math.copysign(1, dx))
32     if abs(dy) > 1:
33         dy = -int(math.copysign(1, dy))
34     coluna = (origem.coluna + dx * progresso) % COLUNAS
35     linha = (origem.linha + dy * progresso) % LINHAS
36     return coluna, linha

```

Fonte: `src/cobrinha/ui/campo.py`, linhas 22–36, na versão do commit `890ad71`.

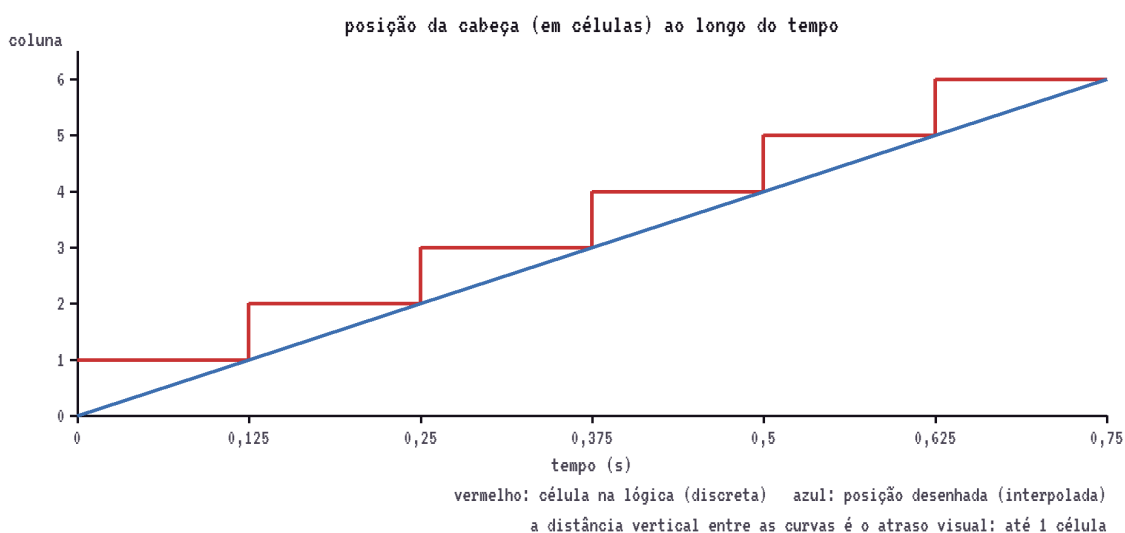
A interpolação linear entre dois pontos P_0 e P_1 , $P(t) = P_0 + t(P_1 - P_0)$, com t entre 0 e 1, é a forma mais simples de produzir quadros intermediários a partir de

quadros-chave (Marschner; Shirley, 2021). As linhas 30 a 33 tratam o caso do modo Sem bordas: quando a cabeça sai por um lado e entra pelo outro, as células de origem e destino estão a 31 colunas de distância, e a interpolação ingênua faria o segmento atravessar o campo inteiro em um passo. O código detecta o salto e o converte em um deslocamento de uma célula "para fora", e o % das linhas 34 e 35 traz o resultado de volta ao campo. O capítulo 12 mostra como esse segmento era desenhado dos dois lados da borda.

11.5.2 O custo: um passo de atraso

O diário de bordo registra a consequência principal já na entrada que introduz a técnica: "o desenho fica até um passo atrás da lógica". A Figura 23 mostra o porquê. Na lógica (curva vermelha), a cabeça muda de célula instantaneamente no início de cada passo. No desenho interpolado (curva azul), ela sai da célula anterior e só chega à célula atual no fim do passo. Quando a lógica já decidiu que a cobra comeu a maçã, a cabeça desenhada ainda está a caminho, e a maçã some antes de a cabeça chegar até ela.

Figura 23 – Posição da cabeça na lógica (degraus) e no desenho interpolado (rampas): o desenho fica até uma célula atrás



Fonte: elaborada pelo autor (2026) com docs/ebook/figuras.py, a 8 passos por segundo.

A 8 passos por segundo, esse atraso chega a 125 ms. Num jogo em que o jogador reage ao que vê, a diferença entre o que está na tela e o que vale para as regras não é só estética: a cobra pode bater num obstáculo que, na tela, ainda parece estar a meia célula de distância.

11.5.3 O que o diário de bordo registra

Os registros de 2 de outubro de 2026 permitem montar o quadro de prós e contras do Quadro 2.

Quadro 2 – Prós e contras do movimento interpolado, segundo o diário de bordo

A favor	Contra
Movimento contínuo, sem "pulos" de célula em célula	O desenho fica até um passo atrás da lógica: a comida some antes de a cabeça chegar
Regras e testes do domínio inalterados: colisão, comida e pontos continuam no passo discreto	Os sprites continuam presos à grade: a orientação da cabeça e das curvas muda de uma vez, por célula, e só a posição é suavizada
A técnica é padrão na literatura (Fiedler, 2004)	Mais código: trajetos, fração do passo, interpolação, desenho dos dois lados da borda com recorte e 9 testes a mais
—	Afasta-se do estilo escolhido: o clássico e a pixel art são discretos (decisão D1)

Fonte: elaborado pelo autor (2026) com base no LOG.md (entradas de 2 de outubro de 2026, 22:44 e 23:07).

A decisão final, registrada às 23:07, foi de design: o autor testou o movimento suave e preferiu o discreto, "fiel ao clássico e à pixel art". A remoção foi feita depois do merge dos *pull requests* 1 a 3, "para o histórico registrar a tentativa e a volta", e o número de testes voltou de 222 para 213.

11.5.4 Lições do caso

O caso ilustra três pontos gerais. Primeiro, a separação entre modelo e visão (capítulo 12) permitiu experimentar uma técnica de desenho sem tocar nas regras: a interpolação morava inteira na interface, e a lógica só expôs dois dados novos. Segundo, uma técnica correta e bem documentada na literatura pode não ser a certa para um jogo específico: interpolar é ótimo para simulações físicas a 60 Hz, mas a 8 passos por segundo o atraso fica perceptível. Terceiro, registrar decisões revertidas tem valor: sem o diário de bordo e os *pull requests*, esse aprendizado teria se perdido.

Experimente

Recupere a versão interpolada com `git switch --detach ddb474b` (o merge do *pull request* 2) e jogue no nível 3, o mais rápido. Repare na maçã sumindo antes de a cabeça chegar e na troca brusca de orientação da cabeça nas curvas. Volte com `git switch -`.

12 COLISÃO, RECORTE E ENVOLVIMENTO DE BORDAS

Este capítulo reúne três assuntos que, no Jogo da Cobrinha, ficam na fronteira entre a geometria e as regras do jogo: a **detecção de colisão**, o **recorte** (*clipping*) do que não deve aparecer e o **envolvimento** das bordas no modo Sem bordas. Os três dependem da decisão arquitetural mais importante do projeto, a separação entre o modelo do jogo e a sua visão, e é por ela que o capítulo começa.

12.1 MODELO E VISÃO

Krasner e Pope (1988) descrevem o paradigma *Model-View-Controller* (MVC), usado no Smalltalk-80, como uma divisão em três partes: objetos que tratam do domínio da aplicação (o modelo), objetos que exibem o estado desse domínio (a visão) e objetos que tratam a interação do usuário (o controlador). A ideia central é que o modelo não conheça a visão: ele pode ser exibido de várias formas, ou de nenhuma, sem mudar.

O Jogo da Cobrinha aplica essa ideia de forma direta (decisão E4 do briefing). O pacote `dominio/` é o modelo: grade, cobra, comida, níveis, power-ups, progresso e partida, sem nenhum `import pygame`. As telas em `estados/` fazem o papel de controlador, traduzindo teclas em comandos (`partida.virar(direcao)`), e também orquestram a visão, que mora em `ui/` (Figura 3, no capítulo 2). A partida não sabe o que é um pixel, uma cor ou um sprite; a interface não decide quando a cobra morre.

Essa separação tem três consequências práticas, todas visíveis no projeto:

- **Testabilidade:** a maior parte dos 240 testes roda sem janela. `tests/test_partida.py` simula partidas aleatórias de 500 passos e confere invariantes a cada passo; `tests/test_niveis.py` usa uma busca em largura para provar que nenhum mapa de pedras tem área fechada.
- **Experimentação segura:** o movimento interpolado (capítulo 11) foi implementado e removido mexendo quase só na interface.
- **Reuso:** o piloto automático que grava o GIF de demonstração e as capturas deste ebook joga com o mesmo modelo, sem duplicar regras.

12.2 DETECÇÃO DE COLISÃO

Em jogos com objetos de formas e posições contínuas, a detecção de colisão é um problema geométrico complexo. Ericson (2005) organiza as técnicas em fases: uma fase ampla (*broad phase*), que descarta rapidamente os pares de objetos que não podem colidir, muitas vezes com grades espaciais ou volumes envolventes

simples, como caixas alinhadas aos eixos (AABB, *axis-aligned bounding boxes*); e uma fase estreita (*narrow phase*), que testa a geometria exata dos pares restantes.

Num jogo em grade, todo esse aparato se reduz a uma pergunta simples: **a célula de destino está ocupada?** Como as posições são inteiras e cada objeto ocupa exatamente uma célula, não há sobreposição parcial, tolerância numérica ou ordem de testes a escolher. A V1, que trabalhava em pixels, criava um `pygame.Rect` por segmento a cada passo para testar colisão; a V2 usa um conjunto (`set`) de células ocupadas, em que testar pertinência custa $O(1)$ em média (item D4 do briefing).

O Código 30 mostra a colisão da cobra com o próprio corpo e o movimento que a mantém consistente.

Código 30 – Colisão com o próprio corpo e avanço da cobra (`Cobra.colidiria` e `Cobra.avancar`)

```

81     def colidiria(self, posicao: Posicao) -> bool:
82         """Se a cabeça entrar em `posicao` no próximo passo, bate no próprio corpo?
83
84         A célula da cauda é segura quando a cobra não está crescendo, porque a
85         cauda sai dali no mesmo passo.
86         """
87         if posicao == self.cauda and self._crescimento_pendente == 0:
88             return False
89         return posicao in self._ocupadas
90
91     def avancar(self, nova_cabeca: Posicao) -> None:
92         if self._crescimento_pendente:
93             self._crescimento_pendente -= 1
94         else:
95             # A cauda sai antes de a cabeça entrar: a cabeça pode ocupar a célula da
96             cauda.
97             self._ocupadas.discard(self.segmentos.pop())
98             self.segmentos.appendleft(nova_cabeca)
99             self._ocupadas.add(nova_cabeca)

```

Fonte: `src/cobrinha/dominio/cobra.py`, linhas 81–98.

O detalhe da linha 87 é uma regra fina do Snake: a célula da cauda é segura quando a cobra não está crescendo, porque a cauda sai dali no mesmo passo em que a cabeça entra. Sem essa regra, a cobra morreria ao "perseguir" a própria cauda, o que contraria o jogo clássico. Para que isso funcione, `avancar` retira a cauda antes de inserir a nova cabeça (linhas 92 a 98). O corpo é um `deque`, em que inserir na cabeça e retirar da cauda custam $O(1)$, e o conjunto `_ocupadas` acompanha o `deque` para responder às perguntas de colisão.

12.3 UM PASSO DA PARTIDA

O método `Partida.passo` junta todas as regras de colisão e consumo de itens (Código 31). A Figura 24 mostra o mesmo método como fluxograma, incluindo as

regras de `_comer`.

Código 31 – Um passo da partida: movimento, colisão e itens (`Partida.passo`)

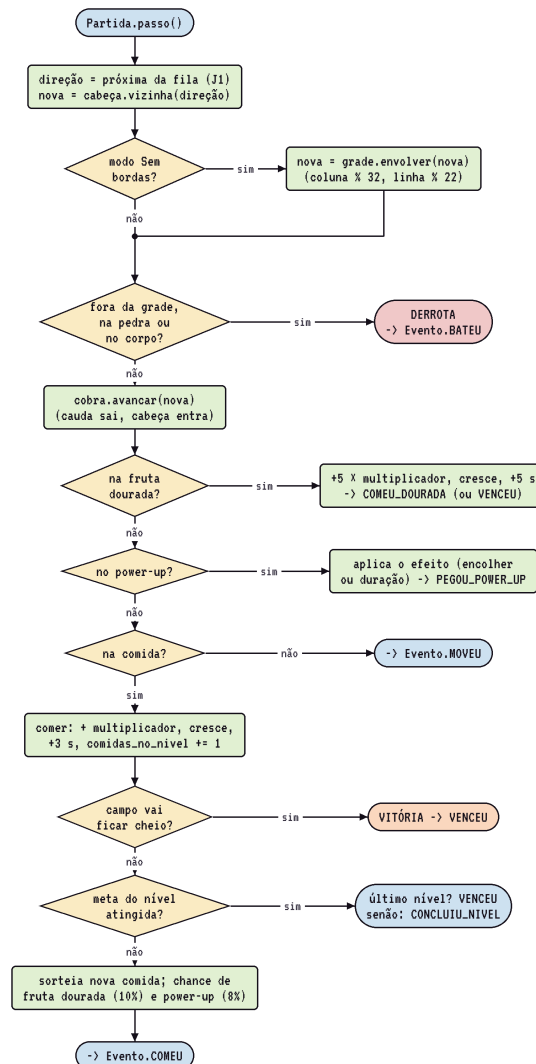
```

161 def passo(self) -> Evento:
162     """Move a cobra uma célula e aplica as regras."""
163     direcao = self.cobra.aplicar_proxima_direcao()
164     nova_cabeca = self.cobra.cabeca.vizinha(direcao)
165     if self.modos is Modos.SEM_BORDAS:
166         nova_cabeca = self.grade.envolver(nova_cabeca)
167
168     if (
169         not self.grade.contem(nova_cabeca)
170         or nova_cabeca in self.obstaculos
171         or self.cobra.colidiria(nova_cabeca)
172     ):
173         self.situacao = Situacao.DERROTA
174         return Evento.BATEU
175
176     self.cobra.avancar(nova_cabeca)
177     if self.fruta_dourada and nova_cabeca == self.fruta_dourada.posicao:
178         return self._comer_fruta_dourada()
179     if self.power_up and nova_cabeca == self.power_up.posicao:
180         return self._pegar_power_up(self.power_up.tipo)
181     if nova_cabeca != self.comida:
182         return Evento.MOVEU
183     return self._comer()

```

Fonte: `src/cobrinha/dominio/partida.py`, linhas 161–183.

Figura 24 – Fluxograma de um passo da partida: mover, colidir, comer, pegar power-up, vencer ou concluir o nível



Fonte: elaborada pelo autor (2026) com base em `src/cobrinha/dominio/partida.py`.

Três observações ajudam a ler o código.

A colisão é testada **antes** de mover (linhas 168 a 174): primeiro se calcula a nova cabeça, depois se pergunta se ela é válida, e só então a cobra avança. Assim, ao bater, a cobra fica parada na posição anterior, e o quadro de "game over" mostra a cabeça encostada no obstáculo, e não dentro dele.

O método devolve um `Evento` (`MOVEU`, `COMEU`, `BATEU` etc.). É assim que o modelo informa a visão sem conhecê-la: a tela de jogo usa os eventos para tocar sons, criar textos flutuantes e parar a música, mas a partida não sabe que nada disso existe.

O fim por tempo, no modo Contra o tempo, não aparece aqui: ele é verificado em `atualizar`, depois dos passos do quadro (capítulo 5), porque depende do relógio, e não do movimento.

12.4 A ENTRADA TAMBÉM É MODELO: A FILA DE DIREÇÕES

O bug B3 da V1 era uma colisão injusta causada pela entrada. A direção nova era comparada com a direção atual da cobra, que mudava a cada tecla. Andando para a direita, se o jogador apertasse ↑ e ← rapidamente, dentro do mesmo passo, a primeira tecla mudava a direção para cima e a segunda, comparada com "cima", era aceita, e a cobra virava para a esquerda, dentro do próprio pescoço.

A V2 resolve isso no modelo, com uma fila de até dois comandos, em que cada comando é validado contra o **último já enfileirado** (`Cobra.virar`, em `dominio/cobra.py`, linhas 60 a 73). As duas teclas viram duas curvas seguidas, uma em cada passo. A correção é coberta por testes, sem janela.

12.5 RECORTE (*CLIPPING*)

Recorte é a operação de descartar as partes de uma primitiva que ficam fora de uma região, normalmente a área visível. Sutherland e Hodgman (1974) apresentaram um algoritmo clássico para recortar polígonos contra uma janela convexa: o polígono é recortado contra cada borda da janela, uma de cada vez, e a saída de cada etapa é a entrada da seguinte.

Em gráficos raster, o recorte também acontece no nível dos pixels. Todo `blit` do `pygame-ce` é recortado automaticamente aos limites da superfície de destino, e cada superfície tem ainda uma **área de recorte** configurável com `set_clip`, que `blit` e `fill` respeitam (Pygame-ce Developers, c2023). Desenhar fora da área não é um erro: os pixels simplesmente não são escritos.

O jogo atual quase não precisa disso, porque tudo é desenhado em células inteiras dentro do campo. A versão com movimento interpolado precisava, e o Código 32, recuperado do histórico do Git, mostra como.

Código 32 – Recorte ao campo e desenho dos dois lados da borda (versão removida com o movimento interpolado)

```

140     @staticmethod
141     def _blit_no_campo(
142         superficie: pygame.Surface, imagem: pygame.Surface, coluna: float, linha: float
143     ) -> None:
144         """Desenha numa posição fracionária do campo; na borda, repete do outro lado."""
145         x = round(coluna * TAMANHO_CELULA)
146         y = ALTURA_HUD + round(linha * TAMANHO_CELULA)
147         largura = COLUNAS * TAMANHO_CELULA
148         altura = LINHAS * TAMANHO_CELULA
149         anterior = superficie.get_clip()
150         superficie.set_clip(pygame.Rect(0, ALTURA_HUD, largura, altura))
151         superficie.blit(imagem, (x, y))
152         if x > largura - TAMANHO_CELULA:
153             superficie.blit(imagem, (x - largura, y))

```

```

154         if y > ALTURA_HUD + altura - TAMANHO_CELULA:
155             superficie.blit(imagem, (x, y - altura))
156         if x > largura - TAMANHO_CELULA:
157             superficie.blit(imagem, (x - largura, y - altura))
158         superficie.set_clip(anterior)

```

Fonte: `src/cobrinha/ui/pecas.py`, linhas 140–158, na versão do commit `e22164c`.

Um segmento que desliza para fora da borda direita, no modo Sem bordas, precisa aparecer em parte na direita e em parte na esquerda do campo. O método faz isso desenhando o mesmo sprite uma segunda vez, deslocado de uma largura (ou altura) do campo, quando ele passa da borda direita ou da inferior, e até uma quarta vez no canto (linhas 151 a 157). Para que as partes excedentes não invadam o HUD nem saiam do campo, define uma área de recorte igual ao retângulo do campo (linha 150), restaurando a anterior no fim (linha 158). É o recorte por região retangular, a forma mais simples do problema estudado por Sutherland e Hodgman (1974), feito pela SDL no momento da cópia dos pixels.

12.6 ENVOLVIMENTO DE BORDAS: O MODO SEM BORDAS

No modo Sem bordas, a cobra que sai por um lado do campo entra pelo lado oposto. Topologicamente, o campo deixa de ser um retângulo e passa a ser um **toro**, a superfície de uma rosquinha, em que as bordas opostas estão coladas. No modelo, isso é uma linha de código, mostrada no Código 33.

Código 33 – Teste de pertinência e envolvimento das bordas (`Grade.contem` e `Grade.envolver`)

```

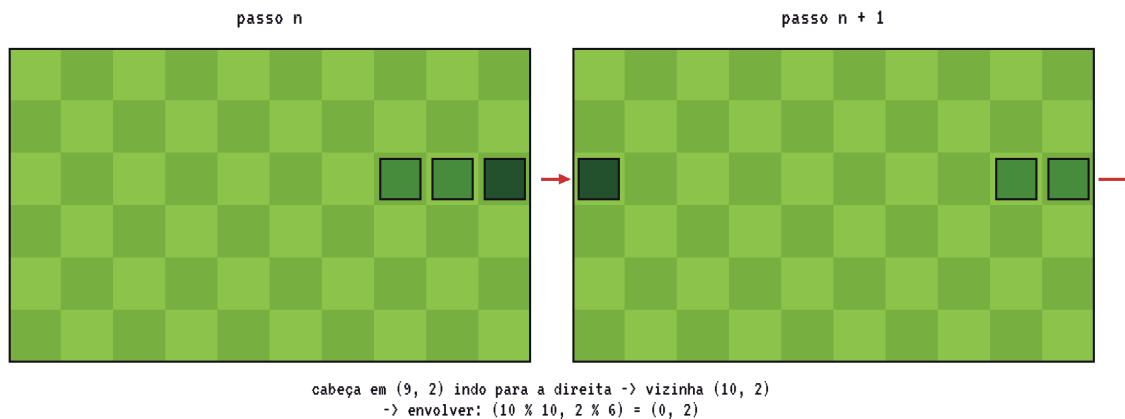
49     def contem(self, posicao: Posicao) -> bool:
50         return 0 <= posicao.coluna < self.colunas and 0 <= posicao.linha < self.linhas
51
52     def envolver(self, posicao: Posicao) -> Posicao:
53         """Leva uma posição de fora do campo para o lado oposto (modo sem bordas)."""
54         return Posicao(posicao.coluna % self.colunas, posicao.linha % self.linhas)

```

Fonte: `src/cobrinha/dominio/grade.py`, linhas 49–54.

O operador `%` (resto da divisão) do Python devolve sempre um valor entre 0 e o divisor menos 1, mesmo para números negativos: $-1 \% 32 = 31$. Por isso, a mesma expressão trata a saída pela direita (coluna 32 vira 0) e pela esquerda (coluna -1 vira 31). Na `Partida.passo`, o envolvimento acontece logo depois do cálculo da nova cabeça, e só no modo Sem bordas (`partida.py`, linhas 165 e 166). Nos outros modos, a posição fora do campo falha no teste `contem` e a cobra bate. A Figura 25 ilustra o caso numa grade pequena.

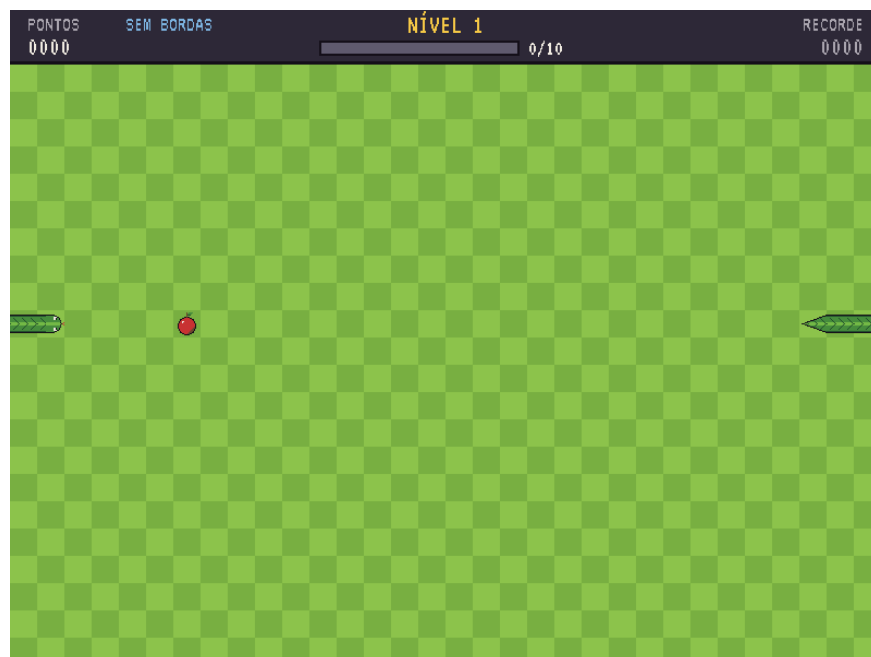
Figura 25 – Envolvimento de bordas numa grade de 10×6 : a cabeça em (9, 2) indo para a direita aparece em (0, 2)



Fonte: elaborada pelo autor (2026) com docs/ebook/figuras.py.

O desenho também precisa saber do envolvimento. Para escolher a peça e a rotação de cada segmento (capítulo 8), `classificar_pecas` calcula a direção entre segmentos vizinhos, e dois vizinhos em lados opostos do campo estão, numericamente, a 31 colunas de distância. A função `direcao_entre` (ui/pecas.py, linhas 68 a 80) trata esse salto como um passo no sentido contrário, e a cobra é desenhada corretamente atravessando a borda, como mostra a Figura 26.

Figura 26 – Modo Sem bordas: a cobra atravessa a borda direita e continua pela esquerda do campo



Fonte: captura gerada pelo autor (2026) com docs/ebook/capturas.py.

12.7 SORTEIO DA COMIDA SEM LAÇO CEGO

Uma última "colisão" é a da comida com o que já ocupa o campo. A V1 sorteava posições ao acaso até achar uma livre, e com o campo cheio esse laço nunca terminava (B7). A V2 sorteia diretamente entre as células livres (`dominio/comida.py`, linhas 9 a 19) e, se não houver nenhuma, devolve `None`, que a partida trata como vitória. O custo é percorrer as 704 células da grade uma vez por comida, desprezível perto dos 16,7 ms de um quadro.

Experimente

Em `tests/test_cobra.py`, escreva um teste que crie uma cobra de 4 segmentos em formato de "U" e confira que `colidiria` devolve `False` para a célula da cauda. Depois chame `crescer()` e confira que a mesma célula passa a ser uma colisão. Rode `pytest tests/test_cobra.py`.

13 INTERFACE E MULTIMÍDIA

Os capítulos anteriores trataram do campo de jogo. Este trata do que está em volta dele: o HUD, os menus, os textos e o som. São assuntos que a CG compartilha com as áreas de interação humano-computador e de multimídia, e que no Jogo da Cobrinha seguem os mesmos princípios do resto do código: tudo é desenhado ou sintetizado por programa, a partir da mesma paleta e das mesmas regras.

13.1 INTERFACE DESENHADA POR CÓDIGO

Na V1, menus, créditos, tela de fim de jogo e transições eram imagens feitas no Canva, com o texto embutido. Mudar uma opção exigia refazer a arte, e a navegação era só por teclas numéricas e letras soltas (briefing, seção 2.3). A V2 desenha toda a interface por código (item I1): os textos são renderizados com uma fonte, os menus são listas de itens navegáveis por setas, WASD e mouse, e o HUD é uma faixa própria de 50 px no topo, fora da área de jogo (item I2).

O Código 34 mostra o começo do desenho do HUD: o fundo cinza-escuro, uma borda preta de 3 px separando-o do campo e os contadores de pontos e recorde. Os elementos seguintes dependem do estado da partida, como o modo de jogo, os power-ups ativos e o aviso de som desligado.

Código 34 – Desenho do HUD (desenhar_hud)

```

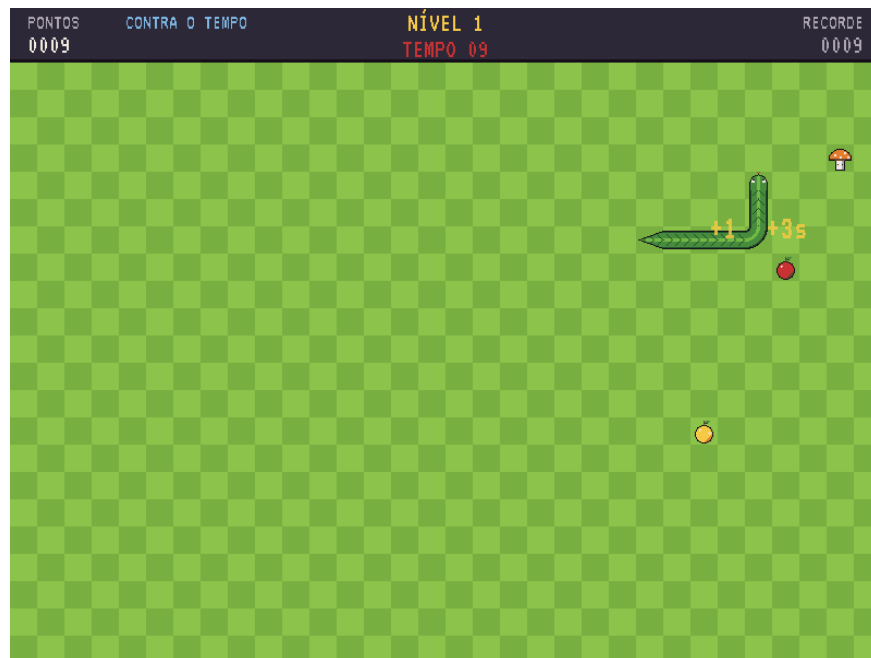
45 def desenhar_hud(superficie: pygame.Surface, dados: DadosHud) -> None:
46     superficie.fill(Paleta.CINZA_ESCURO, (0, 0, LARGURA_JANELA, ALTURA_HUD))
47     superficie.fill(
48         Paleta.PRETO, (0, ALTURA_HUD - ESPESSURA_BORDA, LARGURA_JANELA, ESPESSURA_BORDA)
49     )
50     _desenhar_contador(superficie, "PONTOS", dados.pontos, Paleta.BRANCO, esquerda=True)
51     _desenhar_contador(superficie, "RECORDE", dados.recorde, Paleta.CINZA_CLARO,
52         esquerda=False)
53     _desenhar_nivel(superficie, dados)
54     if dados.modos:
55         modo = texto.renderizar(dados.modos.upper(), TamanhoFonte.MINIMO,
56             Paleta.AZUL_CLARO)
57         superficie.blit(modo, modo.get_rect(topleft=(MARGEM + 90, Y_ROTULO)))
58     if dados.efeitos:
59         resumo = " ".join(f"{rotulo} {math.ceil(restante)}" for rotulo, restante in
60             dados.efeitos)
61         imagem = texto.renderizar(resumo, TamanhoFonte.MINIMO, Paleta.AMARELO)
62         superficie.blit(imagem, imagem.get_rect(topleft=(MARGEM + 90, Y_VALOR)))
63     if dados.mudo:
64         imagem = texto.renderizar("MUDO (M)", TamanhoFonte.MINIMO, Paleta.VERMELHO)
65         superficie.blit(imagem, imagem.get_rect(topright=(LARGURA_JANELA - 160,
66             Y_ROTULO)))

```

Fonte: `src/cobrinha/ui/hud.py`, linhas 45–63.

O HUD recebe um objeto `DadosHud`, montado pela tela de jogo a partir da partida (capítulo 6). É o modelo e a visão de novo: o HUD não lê a partida diretamente, só os números que precisa mostrar. No modo Contra o tempo, a barra de progresso dá lugar a um relógio, que fica vermelho abaixo de 10 s (Figura 27).

Figura 27 – Modo Contra o tempo: no centro do HUD, o relógio em vermelho, abaixo de 10 segundos, no lugar da barra de progresso



Fonte: captura gerada pelo autor (2026) com `docs/ebook/capturas.py`.

13.2 TEXTO: FONTE BITMAP, CACHE E LINHA DE BASE

Desenhar texto numa interface raster é rasterizar as formas das letras (os **glifos**) de uma fonte. O jogo usa a fonte VT323, de licença livre (SIL Open Font License), que imita o texto de terminais antigos e combina com a pixel art. A escolha tem uma história registrada no diário de bordo da Fase 3: a primeira fonte testada, a Press Start 2P, desenhava as maiúsculas acentuadas encolhidas ("NÍVEL" aparecia como "NiVEL") e não diferenciava Ó, Ô e Õ das minúsculas. Num jogo em português, isso era inaceitável. Três alternativas de licença livre foram comparadas lado a lado, e a VT323 foi a única com as maiúsculas acentuadas corretas e traços uniformes. O teste `test_fonte_tem_maiusculas_acentuadas_de_verdade` impede que o problema volte.

Três decisões do código de texto são de CG:

- **Sem suavização:** o texto é renderizado com `antialias=False` (Código 35, linha 15). As bordas ficam serrilhadas, como na pixel art, e o texto usa só a cor pedida, sem tons intermediários.

- **Cache:** rasterizar texto custa caro, e o HUD mostra os mesmos textos quadro após quadro. O decorador `lru_cache` guarda as últimas 256 combinações de texto, tamanho e cor, e o texto só é rasterizado de novo quando o valor muda (item D3 do briefing). Na V1, a fonte era recriada duas vezes por quadro.
- **Linha de base:** as maiúsculas acentuadas sobem acima da altura normal da fonte, e o pygame-ce aumenta a imagem do texto para cima para acomodá-las. Centralizada pelo topo, uma palavra com acento pareceria 3 px mais baixa que as outras. A função `_excesso_acima` mede, com as métricas dos glifos, quanto o texto passa do topo da fonte, e o desenho compensa esse valor, mantendo todas as palavras alinhadas pela **linha de base** (linhas 18 a 28).

Código 35 – Texto sem suavização, com cache e medida do excesso acima da fonte

```

11 @lru_cache(maxsize=256)
12 def renderizar(texto: str, tamanho: int, cor: Cor) -> pygame.Surface:
13     """Superfície com o texto, reaproveitada enquanto o valor não mudar."""
14     # antialias=False mantém as bordas nítidas, no estilo pixel art.
15     return recursos.fonte(tamanho).render(texto, False, cor)
16
17
18 @lru_cache(maxsize=256)
19 def _excesso_acima(texto: str, tamanho: int) -> int:
20     """Pixels que o texto sobe além da altura normal da fonte.
21
22     Maiúsculas acentuadas (Í, Ó, Ê) passam do topo da fonte, e o pygame aumenta a
23     imagem por cima: a linha de base desce e o texto parece fora do alinhamento.
24     """
25     fonte = recursos.fonte(tamanho)
26     metricas = [m for m in fonte.metrics(texto) if m is not None]
27     topo_glifos = max((m[3] for m in metricas), default=0)
28     return max(0, topo_glifos - fonte.get_ascent())

```

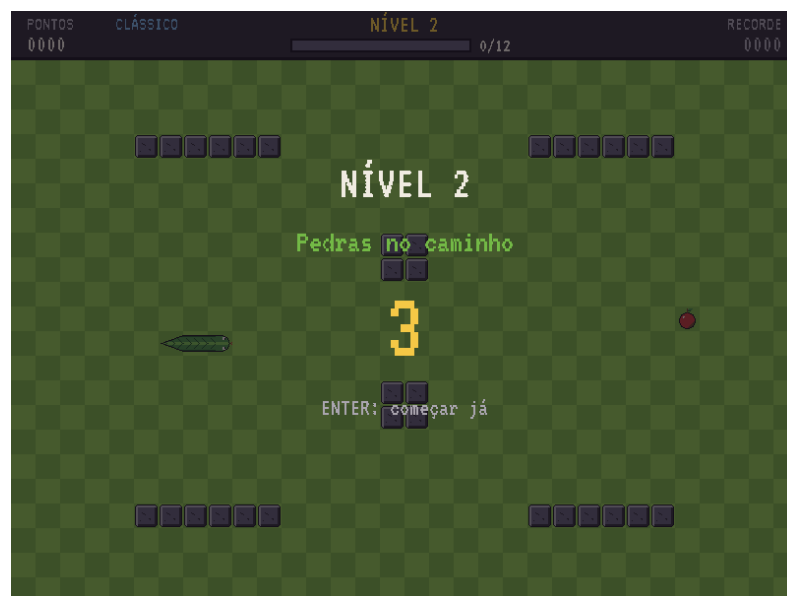
Fonte: `src/cobrinha/ui/texto.py`, linhas 11–28.

Um teste do projeto vai além: ele lê o código das telas, extrai todos os textos exibidos e confere se cada caractere gera pixels na fonte. O teste nasceu de um problema encontrado nas capturas da Fase 4: as setas "← →" informavam métricas, mas o glifo era vazio, e apareciam em branco na tela. O mesmo problema reapareceu na produção deste ebook, nos fluxogramas desenhados com a VT323, e foi resolvido da mesma forma: o gerador de figuras troca os símbolos sem desenho e falha se sobrar algum (`docs/ebook/figuras.py`, função `preparar`).

13.3 TELAS DE TRANSIÇÃO

A contagem 3-2-1 antes de cada nível (Figura 28) é um exemplo de como as peças dos capítulos anteriores se combinam numa tela de interface: ela é empilhada sobre a partida (capítulo 6), escurece o campo com um véu semitransparente (capítulo 10), escreve o nível e o nome da fase com a fonte do jogo e conta o tempo com o `dt` do laço (capítulo 5). O jogador pode pulá-la com Enter, Espaço ou um clique.

Figura 28 – Contagem antes do nível 2, empilhada sobre a partida, com o véu escuro e o nome da fase



Fonte: captura gerada pelo autor (2026) com `docs/ebook/capturas.py`.

13.4 SOM SINTETIZADO POR CÓDIGO

O som não é gráfico, mas é parte da experiência multimídia do jogo e segue a mesma filosofia dos sprites: os 15 arquivos WAV do jogo, entre efeitos e músicas, são **sintetizados** pelo script `ferramentas/gerar_sons.py`, só com a biblioteca padrão do Python.

13.4.1 Amostragem

Um som digital é, como uma imagem raster, uma sequência de **amostras**: valores da pressão do ar medidos a intervalos regulares. A **taxa de amostragem** do jogo é de 22 050 amostras por segundo. Pelo teorema da amostragem, uma taxa f_s só representa frequências até $f_s / 2$, a frequência de Nyquist (Roads, 1996), aqui 11 025 Hz, suficiente para os sons simples de um jogo retrô. Cada amostra é gravada

com 16 bits, em canal único (mono), pelo módulo `wave` da biblioteca padrão do Python.

O paralelo com o capítulo 3 é direto: a taxa de amostragem está para o som assim como a resolução está para a imagem, e o número de bits por amostra está para o som assim como o número de bits por canal de cor está para a imagem.

13.4.2 Ondas e envelope

Os sons de jogos antigos (*chiptune*) eram produzidos por circuitos que geravam ondas simples. O gerador imita isso com duas formas de onda, definidas como funções da **fase** (a fração do ciclo em que a onda está): a onda quadrada, que vale +1 na primeira parte do ciclo e -1 na segunda, e a onda triangular, que sobe e desce linearmente (Código 36).

Código 36 – Ondas quadrada e triangular, frequência das notas e tom com envelope

```

23 def quadrada(ciclo_ativo: float = 0.5) -> Onda:
24     return lambda fase: 1.0 if fase % 1.0 < ciclo_ativo else -1.0
25
26
27 def triangular(fase: float) -> float:
28     return 4 * abs(fase % 1.0 - 0.5) - 1
29
30
31 def nota(nome: str) -> float:
32     """Frequência de uma nota no formato 'A4', 'C#5', 'Eb3'."""
33     semitons = {"C": -9, "D": -7, "E": -5, "F": -4, "G": -2, "A": 0, "B": 2}
34     base, oitava = nome[:-1], int(nome[-1])
35     valor = semitons[base[0]] + (1 if "#" in base else -1 if "b" in base else 0)
36     return 440.0 * 2 ** ((valor + 12 * (oitava - 4)) / 12)
37
38
39 def tom(
40     frequencia_inicial: float,
41     duracao: float,
42     onda: Onda,
43     frequencia_final: float | None = None,
44     volume: float = 0.5,
45     ataque: float = 0.005,
46     soltura: float = 0.03,
47 ) -> list[float]:
48     """Um tom com envelope linear e, se pedido, deslizando de uma frequência para
49     outra."""
50     frequencia_final = frequencia_final or frequencia_inicial
51     total = int(duracao * TAXA)
52     amostras, fase = [], 0.0
53     for i in range(total):
54         progresso = i / max(total - 1, 1)
55         fase += (frequencia_inicial + (frequencia_final - frequencia_inicial) *
56         progresso) / TAXA
57         tempo, restante = i / TAXA, (total - i) / TAXA
58         envelope = min(1.0, tempo / ataque if ataque else 1.0, restante / soltura)

```

```

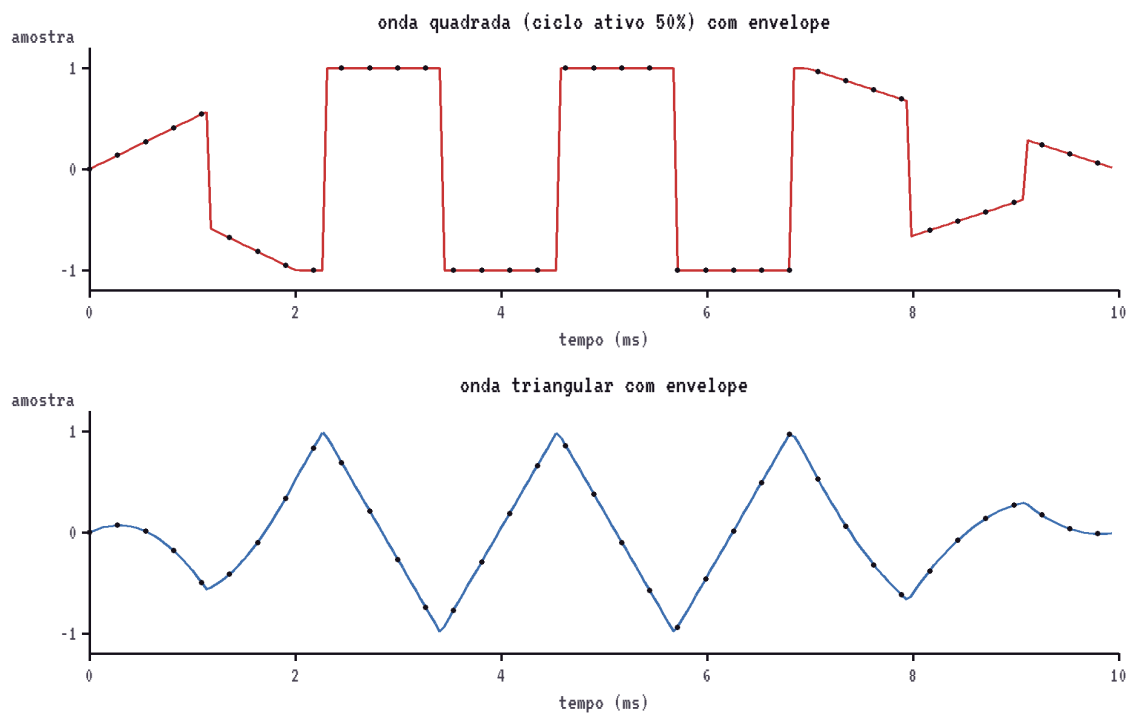
57     amostras.append(onda(fase) * envelope * volume)
58     return amostras

```

Fonte: ferramentas/gerar_sons.py, linhas 23–58.

A função `nota` usa a afinação temperada: cada semitom multiplica a frequência por $2^{1/12}$, a partir do lá central (A4) em 440 Hz. A função `tom` gera as amostras de uma nota com um **envelope** linear: o volume sobe do zero em 5 ms (ataque) e desce a zero nos últimos 30 ms (soltura), o que evita estalos no começo e no fim do som (Roads, 1996). A Figura 29 mostra 10 ms das duas ondas com o envelope aplicado; os pontos marcam algumas das amostras.

Figura 29 – Dez milissegundos das ondas quadrada e triangular a 440 Hz, com envelope e amostras a 22 050 Hz



Fonte: elaborada pelo autor (2026) com docs/ebook/figuras.py, reproduzindo as fórmulas de ferramentas/gerar_sons.py.

13.4.3 Volume medido, e não suposto

O diário de bordo registra um episódio instrutivo. Depois da Fase 3, o autor relatou que o áudio "não estava funcionando". O diagnóstico mostrou que o som tocava, mas baixo: a normalização do gerador só **reduzia** o volume quando o pico passava de 0,9, nunca aumentava, e, somada aos volumes padrão conservadores, deixava os sons cerca de 10 dB abaixo do ideal. A correção normaliza todos os sons para um pico de 0,89 (−1 dBFS), com intensidade relativa por som, e a lição ficou

registrada: "mudanças de áudio são conferidas também pelo pico que chega ao Windows". É o equivalente sonoro de conferir as capturas de tela em vez de supor que o desenho está certo.

13.4.4 Som ligado aos eventos

Os sons são disparados pelos eventos que a partida devolve a cada passo (capítulo 12). A tela de jogo tem uma tabela que associa cada evento a um efeito sonoro (`estados/jogando.py`, linhas 44 a 52): comer toca a mordida, bater toca a batida, concluir o nível toca a fanfarra. A música também reage ao jogo: cada fase tem sua faixa, a música pausa junto com o jogo e para "na hora" ao bater, antes mesmo da animação de morte. Como a partida só emite eventos e não conhece o áudio, esse comportamento pôde ser coberto por testes (`tests/test_musica.py`) sem nenhum dispositivo de som.

Experimente

Em `ferramentas/gerar_sons.py`, no efeito "contagem" (o bipe do 3-2-1), troque `q25` por `triangular` e regenere os sons com `python ferramentas/gerar_sons.py`. Compare o timbre: a onda quadrada tem muitos harmônicos e soa "áspera"; a triangular, com harmônicos mais fracos, soa mais suave. Desfaça a mudança com `git checkout -- ferramentas src/cobrinha/assets/sons`.

14 CONSIDERAÇÕES FINAIS

Este ebook percorreu os conceitos centrais da Computação Gráfica 2D usando como fio condutor um projeto real, o Jogo da Cobrinha, em suas três versões. O Quadro 3 resume a correspondência entre cada conceito e o lugar do código em que ele aparece.

Quadro 3 – Conceitos de Computação Gráfica e onde aparecem no Jogo da Cobrinha

Conceito	Onde aparece no jogo	Capítulo
Raster, framebuffer e buffer duplo	<code>pygame.Surface</code> , <code>Jogo._desenhar</code> e <code>display.flip()</code>	3
Sistemas de coordenadas e transformação afim	Grade de 32 × 22 células e <code>celula_para_pixel</code>	4
Laço de jogo e passo de tempo fixo	<code>Jogo.executar</code> e o acumulador de <code>Partida.atualizar</code>	5
Algoritmo do pintor, blit e camadas	<code>EstadoJogando.desenhar</code> , fundo pré-renderizado e pilha de telas	6
Formas implícitas e rasterização	<code>ferramentas/gerar_sprites.py</code> (maçã, curva, contorno)	7
Rotação e escala de imagens raster	Peças giradas em 90°, vizinho mais próximo e <code>pygame.SCALED</code>	8
RGB, paleta e quantização	Classe <code>Paleta</code> e o corte pela mediana no GIF	9
Canal alfa e operador <i>over</i>	Véus, fade de transição e textos flutuantes	10
Animação por tempo e interpolação	Flutuação, pisca-pisca e o movimento interpolado revertido	11
Colisão, recorte e envolvimento	Conjunto de células ocupadas, <code>set_clip</code> e <code>Grade.envolver</code>	12
Interface e som	HUD, fonte VT323 e síntese de ondas em <code>gerar_sons.py</code>	13

Fonte: elaborado pelo autor (2026).

Três ideias atravessam todos os capítulos.

A primeira é a **separação entre modelo e imagem**. O jogo nunca guarda a posição da cobra em pixels, e as regras nunca dependem de como algo é desenhado. Essa separação, proposta no briefing da V2 como resposta direta aos bugs da V1, é o que permitiu testar o jogo sem janela, gerar as capturas deste ebook com um piloto automático e experimentar o movimento interpolado sem risco para as regras.

A segunda é **fazer o trabalho uma vez**. O fundo é pré-renderizado, as rotações das peças são calculadas ao criar a tela, as imagens são convertidas para o formato da tela ao carregar, e os textos ficam em cache. Em todos os casos, a

técnica é trocar um pouco de memória por tempo de quadro, uma das trocas mais comuns da CG interativa.

A terceira é **gerar por código**. Sprites, sons, o GIF de demonstração, as capturas e as figuras deste ebook são produzidos por scripts versionados no repositório. Isso torna o resultado reproduzível e transforma cada asset em documentação de como ele foi feito, o que é especialmente valioso num trabalho acadêmico: quem quiser entender um sprite pode ler a função que o desenha.

O estudo de caso do movimento interpolado mostrou, por fim, que conhecer a teoria não substitui o julgamento de projeto. A interpolação é uma técnica correta e bem fundamentada na literatura, e foi bem implementada; mesmo assim, para um Snake a 8 passos por segundo, com pixel art e jogabilidade discreta, o atraso visual e a perda de estilo pesaram mais. O diário de bordo, ao registrar a tentativa e a volta, transformou uma reversão em aprendizado.

14.1 TRABALHOS FUTUROS

O próprio backlog do projeto, na seção 8.4 do briefing, aponta caminhos que estenderiam os temas deste ebook: partículas e tremor de tela (animação e efeitos), uma versão para navegador com pygbag (portabilidade do pipeline), um editor de níveis (interação e modelagem) e o remapeamento de teclas (interface e acessibilidade). Do ponto de vista da CG, um exercício interessante seria implementar à mão, sem a SDL, operações que o jogo hoje delega à biblioteca, como a rotação de uma imagem por um ângulo qualquer, com vizinho mais próximo e com interpolação bilinear, ou o operador *over* com alfa pré-multiplicado, e comparar os resultados com os do pygame-ce.

REFERÊNCIAS

ANGEL, E.; SHREINER, D. **Interactive computer graphics**: a top-down approach with WebGL. 8. ed. Hoboken: Pearson, 2020.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023**: informação e documentação: referências: elaboração. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 10520**: informação e documentação: citações em documentos: apresentação. Rio de Janeiro: ABNT, 2023.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2024.

AZEVEDO, E.; CONCI, A. **Computação gráfica**: teoria e prática. Rio de Janeiro: Elsevier, 2003.

BRESENHAM, J. E. Algorithm for computer control of a digital plotter. **IBM Systems Journal**, [s. l.], v. 4, n. 1, p. 25-30, 1965. DOI: <https://doi.org/10.1147/sj.41.0025>.

ERICSON, C. **Real-time collision detection**. San Francisco: Morgan Kaufmann, 2005.

FIEDLER, G. Fix your timestep! **Gaffer On Games**, [s. l.], 10 jun. 2004. Disponível em: https://gafferongames.com/post/fix_your_timestep/. Acesso em: 7 out. 2026.

GAMMA, E. *et al.* **Design patterns**: elements of reusable object-oriented software. Reading: Addison-Wesley, 1994.

HEARN, D.; BAKER, M. P.; CARITHERS, W. R. **Computer graphics with OpenGL**. 4. ed. Boston: Pearson, 2011.

HECKBERT, P. Color image quantization for frame buffer display. **ACM SIGGRAPH Computer Graphics**, New York, v. 16, n. 3, p. 297-307, jul. 1982. DOI: <https://doi.org/10.1145/965145.801294>.

HUGHES, J. F. *et al.* **Computer graphics**: principles and practice. 3. ed. Upper Saddle River: Addison-Wesley, 2014.

KRASNER, G. E.; POPE, S. T. A cookbook for using the model-view controller user interface paradigm in Smalltalk-80. **Journal of Object-Oriented Programming**, [s. l.], v. 1, n. 3, p. 26-49, 1988.

LASSETER, J. Principles of traditional animation applied to 3D computer animation. **ACM SIGGRAPH Computer Graphics**, New York, v. 21, n. 4, p. 35-44, jul. 1987. DOI: <https://doi.org/10.1145/37401.37407>.

MARSCHNER, S.; SHIRLEY, P. **Fundamentals of computer graphics**. 5. ed. Boca Raton: CRC Press, 2021.

NYSTROM, R. **Game programming patterns**. [S. l.]: Genever Benning, 2014. Disponível em: <https://gameprogrammingpatterns.com/>. Acesso em: 7 out. 2026.

PILLOW (PIL Fork) 12.3.0 documentation: Image module. [S. l.]: Read the Docs, [2026?]. Disponível em: <https://pillow.readthedocs.io/en/stable/reference/Image.html>. Acesso em: 7 out. 2026.

PORTER, T.; DUFF, T. Compositing digital images. **ACM SIGGRAPH Computer Graphics**, New York, v. 18, n. 3, p. 253-259, jul. 1984. DOI: <https://doi.org/10.1145/800031.808606>.

PYGAME-CE DEVELOPERS. **pygame-ce documentation**. Versão 2.5.9. [S. l.], c2023. Disponível em: <https://pyga.me/docs/>. Acesso em: 7 out. 2026.

PYGAME-CE DEVELOPERS. **pygame-ce 2.5.8**: código-fonte, arquivo `src_c/display.c`. [S. l.]: GitHub, 2026. Disponível em: https://github.com/pygame-community/pygame-ce/blob/2.5.8/src_c/display.c. Acesso em: 7 out. 2026.

ROADS, C. **The computer music tutorial**. Cambridge: MIT Press, 1996.

SDL. `SDL_HINT_RENDER_SCALE_QUALITY`. In: **SDL2 Wiki**. [S. l.], [20--]. Disponível em: https://wiki.libsdl.org/SDL2/SDL_HINT_RENDER_SCALE_QUALITY. Acesso em: 7 out. 2026.

SMITH, A. R. **A pixel is not a little square, a pixel is not a little square, a pixel is not a little square! (And a voxel is not a little cube)**. Redmond: Microsoft, 1995. (Technical Memo, 6). Disponível em: https://www.alvyray.com/Memos/CG/Microsoft/6_pixel.pdf. Acesso em: 7 out. 2026.

SUTHERLAND, I. E. Sketchpad: a man-machine graphical communication system. In: SPRING JOINT COMPUTER CONFERENCE, 1963, Detroit. **Proceedings** [...]. New York: ACM, 1963. p. 329-346. DOI: <https://doi.org/10.1145/1461551.1461591>.

SUTHERLAND, I. E.; HODGMAN, G. W. Reentrant polygon clipping. **Communications of the ACM**, New York, v. 17, n. 1, p. 32-42, jan. 1974. DOI: <https://doi.org/10.1145/360767.360802>.

GLOSSÁRIO

Acumulador

Variável que guarda o tempo real ainda não consumido pela simulação, no passo de tempo fixo. A cada quadro recebe o Δt e, a cada passo executado, perde um intervalo.

Algoritmo do pintor

Método de visibilidade em que os objetos são desenhados do mais distante para o mais próximo, cada um cobrindo os anteriores.

Alfa (canal)

Quarto canal de uma imagem RGBA, que indica a opacidade (cobertura) de cada pixel: 0 é transparente e o valor máximo é opaco.

Amostragem

Representação de um sinal contínuo (imagem ou som) por valores medidos em pontos ou instantes regulares.

Blit

Cópia de um bloco retangular de pixels de uma superfície para outra (*bit block transfer*), combinando as cores quando há transparência.

Buffer duplo

Técnica em que o quadro é desenhado num buffer invisível e só exibido quando está completo, evitando que o usuário veja o desenho pela metade.

Coordenadas homogêneas

Representação de um ponto do plano (x, y) como $(x, y, 1)$, que permite escrever translações, rotações e escalas como multiplicações de matrizes 3×3 .

Corte pela mediana (*median cut*)

Algoritmo de quantização de cores que divide recursivamente o conjunto de cores da imagem na mediana do eixo de maior variação, de modo que cada cor da paleta represente uma parte igual dos pixels.

Envelope

Variação do volume de um som ao longo do tempo (ataque, sustentação, soltura), que evita estalos e dá forma ao timbre.

Envolvimento de bordas

Regra em que um objeto que sai por uma borda do campo reaparece na borda oposta, como na superfície de um toro.

Fade

Transição em que a imagem surge do preto (ou desaparece no preto) pela variação gradual da opacidade de uma camada.

Framebuffer

Região de memória que guarda a cor de cada pixel da imagem a ser exibida.

Glifo

Desenho de um caractere numa fonte específica.

HUD

Faixa de informações sobreposta ou adjacente à área de jogo (pontos, nível, tempo).

Interpolação linear

Cálculo de um valor intermediário entre dois valores conhecidos, proporcional a um parâmetro entre 0 e 1.

Laço de jogo

Laço principal que, a cada volta, processa a entrada, atualiza o estado e desenha o quadro, controlando a passagem do tempo.

Paleta

Conjunto limitado de cores usado por uma imagem ou por um jogo; na cor indexada, tabela que traduz índices em cores RGB.

Passo de tempo fixo

Técnica em que a simulação avança em intervalos de duração constante, independentes da taxa de quadros do desenho.

Pixel

Menor elemento de uma imagem raster; uma amostra de cor numa posição da grade.

Pixel art

Estilo gráfico em que cada pixel é posicionado deliberadamente e permanece visível como bloco de cor.

Quantização de cores

Redução do número de cores de uma imagem, escolhendo uma paleta e mapeando cada cor original para a mais próxima dela.

Raster

Representação de uma imagem como matriz retangular de pixels.

Recorte (*clipping*)

Eliminação das partes de uma primitiva ou imagem que ficam fora de uma região, normalmente a área visível.

Sprite

Imagem pequena, normalmente com transparência, desenhada sobre o fundo para representar um objeto do jogo.

Superfície (pygame.Surface)

Objeto do pygame-ce que representa uma imagem raster com resolução e formato de pixel fixos.

Taxa de quadros

Número de imagens desenhadas por segundo (FPS).

Transformação afim

Transformação geométrica que combina uma transformação linear (rotação, escala) com uma translação.

Vizinho mais próximo

Método de reamostragem em que cada pixel novo copia a cor da amostra original mais próxima, sem criar cores intermediárias.

APÊNDICE A – EXPERIMENTE VOCÊ MESMO

Os exercícios abaixo usam o próprio repositório do jogo. Para prepará-lo, siga a seção "Rodar pelo código-fonte" do README: clonar o repositório, criar um ambiente virtual e instalar com `pip install -e ".[dev]"`. Antes de cada exercício, crie uma branch (`git switch -c estudo/nome`), e ao final volte à `main` e apague-a. Assim nenhuma mudança se perde nem fica para trás.

CAPÍTULOS 3 E 4: RASTER E COORDENADAS

1. Rode o jogo com `python -m cobrinha --debug` e observe o FPS no canto inferior direito. Em seguida, mude `FPS` em `src/cobrinha/config.py` para 20 e jogue de novo. A cobra continua na mesma velocidade? Por quê? (Releia o capítulo 5.)
2. Mude `ALTURA_HUD` para 100 em `config.py`. Quantas linhas de código precisaram mudar para o campo inteiro descer? O que isso diz sobre a função `celula_para_pixel`?
3. Calcule quantos bytes ocupa um quadro do jogo com 4 bytes por pixel e quanto ocuparia em tela cheia numa resolução de 1920×1080 . Por que o jogo continua desenhando em 800×600 mesmo em tela cheia?

CAPÍTULO 5: LAÇO DE JOGO E PASSO FIXO

4. Em `tests/test_partida.py`, escreva um teste que crie uma `Partida`, chame `atualizar(0.124)` e confira que a cobra não andou; depois chame `atualizar(0.002)` e confira que ela andou uma célula. Qual é o intervalo do passo no nível 1?
5. Chame `atualizar(5.0)` (um travamento de 5 s) e conte quantos passos a cobra deu. Relacione o resultado com `MAX_PASSOS_POR_QUADRO`.

CAPÍTULO 6: PIPELINE DE DESENHO

6. Em `EstadoJogando.desenhar`, mova a linha que desenha a cobra para antes do fundo do campo. O que acontece? Explique com o algoritmo do pintor.
7. Comente a linha que desenha os textos flutuantes e coma uma maçã. Depois, recoloque-a antes da cobra. Em qual posição o "+1" fica mais legível?

CAPÍTULOS 7 E 8: PIXEL ART E TRANSFORMAÇÕES

8. Mude `ESPESSURA = 17` em `ferramentas/gerar_sprites.py` para 13, regenere os sprites e jogue. A curva continua se encaixando no corpo reto? Por quê?
9. Na função `comida`, troque `raio * 0.9` por `raio * 0.3` na condição da sombra. Regenere e compare. Que reta define a sombra agora?
10. Num terminal Python, gire um sprite em 45° com `pygame.transform.rotate` e compare o tamanho com o original. Salve o resultado e examine as bordas ampliadas.

CAPÍTULOS 9 E 10: COR E COMPOSIÇÃO

11. Abra `docs/ebook/figuras/captura-jogando.png` com a Pillow e quantize-a para 16 cores com e sem pontilhado (`dither`). Compare os resultados com uma lupa.
12. Calcule a cor de um pixel da grama escura (124, 178, 66) sob o véu azul da câmera lenta e confira com a captura `captura-camera-lenta.png`, no pixel (35, 70).
13. Mude `OPACIDADE_VEU_LENTO` em `src/cobrinha/estados/jogando.py` para 120. O efeito fica mais claro? O campo continua legível?

CAPÍTULO 11: ANIMAÇÃO

14. Troque `round(math.sin(...))` por `math.sin(...) * 3` em `_desenhar_flutuando`. O que acontece com a nitidez da maçã? (Dica: o `blit` aceita posições fracionárias?)
15. Mude a curva de opacidade dos textos flutuantes de `1 - progresso**2` para `1 - progresso`. Qual das duas deixa o "+1" mais legível?
16. Recupere a versão com movimento interpolado (`git switch --detach ddb474b`) e jogue no nível 3. Anote o momento em que a maçã some e onde está a cabeça desenhada nesse instante.

CAPÍTULOS 12 E 13: COLISÃO, INTERFACE E SOM

17. Escreva um teste para a regra da cauda segura em `Cobra.colidiria`, com e sem crescimento pendente.

18. No modo Sem bordas, saia pela borda de cima. Qual linha a cabeça ocupa no passo seguinte? Confira com `Grade.envolver`.
19. Em `ferramentas/gerar_sons.py`, troque a onda do bipe da contagem e regenere os sons. Descreva a diferença de timbre.

APÊNDICE B – COMO ESTE EBOOK FOI GERADO

Como os sprites e os sons do jogo, este ebook é gerado por código. O texto-fonte, os scripts e as figuras ficam na pasta `docs/ebook/` do repositório, e qualquer pessoa pode regenerar o PDF a partir deles.

ESTRUTURA DA PASTA

Caminho	Conteúdo
<code>texto/</code>	Capítulos e demais seções em Markdown
<code>figuras/</code>	Figuras geradas (PNG)
<code>figuras.py</code>	Fluxogramas, diagramas, gráficos, folha de sprites e paleta (só Pillow)
<code>capturas.py</code>	Capturas de tela e figuras que usam o pygame-ce (rotação, composição alfa e quantização)
<code>gerar_ebook.py</code>	Montagem do HTML e impressão do PDF
<code>estilo.css</code>	Apresentação conforme a ABNT
<code>metadados.toml</code>	Dados da capa e da folha de rosto

COMO GERAR

Com o ambiente virtual do projeto ativo, instale as dependências do ebook e rode o script:

```
pip install -e ".[ebook]"
python docs/ebook/gerar_ebook.py
```

O script regenera todas as figuras e capturas e grava o PDF em `docs/ebook/ebook-computacao-grafica-cobrinha.pdf`. Para gerar só o PDF, reaproveitando as figuras, use a opção `--so-pdf`. A impressão usa um navegador baseado no Chromium (Microsoft Edge ou Google Chrome) sem janela; o caminho pode ser indicado pela variável de ambiente `EBOOK_NAVEGADOR`.

COMO FUNCIONA

Figuras. Os fluxogramas são desenhados por um pequeno motor escrito sobre a Pillow, com nós (terminal, processo, decisão) e setas ortogonais posicionados por coordenadas no próprio código. O texto usa a fonte VT323 do jogo, e o script falha se algum caractere não tiver desenho nessa fonte. As capturas de tela rodam o jogo de verdade com o driver de vídeo `dummy` da SDL, como

ferramentas/gravar_demo.py, reaproveitando o piloto automático desse script e uma semente fixa: as imagens saem sempre iguais.

Trechos de código. Os capítulos não contêm código copiado à mão. Um bloco como `::: codigo #executar src/cobrinha/jogo.py 92-105` é substituído, na montagem, pelas linhas 92 a 105 do arquivo real, com os números de linha. Trechos de versões antigas, como os do movimento interpolado, são lidos do histórico com `git show`, a partir de um prefixo de commit (por exemplo, `e22164c:src/cobrinha/ui/pecas.py`). Se um arquivo mudar e o intervalo deixar de existir, a montagem falha em vez de publicar um trecho errado.

Numeração e sumário. Seções, figuras, quadros e códigos são numerados automaticamente, e as referências cruzadas no texto (`@fig:nome`, `@qua:nome`, `@cod:nome`) viram "Figura 3", "Quadro 1" etc. O PDF é impresso uma primeira vez; o script lê os marcadores (*outline*) do PDF para descobrir em que página caiu cada título e cada legenda, preenche o sumário e as listas e imprime de novo, até a paginação se estabilizar.

NORMAS SEGUIDAS

A apresentação segue a NBR 14724 (Associação Brasileira de Normas Técnicas, 2024): papel A4; margens de 3 cm à esquerda e em cima e de 2 cm à direita e embaixo; fonte tamanho 12 no texto e menor nas citações longas, legendas e fontes das ilustrações; espaçamento de 1,5 no texto; páginas contadas a partir da folha de rosto e numeradas a partir da primeira folha textual, no canto superior direito; e legendas das ilustrações na parte superior, com a fonte embaixo. As citações seguem a NBR 10520 (Associação Brasileira de Normas Técnicas, 2023), no sistema autor-data, e as referências seguem a NBR 6023 (Associação Brasileira de Normas Técnicas, 2018).